

BRIDGE

Documentation of the creation of a trick-taking card game by Alfie William Pearce



The Perfect Game of Bridge – From (Downey, 2025)

DATE

THE SIXTH FORM COLLEGE
Colchester

Contents

Analysis	6
Introduction	6
Stakeholders	6
Research.....	7
Rules of Bridge	7
Game Play	7
Scoring.....	8
Game Variants and Why I Will Make Generic Rubber Bridge.....	9
Online Implementations	10
Fun Bridge (FunBridge, 2025)	10
Bridge Base ONLINE (Bridge Base On Line).....	11
Cardgames.io BRIDGE (Rauðás Games)	12
247 BRIDGE (24/7 Games LLC)	13
Discussions with Kade: Designing the Stakeholder-Required Features.....	14
Project Specification	15
Project Features	15
Moscow Analysis.....	16
Limitations.....	18
Hardware & Software Requirements.....	18
Design.....	19
Project Decomposition.....	19
User Interface	20
Menu	20
Core Display	21
Bidding	23
Scores	25
Data Storage.....	26
Internal Storage	26
External Storage	27
Validation	27
Algorithms.....	29
Server-Side	29
Client-Side	32
Application Protocol.....	34
Development.....	35
Iteration Plan.....	35
Proofs of Concept.....	36
Iteration 1 – Heuristic ‘AI’	36

Iteration 2 - Local network chat-app.....	37
Iteration 2.1 - Lobbies and Concurrent Games.....	39
Iteration 2.2.1 - Databases.....	40
Iteration 2.2.2 – Sanitisation.....	40
Iteration 2.3 – Turn-Taking	41
Iteration 2.4 - WAN	42
Iteration 3.1.1 - Card art	43
Iteration 3.1.5 - Rendering cards	44
Iteration 3.2 - Layout of cards.....	45
Iteration 3.3 – Animation.....	46
Iteration 3.4 - Graphical enhancement on clicks	46
Iteration 4 - Game Logic in a Finite State Machine.....	47
Actual Iterations.....	49
Iteration 1 - A deck of Cards – M1	49
Iteration 2 – The Game States – M2, M3, M4, M5	51
Iteration 2.5 – Random Robot Opponents – M6	54
Iteration 3 – Node.js – S2 & S3	55
Iteration 4 – Advanced Opponents – S1	57
Iteration 5 – Multiplayer – S3	63
Iteration 6 – GFX & SFX – M4 & S4 & S5	65
Iteration 6.2 – Audio – S5.....	77
Iteration 7 - WAN	79
Iteration 8 – Clean-up	80
Testing.....	81
C1 – Lobby Setup & C2A – Private Lobbies	81
M1 – Dealing	81
M2 – Bidding	81
M3 - Trick-Taking.....	82
M4 – Scorecard & M5 - Games & Rubbers	82
M6 – Random Opponents & S1 – Heuristic `AI`	82
S2 – Server & S3 - Client.....	82
S4 – Audio	83
S5 – Animation	83
C2B – Public Lobby.....	83
C3 – Tutorial	83
C4 – User Profiles	83
C5 – Friend List & Quick Invite	83
C6 – Rubber Records.....	83
W1 – AR/VR.....	83

W2 – AI	84
W3 – Variants.....	84
Stakeholder Testing	85
User Interface.....	86
Game Play	86
Summary	86
Stakeholder Results.....	87
User Interface.....	87
Game Play	87
Summary	87
Evaluation	88
Success Criteria	88
Essential Gameplay – M1/2/3/4/5/6	88
Lobbies – S2/3 C1/2A/2B.....	88
Smart Opponents – M6 S1	88
Server & Client – S2/3	88
UI & UX – M4 S4/5.....	89
User Profiles – C4/5/6.....	89
Tutorial & Help – C3	89
Complex Additional Features – W1/2/3	89
Assessment of Usability Features	90
Robustness.....	91
Limitations.....	91
Maintenance Issues	91
Future Developments	92
Bibliography	93
Appendices.....	94
Final Hierarchy – scripts and files	94
node_modules/	94
.prettierrc.....	94
Eslint.config.js	94
Package.json.....	95
Package-lock.json.....	96
Server.js.....	97
Src/lobbies.js.....	111
Src/player.js	111
Src/game.js.....	122
Src/bidding.js	131
Src/trick.js	135

Src/score.js.....	138
Public/index.html.....	144
Public/client.js – due to my chrome book’s clipboard some formatting doesn’t work	146
Public/drawGame.js	166
Public/buttons.js	188
Public/cards.js	190
Public/Art/favicon.png	192
Public/Art/Sprite_Sheet.png	192
Public/Audio/Music.wav	193
Public/Audio/Card.mp3	193
Public/Audio/Button.mp3.....	193

Analysis

Introduction

For my project, I am going to create a digital version of the card game Rubber Bridge, a variation of the well-known game Contract Bridge. Bridge is a 4-player trick-taking card game played in teams of 2, where players bid to determine a contract and then take turns playing cards to win tricks – rounds where the highest card of the leading suit (or a trump suit) wins. The game rewards bidding strategy, memory, and probability-based decision making.

Bridge evolved from games like Whist and Biritch, with the modern version solidified by Harold S. Vanderbilt in 1925 through innovations in scoring and rules. While the historical background is not central to this implementation, it contextualises the game's lasting popularity and complexity (American Contract Bridge League, 2014)

At its core, the system requires: A standard 52-card deck, 4 players with individual hands, logic for shuffling and dealing decks of cards, and the taking of tricks, scoring, and bidding. These can be cleanly modelled using OOP. Cards, players, and game instances can be encapsulated as classes, which improves modularity and reduces redundancy, improving the overall maintainability of the code.

To allow multiplayer interaction, the system will use a client-server architecture. The server will manage player accounts, game instances, scoring and card distribution. The client will handle the visual presentation, showing each player only the information, they are meant to see. Communications between client and server will be handled via Socket.io.

To simulate opponents when fewer than 4 human players are present, I will implement heuristic-based AI agents. However, these won't use machine-learning algorithms, so technically they aren't AI, but they will use rule-based systems using probabilistic logic and possibly even bidding conventions. The predictability of the rules makes this feasible with pre-defined tactical models and decision trees.

The main scripting language used will be JS due to its versatility and its capacity to make web applications. Graphical functions will be using the p5.js graphics library for displaying the game, alongside HTML5 and CSS for controlling the viewport – removing margins, setting metadata and linking multiple JS scripts together.

Stakeholders

The key stakeholder for this project is my friend Kade, who originally proposed the idea. She and I frequently play Bridge at lunch with friends and wants a digital version to play casually outside of school. As a programmer herself, she will contribute to early testing, offer technical feedback, and evaluate the interface from both a player's and developer's perspective. The solution benefits her by allowing solo or online play when meeting in person isn't possible.

My friendship group also has a stake in the project. Several of them are Computer Science students who can test the system for bugs and evaluate its logic. Others are casual players, who can give feedback on the UI/UX. The online multiplayer feature also supports their need to play Bridge together even when they aren't physically in the same place.

My family has supported my past game projects and will again provide testing and feedback. They're interested in learning Rubber Bridge and find current digital options lacking – either being computer-only or requiring 4 players – overall being hard to learn through. This version will allow them to learn gradually by mixing human and computer-controlled opponents, making the game more accessible and educational.

Finally, I am a stakeholder. This coursework counts for 20% of my A-Level grade, and I'm aiming for top marks to pursue a future in IT. This project will help me strengthen my programming skills, particularly in networking and real-time applications, and will also give me experience managing stakeholders and implementing a system from concept to completion.

Research

My research is divided into 3 key parts:

1. [Find out the rules of Bridge in detail, specifically Rubber Bridge and not Duplicate Bridge](#)
2. [Explore and evaluate various existing implementations](#)
3. [Consulting with Kade and other stakeholders and find out exactly how she believes the game should be \(expectations and goals\)](#)

Once these parts are researched, I will be able to clearly define requirements for this project.

Rules of Bridge

Game Play

Bridge is a four-player card game played in two fixed teams of two, using a standard 52-card deck. The main goal is to win points through successful bidding and trick-taking. Gameplay is traditionally divided into two distinct phases: bidding (the auction) and card play (trick-taking).

Bridge is a game with fundamentally simple rules that become progressively complicated. Some key terminology for this game is:

- Players sit opposite their teammate (forming a pair) – the players are labelled North, East, South, West.
- If the pair wins the bid, then whoever placed the bid is the declarer, and will play their teammate's (the dummy's) hand.
- The other pair are called the defender.
- High Cards have a ranking system (HCP) so Ace:4, king:3, queen:2, jack:1 – better hands have higher HCP

(The rest of the gameplay research will be as accordance to (GOTO Games, 2025))

Bidding

The dealer makes the first call (opener) then the bidding proceeds clockwise. A bid contains:

1. A number from 1-7 (level)
2. A trump suit (spades, hearts, diamonds, clubs) or notrump

The level refers to number of tricks (six plus the level) a pair has contracted to make. If a player thinks that the last bid made by an opponent is optimistic, they can double it, and this can be redoubled by an opponent. The next player (clockwise) will have 3 options:

- Pass
- Double or Redouble
- Or make a new bid

The bidding ends when 3 subsequent players say pass, and so the final bid made becomes the contract.

Card Play

Player to the left of declarer starts the play by making the opening lead. After this the dummy places their 13 cards face-up on the table. After this the players (clockwise) take turns playing their cards (1 each). They have three options:

- Play a card in the current suit (if possible, then must) but if they have no cards of said suit then:
- Play a card in the trump suit (would net the most points)
- Discard a card (place any)

Whoever plays the highest card in the suit or, better, trump suit wins the trick. A play consists of 13 tricks – playing all cards.

Scoring

Scoring in Rubber Bridge is complex, involving both strategic depth and careful calculations. According to (No Fear Bridge, 2013), scoring is fundamentally based on whether the contract is fulfilled by the declarer or successfully stopped by the defenders. For obtaining a positive score you must win at least as many tricks as your contract states. And for a defender to obtain positive points they must prevent the declarer from taking the number of tricks they are contracted for. On the right is a generic scorecard. In Rubber Bridge only tricks bid and made are scored below the line and count towards the game, everything else is above the line. There are no bonuses scored for making games or part scores until the end of the rubber.

A sample Rubber Bridge scorecard (see right) separates points into two distinct categories for the two different pairs “We” and “They”:

- Below the line – these points contribute toward winning a game – successful tricks
- Above the line – includes all other points from overtricks to bonuses and penalties

To score positively the declarer’s team must take at least as many tricks as their bid committed them to. If they succeed, they earn contract points below the line, so contributing toward a game. Once a pair reaches 100 points below the line, they win the game, and a line is drawn across both teams. Two games won results in the Rubber being concluded.

Bonuses for the winning rubber	Bonus points achieved
Win 2-0 (straight)	700
Win 2-1 (comeback)	500

Bridge is a rather long card game, and as such, very often, rubbers will have to end early. There is a 300-point bonus for a game in an unfinished rubber and 50 for a part score in an unfinished rubber.

There are two other types of bonus points:

- The Insult bonus, where 50 or 100 points are added if the contract is doubled or redoubled respectively. – This is added above the line.
- The Slam bonus, where 500 or 1,000 points are added above the line if the contract made is of level 6 or 7 respectively. This is 750 or 1,500 if the pair is vulnerable

A pair is vulnerable after they win a game.

The scores in both columns are totalled at the end of the rubber, and the winner is the one with the highest total score at the end of the rubber.

Trick Value by Contract Type

According to (The Bridge World Magazine, 2025) trick values vary by contract:

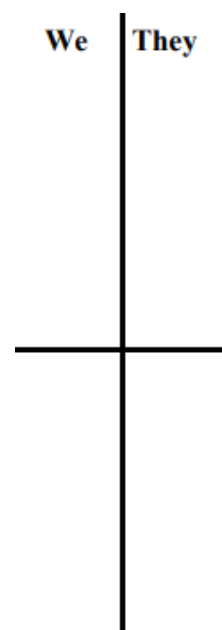
Contract Type	Points per trick obtained
Notrump (NT)	40 for first trick, 30 for subsequent tricks
Major suits (♥, ♠)	30
Minor suits (♦, ♣)	20

Examples:

- 2♣ bid, 8 tricks taken – contract fulfilled: 40 points below the line
- 1♥ bid, 9 tricks taken – contract fulfilled with 2 overtricks: 30 points below and 60 above the line
- 3♠ bid, 7 tricks taken – contract failed: no points for declarer and defenders earn penalty points

Undertrick Penalty Points

If the declarer fails to make a contract, defenders earn penalty points above the line based on the number of undertricks and whether the declaring team is vulnerable or not. The (American Contract Bridge League, 2014)



provides a clear table for showing undertrick penalties. These values increase significantly if the declarer's side is vulnerable.

Undertricks	Points per undertrick					
	Vulnerable			Not vulnerable		
	Undoubled	Doubled	Redoubled	Undoubled	Doubled	Redoubled
1st undertrick	100	200	400	50	100	200
2nd and 3rd, each		300	600		200	400
4th and each subsequent		300	600		300	600

Relevance to My Program

Understanding the intricacies behind scoring in Bridge is crucial to designing the logic in my implementation. My program will have to:

- Track points above and below the line accurately per hand for each pair
- Maintain vulnerability status
- Implement rubber win conditions and related bonuses
- Calculate penalty points correctly probably using an array to store penalties

This detailed scoring logic is second only to the multiplayer and computerised enemies in being the most algorithmically complex parts of this project.

Game Variants and Why I Will Make Generic Rubber Bridge

There are 2 primary variations of Bridge: Rubber & Duplicate, both catered to different styles of play.

According to (The Bridge World Magazine, 2025), Duplicate Bridge is especially popular in tournament settings. In this version, identical hands are dealt to different tables of players, removing the element of chance from the shuffle and placing a sharper focus on strategy and decision-making.

Rubber Bridge, by contrast, is more casual and variable – often played socially rather than competitively. It allows for a longer form of play, with pairs building scores across multiple games. There are also many variations of Rubber Bridge:

For example, Nullos Bridge – as described by (shawnomancy , 2025) - introduces the concept of a nullo (nil) bid, where a player contracts not to take any tricks.

Another popular variant is Chicago Bridge, according to (The Official Encyclopedia of Bridge, 2001), where a rubber consists of exactly 4 deals. The duration is likely to be consistent (~20 minutes). It also allows 4-7 players to participate by methods of partnership rotation. On first deal no one is vulnerable, on second and third the dealer is vulnerable, and all are vulnerable on fourth. This makes it more flexible for casual group settings.

Why I Have Chosen to Make Generic Rubber Bridge:

Despite these intriguing variations, I’ve chosen generic Rubber Bridge for this project. This decision is driven by practicality and personal experience: it’s the version I play with Kade and my friends, and it captures the core mechanics of Bridge without introducing excessive complexity – useful for people who want to learn how to play. It will also align with the traditional ruleset, providing a more solid foundation upon which to build my program.

That said, if I had more time, collaborators, and resources, I would consider building a modular system allowing players to toggle between variants – Rubber, Duplicate, or even more niche formats. This would enrich the UX and provide a valuable education tool for those looking to explore the breadth of the game

Online Implementations

After a quick Google search, I found many examples of Bridge online and below are the ones I found most interesting for this project.

Fun Bridge (FunBridge, 2025)

Fun Bridge requires users to create an account before playing. New accounts get 50 free deals and another 50 granted upon email confirmation and then 3 every day, which is clearly aimed at long-term engagement, though this may discourage casual play.

The game's dashboard is visually sleek but heavily feature-packed (no picture as I do not plan to make a dashboard) - solo play, play with a partner, events, news and lessons, which can feel cluttered and overwhelming especially for new players.

I played a game of Mini bridge as this is the most alike what I'll make. And here's what I noticed:

The bidding screen (see right) presents a clean interface. (In this example, which is for learning, North led the open with a 4♠ bid and then everyone passed). South is the player here and 14HCP means that there is 14 High Card Points (HCP). Also during Bidding it shows a simple bid table showing all 35 available bids in an accessible layout, and additional info (see

below) about your teammate, which can help people learn and informs bidding decisions.

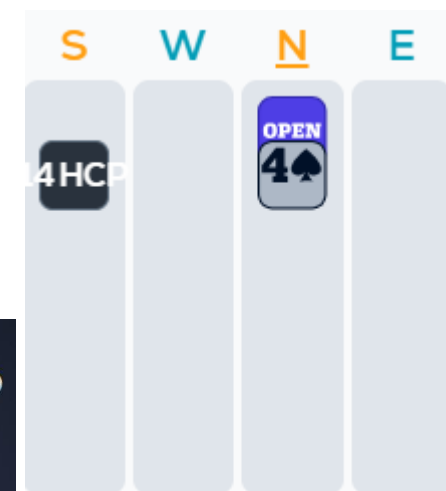
The card play interface (see left) is very clear showing all hands, with a clear central trick layout (see right). The player names are showing prominently which is important to reflecting the social element of Bridge.

A thoughtful quality of life feature shows the player every card in a suit (see example left) when you press on any card of the suit (which can also help the coding checks to make sure the placement is viable). Also, if only 1 card is playable it is auto played, speeding up gameplay.

The hand layout is efficient: the dummy's cards displayed in a tight rectangular format, conserving screen space, whilst the players are naturally fanned.

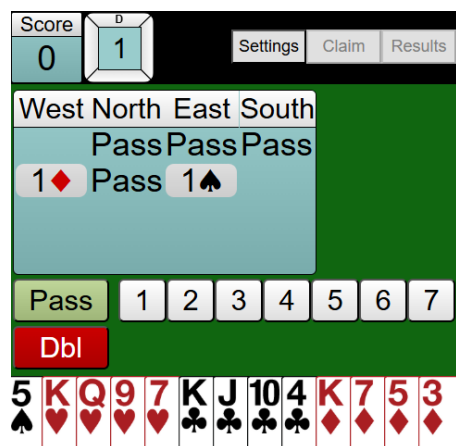
Features to Take Inspiration From: Clear intuitive bid interface and trick display, handy suit filter for ease of play, partner hand breakdown for strategic support, an effective layout that saves space and avoids clutter, and names to add a sense of realism though the avatars are lacklustre and don't feel too realistic.

Drawbacks to Avoid Replicating: The freemium deal system discourages casual play, and overly complex dashboard hinders quick play.



Bridge Base ONLINE (Bridge Base On Line)

Unlike Fun Bridge, Bridge Base Online allows limited access without an account. A leader board (see right) is shown on the right of the screen though takes a lot of space which could cause confusion and takes up much of the screen essential real estate which the problem caused from this is visible below.



Upon entering a game, I was surprised how visually busy the interface (see left) was. The bidding table resembles that of Fun Bridge, but the general layout lacks clarity and space.

Once card play begins, the problem worsens (see left) as the dummy's hand appears awkwardly close to the player's. There is little white space and important elements feel rather cramped.

Compared to Fun Bridge, the social experience is also lacking. Opponent hands aren't visible at all – not even the backs of the cards and there are no names nor avatars which loses the game's social appeal (not very useful for someone wanting a casual experience).

The results screen (see right) shows all players hands, tricks taken, contracts bid and final scores, however the scoring display doesn't clearly show 'above the line' and 'below the line' points making traditional Rubber Scoring harder to follow.

Players online: 9076		
1	tomaszch52	24410
2	vicbro	19670
3	bonnafjm52	16860
4	nehruova9	15720
5	dglucksmann14	15650
6	fred33	15430
7	no_heart1	14510
8	coinerbop1	13780
9	maciej424255	13240

Player Name	GO
-------------	----

#	Result	NS	EW	Score	Tricks	Contract
1	3♠E+3			230		

North	West	East	South
♠ Q932 ♥ A1065 ♦ 108 ♣ 976	♠ A76 ♥ 82 ♦ AQ92 ♣ Q853	♠ KJ1084 ♥ J43 ♦ J64 ♣ A2	♠ 5 ♥ KQ97 ♦ K753 ♣ KJ104

Features to Take Inspiration From: A visible deal counter showing the current deal in the rubber – useful for tracking game progression, a history display showing previous deals, contracts, results, and hands – so could be implemented as a timeline, the idea of showing all hands post-deal is great for reviewing play, especially in a learning context, I also learnt that dummy hands should be rotated (not stacked or mirrored) to improve visual clarity during play.

Drawbacks to Avoid Replicating: The interface is cluttered and disorienting – spacing and layout needs clear organisation for beginners, overlapping or adjacent hand layouts (dummy vs player) creates confusion, the lack of visual presence for opponents reduces immersion and the crucial social feel for casual play, and scoring needs to clearly distinguish 'above' and 'below the line' to aid those that are used to playing or want to learn Rubber Bridge Conventions.

Cardgames.io BRIDGE (Rauðás Games)

Cardgames.io is home to a large variety of casual card games including a simple yet functional Bridge implementation.

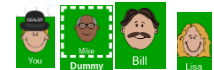
This version stands out for its minimalist UI (see right), which may lack polish but more than makes up for it in clarity and accessibility unlike Bridge Base Online. The simplicity makes it extremely beginner friendly, offering a clean experience with minimal distractions. The interface includes avatars and names, and clearly marks the dealer, dummy and your position, all of which help to make the game feel more social – like you are playing against real people on an actual table.

A small yellow info box (see below) sits neatly on the screen, indicating whose turn it is or other context-based cues - an elegant learning aid I haven't considered before.

0/5 3/9

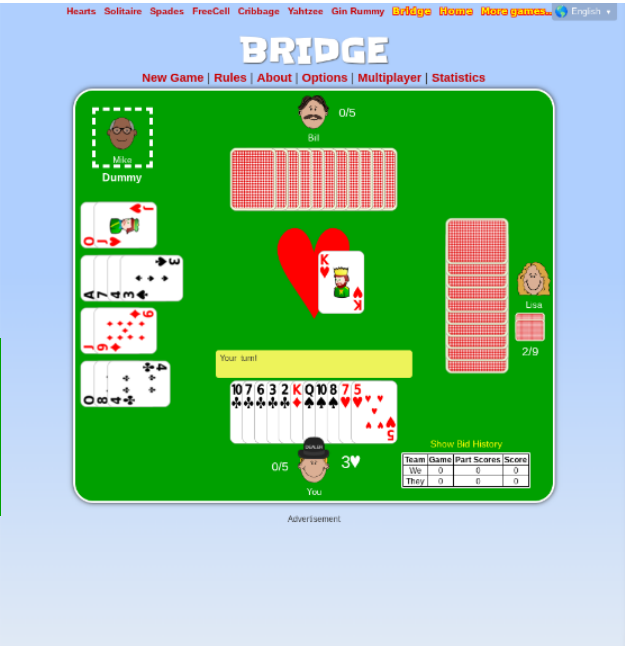
Your turn!

There's also a counter of tricks won per pair in comparison to how many they need to earn points on the deal (see above) and a bid history (see right) - a scoresheet - which is useful and unintrusive – ideal for keeping plays informed but not overwhelming them. The only major drawback is that the game doesn't offer guidance on which cards can legally be played during a turn, which could be confusing to newcomers.



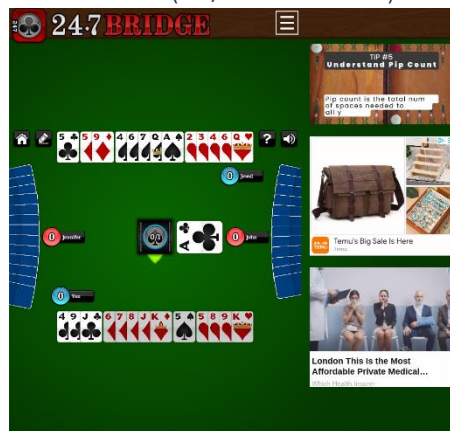
Show Bid History

Team	Game	Part Scores	Score
We	0	0	0
They	0	0	0



Features to Take Inspiration From: Avatars and names make the game feel personal and immersive, clear indication of player roles enhances situational awareness alongside the yellow information box and minimally invasive score card, and the overall layout makes all hands visible also improving on transparency and realism. It is also very important to have clarity in the UI and this implementation excels at that.

Drawbacks to Avoid Replicating: No visual guidance for legal plays could discourage beginners or cause errors, the simplicity is great but could feel too bare for most players especially people knowledgeable of the game, the lack of sounds may also make the gameplay feel sterile with long sessions, which Rubber Bridge often causes.



This implementation offers a clean and relatively beginner-friendly UI (see left) – visually simple and straightforward like that of Fun Bridge, but unfortunately this is undermined by a swarm of intrusive adverts (see right). Declining cookies appeared to worsen the experience, with one ad even blocked by my browser, likely due to unsafe or intrusive content which happens when using 3rd party disreputable ad-networks. These ads are not just a distraction; they risk pushing players away entirely by making the interface feel cluttered and unsafe.

That said there are several strong design choices here. One standout is the neat fan-out animation when cards are dealt from the middle of the screen, giving a physical, tactile feel to gameplay. The bidding table (see right) is also commendably easy to read, presented in a clear and digestible format.

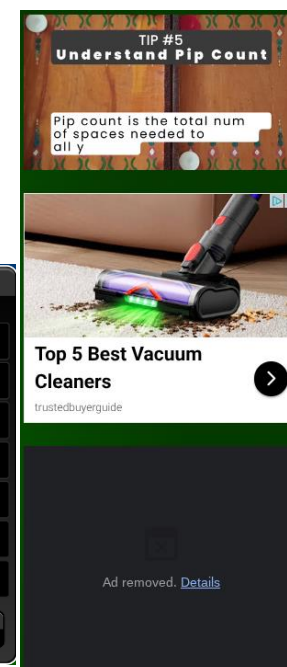
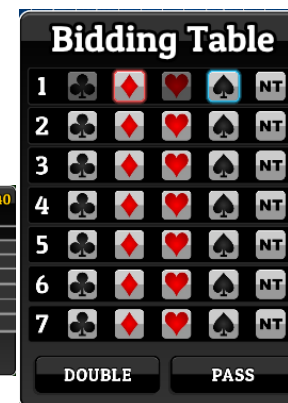


Scrolling the page reveals helpful guides (example on left) on how to play including screenshots (could be done simply in html), which are excellent for helping new players. The score table (see right) – shown after three deals – is elegant and effective. I particularly liked how it isolates penalties in a separate row labelled 'Undertricks', making the breakdown of points more



understandable. The scoreboard button (see left) opens a more detailed view without disrupting play, which is an especially useful feature.

3/2 You & Jewel Made Contract +40		
Game	You & Jewel	Jennaife & John
1	+40	0
2	0	0
3	0	0
Overtricks	+20	0
Undertricks	50	200
Total Points	110	200



In the centre of the screen (see left), an animation section indicates whose turn it is, whether the required six tricks have been reached and the current contract status. This is an effective way to bring motion to an otherwise static screen and keeps the player always informed.

Although the game uses procedural names (see left) (likely selected from an internal array), it lacks avatars, which could distract slightly from the feeling of playing against real people. Still, the inclusion of simple card sounds, and button feedback enhances immersion and provides welcome audio cues during gameplay.



Features to Take Inspiration From: Animations – e.g. dealing out cards or playing them offers a sense of realism and enhances user engagement, a simple intuitive UI supports learning and casual play, the animated turn indicator adds life to the screen and improves clarity, the name displays help personify computer players, the score breakdown – with penalties separated and totals updated round-by-round - is especially well executed, and a toggle button for more detailed scoring is a strong quality-of-life feature.

Drawbacks to Avoid Replicating: Aggressive advertisements ruin immersion and could deter players completely – if monetisation is ever considered, it must be non-intrusive and from a reputable source, no avatars make opponents feel flat or robotic – adding simple visuals could greatly improve this, and the UI while clear might feel sterile without stronger personality or theme.

Discussions with Kade and other stakeholders: Designing the Stakeholder-Required Features

To ensure the final system feels intuitive, useful and engaging, I played a few games of Rubber Bridge with Kade and some friends, whilst discussing what features would matter most in a digital version of the game. Their feedback, shaped by how we play Bridge socially helped to form a practical vision for this project. From that conversation, several key feature categories emerged.

1. Core Gameplay Mechanics

- Bidding - needs to feel natural and clear. Kade suggested a simple, table-based interface, avoiding complex menus or dropdowns. The goal is to let players focus on strategy, not navigation.
- Card Play - should be assisted but not automatic. Players want the interface to show what they can play but not to play for them.
- Scoring - should be persistent and visual. Kade emphasised how useful it would be to always see the score table with a breakdown at the end of each deal.

2. Multiplayer and Computer Opponents

- Lobbies - players can invite friends and empty seats are filled with computer-controlled players
- Difficulty - she asked for the ability to adjust bot difficulty, allowing players to either learn from this game or have a challenge with it
- Dummy & Computer - if you are the dummy kade suggested the idea of swapping the hands of the declarer (your teammate) and you so you play their cards

These features raise design challenges: synchronising multiplayer connections, managing a heuristic (AI-Esque) logic, and dynamic control of hands. But this is crucial for replicating the social side of Bridge which is very important to this project.

3. Personalisation and Visuals

- Names and Avatars - every player should have names and avatars, even computer opponents.
- Variations - while the game focuses heavily on Rubber Bridge it was suggested to be able to play variants of Rubber Bridge like Pirate and Chicago Bridge.
- Exit - games of Rubber Bridge are naturally long and so there needs to be a requirement to end a game early (conceding)

Reflecting on Scope

Not all these features will likely make it into the first version – some are stretch goals depending on resources. For example, computer difficulty levels or variants. However, this discussion with Kade was instrumental in making sure my project feels real and relevant, and highlighted areas I would have otherwise overlooked, for example the difficulty of enemies and the avatar and name system making it feel like an actually immersive game.

Project Specification

Project Features

The following sections outline all desired features for an advanced, immersive and educational online version of Rubber Bridge. These are derived from the above research and enriched by personal insights from gameplay and UX testing.

Fundamentals of Gameplay

- Fully functional Rubber Bridge ruleset so bidding, declarer/dummy play, trick taking, contract enforcement, scoring above and below the line.
- Playable with 1-4 humans and computerised players so flexible lobbies with bot support to allow play when 4 people aren't available.
- Basic game setup options so changing number of deals, target score, player limits, game code invites, visuals, variants of Bridge, from Chicago and Pirate to Duplicate Bridge.
- Players in multiple simultaneous games on server so architecture to support several games running concurrently.

UI & Visual Design

- Tabletop-inspired card layout so clear indication of dummy, dealer and compass style layout.
- High contrast and minimalist UI so prioritising readability and clarity.
- Dynamic trick taker a method to show the current trick and who won it.
- Bid history panel so to view all past bids in a collapsible sidebar or hover tool.
- Scoreboard visibility so a toggleable score panel showing current rubber with clear separation.
- Information box to aid with contextual game info to help learning the game.
- Procedural (in an array) player names and avatars to add personality and realism to the game
- Light / Dark mode and seasonal themes to increase enjoyment and not every game to feel the same.

Animation & Audio

- Smooth, snappy animations, so card dealing, fanning, flipping, trick movements to improve immersion.
- Animated indicators to clearly show whose turn it is – could be gentle glowing borders or pulsing avatars.
- Sounds effects for immersion should be subtle but can make the game more realistic and satisfying.
- Toggleable background music to avoid long games becoming sterile

Computerised opponents & Assistance

- Varying difficulties for computerised opponents so with different play styles to aid those earning the game.
- In-game hints and coaching mode so toggle hints that suggest optimal plays or points out errors.
- Post-game analysis to show an overview of bids, plays and tricks and what should have been done.
- Undo/redo if playing in solo mode letting a user play or replay a deal to explore alternative decisions.

Learning & Accessibility

- Interactive tutorial so a step-by-step walkthrough of game rules, terminology, conventions, and phases.
- Glossary tooltips so to aid knowledge when hovering over a term like HCP.
- Symbol explanation & accessibility modes optional visual cues and aids for people who have visual difficulties maybe the ability to disable animations and reduce transparency.

Online & Social Features

- Private / public lobbies with invite codes o able to join friends or play against public lobbies.
- Player profiles to store names, stats and win/loss history.
- Friend system to add players and invite them faster.
- Live leader board system local and global rankings with filters.
- Integrated chat system to chat, mute profanity and report, as people might want to talk to each other.

Backend & Data Management

- Persistent user accounts so email-based account system storing game states and preferences.

- Match history logs to view full past saved games, bids, results and trick breakdowns.
- Scalable server architecture to support a growing user base.
- Privacy and security settings so options to hide stats and opt out of leader boards.

Stretch Features (for if resources were truly infinite)

- AR integration to play around a real table
- AI opponent learning so bots improve based on your style of play
- Spectator mode to watch friends' matches.
- Achievement system so trophies for milestones (perfect Rubber and Bid slam)
- Cross-platform play to play seamlessly across desktop, tablet, even phone, console, or smart fridge.

Moscow Analysis

Some of these features are minimum requirements (MUSTs) or with ID of M. and to make clear how necessary certain features are, so I don't waste my time and can maximise my chance of success I have made a MOSCOW graph below, where I split the features into MUSTs, SHOULDs, COULDS, WON'Ts.

- SHOULDs are features that will significantly help the game.
- COULDS are features that will be added if I have time.
- WON'Ts are features where I believe I lack the resources to complete – though could be incorporated in a later version.

ID	Feature	Success Criteria	Measured By
MUST			
M1	Dealing	The system must create and shuffle a standard 52-card deck, then deal 13 cards to each player, in a completely fair method so no one gets an unfair advantage. (Probably using a Fisher-Yate's shuffle)	Every player receives 13 fair cards
M2	Bidding	There must be a bidding phase in which players must be able to bid, pass, double, or redouble culminating in a final contract.	Functional UI allowing legal bid inputs & creation of contract
M3	Trick-Taking	Players take turns playing cards, following Bridge conventions, so unable to place illegal cards. After each 4-card trick a winner is identified and leads the next trick. After all the 13 tricks are played, the scorecard is updated and shown.	Turns rotate, only legal cards can be placed, winner of deal is determined
M4	Scorecard	There must be a scorecard UI displaying current scores: overtricks, bonuses, undertricks, and separating above / below the line scores. This needs to appear at the end of a deal at least.	Live-updating, readable scorecard after each deal
M5	Games & Rubbers	A game ends when a pair reaches 100 points below-the-line (which is marked on the scorecard most likely via a line across the below-line area), and the rubber ends when a pair wins 2 games, in which the winner of said rubber is whomever has highest total score.	The system recognises and displays when games / rubbers are won
M6	Random Opponents	If real players are unavailable, the game needs to use up to 3 automated computerised players. At first these bots will make random but legal moves (so without knowing anything – no heuristics).	Opponent turns occur automatically and legally.
SHOULD			
S1	Heuristic 'AI'	Computer players should follow logical Bridge rules (not just randomness), making legal and strategic plays, based on maths. This is an upgrade to the random bots that will make smart plays with customisable difficulty	Players in the same game share a consistent state

S2	Server	The server should control core logic (e.g., dealing, scoring and handling hidden information), enabling synchronous multiplayer games. This should allow multiple people to play in the same game.	Bot players behave in a semi-intelligent, strategic way
S3	Client	The client should provide communicate with the server allowing people, from across the world, to access the same game if they have internet. Each user will have their own view with ability to see their hand but preventing access to hidden details such as opponents' cards.	UI supports full interactivity, secrecy & WAN / LAN access
S4	Audio	UI should include immersive audio: sound effects for card movement, input feedback There should also be background music to fill the empty time and make the game not feel like a drag like it often does.	Toggle-able Audio events play at appropriate moments
S5	Animation	Basic animations (e.g., card fanning, moving, flipping & dealing & showing / hiding UI) should make gameplay visually engaging and more immersive.	Events cause visual transitions
COULD			
C1	Lobby Setup	Players may create their own game lobbies. These will need to have configurable options, such as bot difficulty, UI themes & privacy.	Ability to create a lobby and access a setup menu
C2A	Private Lobby via Code	The host of a lobby needs to be able to generate a unique join code that can be sent to other players that can use it to enter the lobby.	Generate unique code and ability to join via said code
C2B	Public Lobby List	Public lobbies appear in a real-time list and players can browse and join if slots are available.	Lobby menu shows all open public games, ability to join.
C3	Tutorial	A toggle-able in-game tutorial guides new users through rules and the UI step-by-step.	A multi-step tutorial appears when enabled.
C4	User profiles	Users should be able to log in using Repl.it accounts, set a username and avatar that appear in-game, and these profiles are stored permanently in a database (maybe SQLite) or using Repl accounts.	Profiles persist, usernames and avatars visible in-game
C5	Friend List & Quick Invite	A friend's system could allow players to save and directly invite friends which could speed up pre-match times	Players can add / remove friends and send direct invites.
C6	Rubber records	Player profiles could store win and loss records, used as a stat that could make a leader board. Matchmaking could also be based on these records.	Stats persist across sessions and display on profile screens.
WON'T			
W1	AR / VR	While immersive, AR / VR support exceeds my scope due to limitation in hardware access, time constraints, and interface complexity. These technologies require specialist equipment and significant development overhead, so will not be implemented.	
W2	AI opponents	Adaptive AI opponents would require data training, reinforcement learning models, and complex decision logic – this would be a significant project on its own. Furthermore, a Rubber of Rubber Bridge can take a while to play too so training this AI would probably take weeks and my computer cannot handle that. As this project focuses on game structure and core logic, AI players will use rule-based behaviour only, with no machine learning features.	
W3	Variants	There are several Bridge variants (such as Duplicate and Chicago) but implementing these would fragment development and detract from delivering a high-quality Rubber Bridge experience. To prioritise depth over breadth, only Rubber Bridge will be supported in this version, though other variants may be added in later iterations.	

Limitations

While the MOSCOW plan outlines an ambitious feature set, there are realistic constraints that limit what will be achieved within the project timeline:

- **Unimplemented Features:** All features listed under WON'T will be excluded due to reasons mentioned above. It is also likely that not all COULD features will be completed, especially those requiring more advanced multiplayer infrastructure (like lobbies and persistent profiles).
- **First-time scale:** I've made many applications in the past, but this is my first large-scale, multi-user project. As I am a solo developer and not a professional team, the system may lack robustness in areas such as error handling, testing coverage, and polish. While I aim to create a secure and stable system, some issues may arise under unexpected user behaviour.
- **Security Risks:** Although no sensitive data (like financial data) will be stored, users will have profiles and usernames. This introduces risk of exposure if the system is hacked. To reduce this, I will not build a custom authentication system – instead, I'll use Repl.it's built-in authentication, which has prebuilt security layers.
- **Code distribution:** To further improve security, I will run as much logic server-side as possible and take steps to harden inputs, including validation and sanitisation to prevent exploits such as SQL or code injection. However, full professional-level protection is beyond the scope of this project and not truly very necessary for this project anyway.
- **Scalability:** The backend will be hosted on a free platform (most likely Repl.it but GitHub or Glitch could work too) which may struggle under heavy traffic. High user numbers could cause performance degradation. While trick-based gameplay is turb-based and tolerant of some communication delay, I will simulate heavy loads during testing to identify performance bottlenecks.

Hardware & Software Requirements

The project will be developed using standard web technologies, with JavaScript as the primary programming language. The interface and animations will be handled using HTML, CSS and the p5.js graphics library. Multiplayer functionality will be a hassle due to issues with software and as such development will be split across environments due to equipment and access constraints:

- I will use my personal Chromebook, where I can freely install and run Node.js and Visual Studio Code using the inbuilt Crostini Linux support.
- At college, however, I am restricted by network security and cannot install the required backend tools. Therefore, I will focus on the front-end and interface development during college hours. Preferably, to solve this, I would like to use Repl.it for the entirety of this project but recently limits have been added to the free version such as only 600 developed minutes per month, which will not be enough.

User authentication will be handled by Repl.it's built-in authenticator, as it is browser-based and platform-independent. Other authentication methods, such as Google Sign-In, often require third-party integration that may be blocked or inconsistent across devices, making Repl.it the more practical choice.

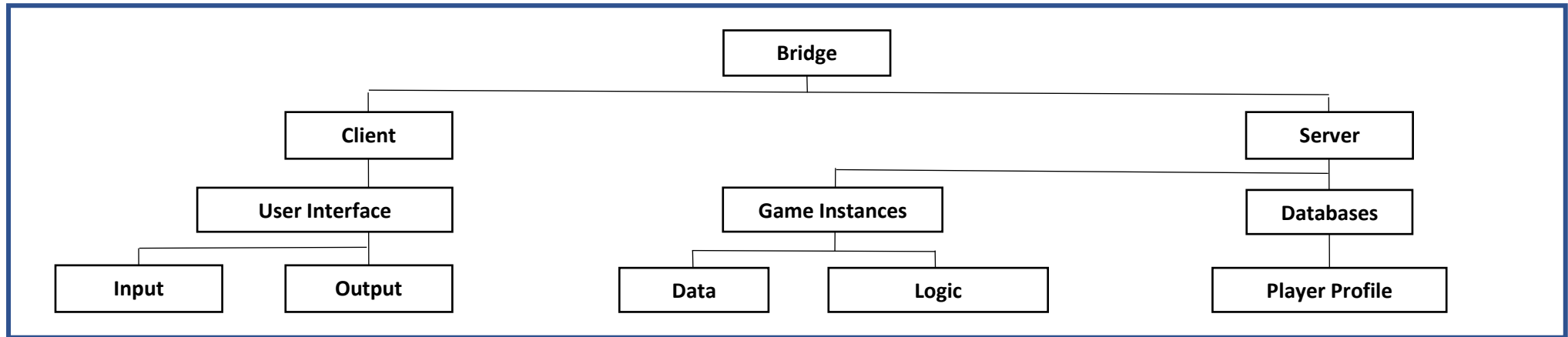
For users the hardware requirements are minimal. The game will run entirely in-browser and will be optimised for low-resource usage. Any device that can run a modern browser should be able to play. Testing will be done across multiple platforms, including Firefox on my laptop, Microsoft Edge on the college computers and Safari on my phone. Additional compatibility testing will include Chrome and other browsers to ensure wide accessibility though this may not be done as thoroughly.

Recommended minimum OS includes Windows 10 for PCs (Edge), Android 8.0 for Android devices (Chrome), any version of MacOS (due to safari being built in), and iOS 10.0, and iPad 13 (both for Safari).

Internet access will be required at all times, as this is a web-based multiplayer game.

Design

Project Decomposition



The logic of this project is rather simple, but it must be designed to allow multiple concurrent games at any given time, and for hidden data to stay hidden such that no one has access to said data thus can't cheat via the terminal. Thus this project will have two main components – the server and the client - and these two will communicate using Node.js and Socket.io.

The server is responsible for most of the game's work: game logic, data storage and move validation. This ensures that players cannot cheat, as all rules, shuffling, and dealing of cards are handled securely and independently of the client. The server also manages game instances – allowing multiple concurrent games – as this requires synchronising these instances across multiple users. The server will also store player profiles and scores for long-time use and so any client can access the data of any others – this is useful for friends systems and for leaderboards where the score of each player must be accessed. Because of this an application protocol will need to be developed – which will indicate how messages are sent from server to client and the meaning behind them. This will wholly be node.js.

The client is a thin layer responsible only for the view and event handling – mouse clicks and user input – as these can't happen on the server. It renders the current game state and sends player inputs (bids, plays) to the server where they can be sanitised and processed. This will primarily be js with P5.js, but will also have little HTML, and CSS animations. This being a thin layer also aids distribution as it will take up little storage space.

This design makes use of the MVC (Model-View-Controller) approach, keeping responsibilities clearly separated.

- The Model is the server's stored data – game instances, databases, and player profiles.
- The View is the client interface – which displays the game state and updates based on server instructions.
- The Controller is the server's game logic – which processes moves, checks validity, and updates the games' states.

By separating these parts, the client remains lightweight and secure, while the server guarantees fairness and consistency. Communication is asynchronous via WebSockets, allowing smooth, real-time updates without the

User Interface

The interface should be rather simple – as this is a learning tool as well as a game. I found some existing implementations, such as Bridge Base Online, can feel very cluttered and my stakeholders agree that this detracts heavily from a game. However, if the UI is too simple it may not be visually appealing, and this could also detract from the game.

The interface must be immersive to give the feeling of playing the game on a table. Because of this, the UI must replicate the look of being at a table – so all players' cards are visible (even though most of them will just be the backside of the cards). It also means that your cards will be bottom middle and the other players' hands are as according to where they sit and thus must be consistent with each other – so NORTH always at top of the SOUTH player and to the right for the EAST player and left for the WEST player. This will work well on larger devices such as monitors but on phones this will result in not much screen real estate and eventually being hard to read – especially with portrait mode – so I could force players to use landscape mode. I also research most common aspect ratios and 16:9 seems to be a standard, so I will make my UI primarily with a 16:9 aspect ratio, however 4:3 and 16:10 is also quite common – so I'll experiment with multiple aspect ratios. This was retrieved from (Wikipedia, 2025).

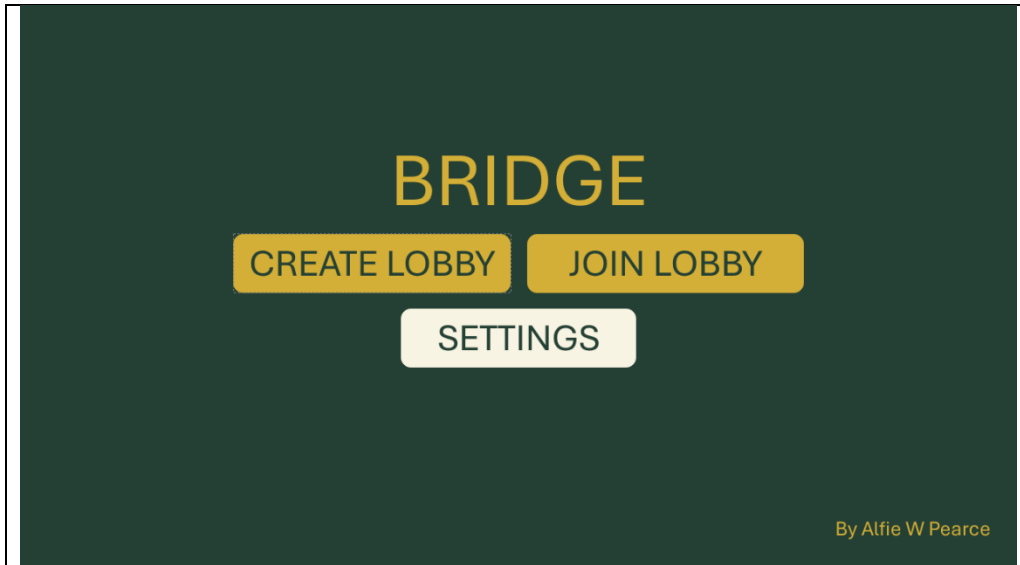
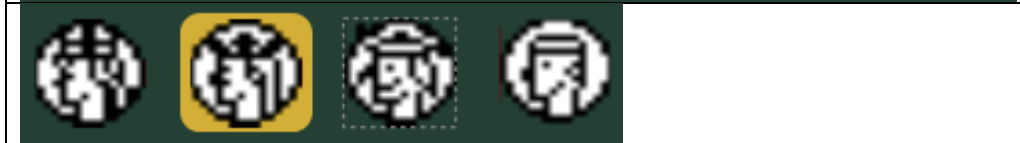
Below is my prototyping of initial designs in order of seeing:

Menu

The requirement for the menu is simple – it must have a play game and settings button. The play game should be a join and create lobby button. And I need to experiment with how the lobbies will work – so ID (like Kahoot) – or public game rooms (like most multiplayer games).

I mocked these up in PowerPoint:

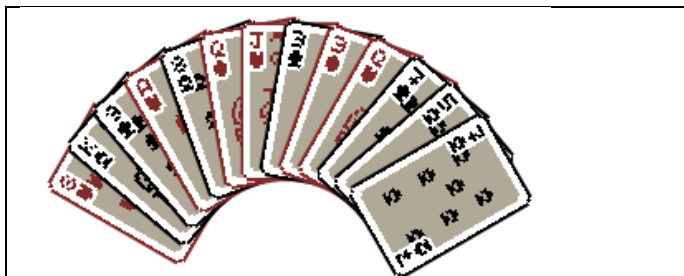
	On the left is my first idea – a very simple play game and settings button on a baize background. I showed this to Kade, and she complained that this would mean there is an intermediary screen between this and playing the game (the lobby screen) - so that increased time before play which is a bad thing. One of my friends is partially sighted and he said that the contrast is good but possibly not good enough and that the buttons should be bigger. Some of my friends also found this to be too simple – but this lacks the title text and watermark so will be less simple eventually. Because of these I am going to finish this – change the background colour and the buttons.
--	--

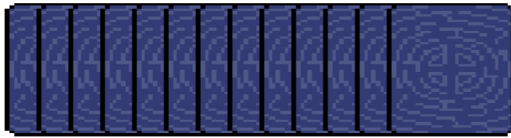
	This design was much preferred by my stakeholders. This is a list of the changes and why: - Play Game button split into CREATE LOBBY button and JOIN LOBBY button these remove the annoying intermediary screen and instantly shows how to play the game. - I changed the font to use purely capitals as this is easier to see – the only uncapitalised part is my name - the water mark – as this isn't essential to be able to read to play the game - I also changed the background to be darker as this further increased the contrast. Next, my stakeholders complained about the avatar selection being omitted so my next version will solve that issue.
	This addition was rather simple, and it seems that my stakeholders like the gold selection box behind the avatars' faces though have mentioned how it is important that these are in fact equidistant and not blurred

Core Display

Next, I'll experiment with the layout of the game – so the placement of the hands, the avatars and names.

These are made with code but lack functionality.

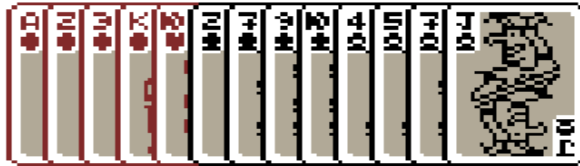
	YOUR hand: As your hand will be the south of your screen this was rather simple – being just a circle which the cards follow and point at you. My stakeholders mentioned that the cards should be in order of suit then pip and that the fan should be more spread out horizontally
--	---



OTHER hand:

These were simple – their only purpose is to boost immersion by showing how many cards other players have thus a simple stack works.

My stakeholders also seemed to like how small this made the hand – so takes up little real estate.

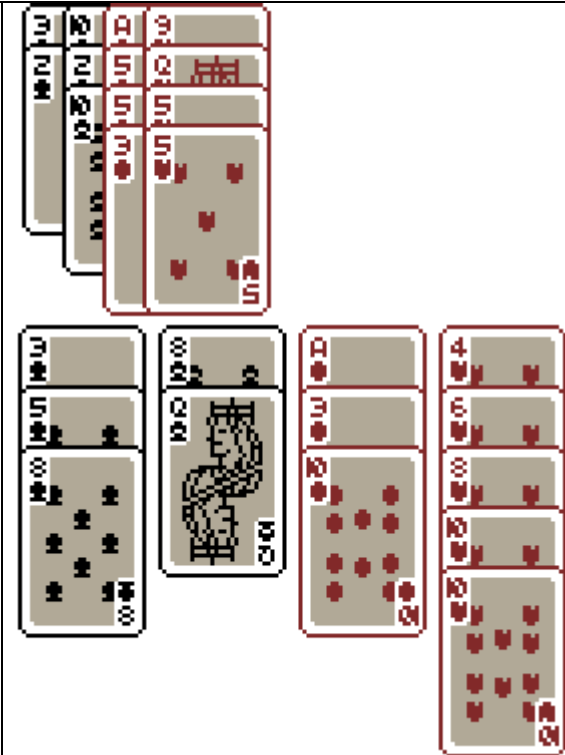


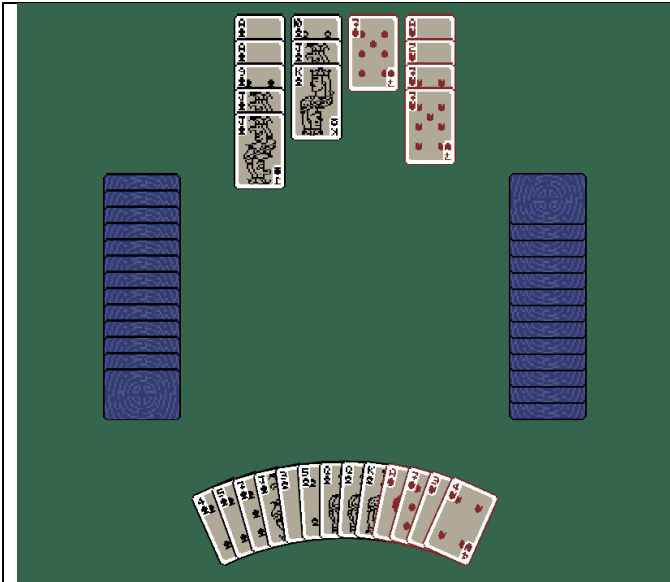
DUMMY hand:

My first idea is on the left. It seemed rather straightforward by stacking the cards the same way that non-dummy card would be stacked. This worked but my stakeholders complained that this not only was hard to read but also took up a rather large amount of screen real estate.

On the top right was my solution to this, by splitting the hand into 4 arrays (one per suit) and display each suit as its own vertical stack and place these suit stacks next to one another. This boosts the readability as it clearly shows each suit but the spacing is lacking and more space is required. Also, Kade said that this should be ordered stacks.

My third attempt is on the bottom right. Here there is more spacing and correct ordering which my stakeholders seemed to prefer.



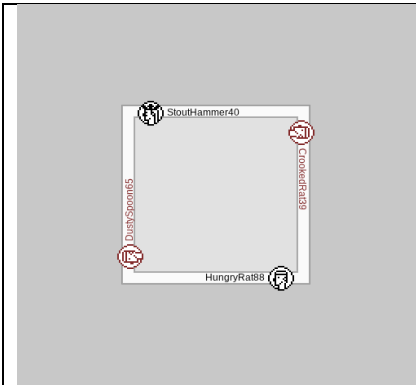


The Layout:

Here is the finalised layout and I showed this to my stakeholders, and they seemed to be fans.

My stakeholders complained about how simple it seemed, but this lacks any buttons or central partition.

Next, I made the central console of the game – this is where information is showed and where tricks are played.



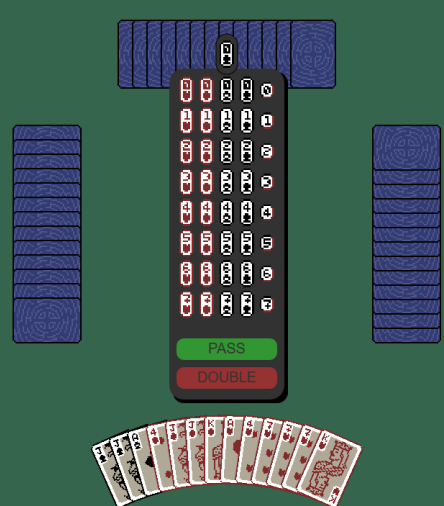
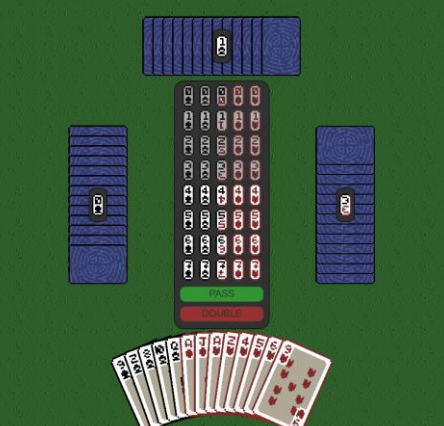
On the left clearly shows the avatars and names and holds space in the centre for playing cards.

My stakeholders asked about rotating the names, but any rotation caused them to be illegible. My stakeholders also seemed to enjoy the space around the names which improve legibility.

Bidding

Now, I'll experiment with the layout of the Bidding table.

These are made with code and lack functionality.

	This is my first attempt at a bidding screen. It shows buttons for your bid options so making a bid, passing or doubling – which becomes redoubling. I had a problem with how the player's deck is hard to view so I'd make an animation which moves your deck further down the screen such that it is fully visible. My stakeholders also had some issues such as the NT cards being of a different height to the Trump cards, and the fact that this has no place to show the current bid and oh who's turn it is to bid.
	This is my second attempt – after addressing most of the problems that my stakeholders have mentioned. I have changed the DBL and RDBL to their full titles DOUBLE and REDOUBLE as I seemed to forget that this is a learning tool so using acronyms isn't intuitive. At the top, I've also added the current bid. I have moved the player's hand downwards and this would be an animation. There are still some prevailing issues that my stakeholders have mentioned: - The size of the NT bid buttons is weirdly small (I made them like this) but this clearly shows that these are different type of bids though I will change this to be the same as the other bid buttons but without any suit drawing. - There isn't any indicator for who is making their play and I've shown Kade some ideas from arrows to changing current bid to placing the current bid over a player's hand and this is what Kade preferred as this shows the history of bids thus allow you to think smartly about making a good quality bid. - Finally, there is no indicator for cards that you can't bid. If the current bid is 6 of spades, then you can't bid between 0-6.
	This is my final work for this screen. I have addressed prior issues – showing all past recent bids, increasing the size of NT bid buttons and blacking out the unplayable bids. I've also done more changes after showing this to some of my stakeholders: - I have moved all the cards and bidding table upwards to give a slight perspective shift and more focus on the Player's own cards. - I have increased the size of everything as some people reported that it was hard to see in detail. - I've added a repeating Baize tile in the background as the pure green background was too simplistic – and my stakeholders didn't like the lack of consistency. - I've removed the shadow as it leads to there being too much visual clutter and it doesn't aid the functionality or the aesthetics.

Scores

Now, I'll experiment with ways to show the player what score they have achieved – including points above and below the line and who has won.

I mocked these up with PowerPoint: (all scores shown reflect the points achieved from the same rubber)

<table><tr><th>Pair</th><th>We</th><th>They</th></tr><tr><td>Games Won</td><td>2</td><td>1</td></tr><tr><td>Part Scores</td><td>1</td><td>1</td></tr><tr><td>Score</td><td>1040</td><td>1070</td></tr></table>		Pair	We	They	Games Won	2	1	Part Scores	1	1	Score	1040	1070	Here I made a simple table which shows number of games won, number of part scores, and total scores. I made this very simple as this would help someone learn. My stakeholders complained about this table a lot. They mentioned that it lacked detail that was necessary and the detail that was listed is useless as knowing the number of part scores isn't useful. They also mentioned that just showing score is better than this table. Eventually, they mentioned that they would want to see the number of points per game and the number of overtricks and undertricks.									
Pair	We	They																					
Games Won	2	1																					
Part Scores	1	1																					
Score	1040	1070																					
<table><tr><th>Pair</th><th>Game 1</th><th>Game 2</th><th>Game 3</th><th>Overtricks</th><th>Undertricks</th><th>Total Points</th></tr><tr><td>We</td><td>120</td><td>0</td><td>860</td><td>+60</td><td>-800</td><td>1040</td></tr><tr><td>They</td><td>0</td><td>240</td><td>30</td><td>0</td><td>0</td><td>1070</td></tr></table>		Pair	Game 1	Game 2	Game 3	Overtricks	Undertricks	Total Points	We	120	0	860	+60	-800	1040	They	0	240	30	0	0	1070	On the left is my revised version showing points per game, overtricks and undertricks and total scores. Kade complained about a few issues still though. Firstly, this is quite tricky to follow and to read. She also mentioned with part scores and bonuses being omitted this made it rather unclear how the points don't add up to the total. This also takes up a lot of real estate possibly too much to fit nicely on a landscape and widescreen monitor and too much for a portrait and narrow screen monitor.
Pair	Game 1	Game 2	Game 3	Overtricks	Undertricks	Total Points																	
We	120	0	860	+60	-800	1040																	
They	0	240	30	0	0	1070																	
<table><tr><td>we</td><td>they</td></tr><tr><td>-800</td><td></td></tr><tr><td>+60</td><td></td></tr><tr><td>120</td><td></td></tr><tr><td></td><td>0</td></tr><tr><td>20</td><td>30</td></tr><tr><td>840</td><td></td></tr><tr><td>1040</td><td>1070</td></tr></table>	we	they	-800		+60		120			0	20	30	840		1040	1070	So, I decided to make my own version of a traditional we/they scoresheet used in actual games of Rubber Bridge. My stakeholders say that this clearly shows the history of prior bids and the scores they gave. It also quickly shows who won what games and with what scores. It is also very simple to understand without any technical vocabulary thus would be good for newer players. It also most closely matches real scoresheets so someone looking to learn the game of Bridge would be able to convert quickly between this and the real thing. Finally, it doesn't take up much screen real estate so it can fit on many screens no matter the aspect ratio.						
we	they																						
-800																							
+60																							
120																							
	0																						
20	30																						
840																							
1040	1070																						

Data Storage

I will heavily make use of OOP in the design of this game. The server will control the rooms and thus the games and will control external storage demand. The client will only provide an interface and handle mouse clicks, because of this most of the data will be stored on the server.

Internal Storage

Storage that is temporary and only stored whilst actively playing the game.

Game Instances

A game instance is created when a player creates a lobby, and players can be added or removed from said game.

The game instances will be stored in a JSON object as the following:

Key		Value	
LobbyId	This is a random string of numbers and letters 0 – 9 a – Z that is created on creation of a room.	Game Object	This is an instance of the Game class – which is mentioned below:

Game Class

This will store the data and the algorithms required to run a game (concurrently to others).

Attributes					
LobbyID	String	Will store the unique id of the current lobby.			
Players	JSON object	This will store the player list via the following JSON:			
		Key		Value	
		SocketID	Created by Socket.IO and unique to each client.	Player Object	This is an instance of the Player class – which is mentioned below:
CurrentPlayer	String	Will store the SocketID of the player (who’s turn it is currently).			
Dealer	Integer	Will store the player who is the dealer this contract			
Declarer	Integer	Will store the player who plays the final bid – thus it’s their contract			
Dummy	Integer	Will store the player who is the dummy			
Pairs	Array	Will store two JSON objects we and they (the two pairs). This will be as follows:			
		Keys			
		Games Won	Integer	Will store number of games won	
		Players	Array	Will store the two players in the pair	
State	Integer	Will store what state the current lobby is in (creating, dealing, bidding, trick-taking, score sheet, done).			
PassCount	Integer	Will store the number of passes done in a row during bidding phase.			
TricksPlayed	Integer	Will store the number of tricks played			
TricksWon	Integer	Will store the number of tricks won this deal.			
TricksLeadSuit	Integer	Will store the number of the leadsuit			
TrickCurrent	Array	Will store all the cards that have been played this trick.			
Deal	JSON	{NUM, TRUMP, DOUBLED} of the current deal.			
Score	JSON	[{below [...], above, penalty, bonus}] *2 This will store the current points of both pairs and where they are achieved and keeping the three game’s below-the-line points separate as these are necessary for determining the ending of games thus the ending of rubbers.			
CurrentGame	Integer	Store what game is currently being played			
Methods					
Constructor		This will have the player (who created the lobby)’s SocketID and add them to the player list. It will then initialise a game.			
Initialise		This will setup the card deck and hands, choosing a random dealer – or rotating the current dealer clockwise.			

ProccessAction	This will be called by the server to parse the action that the player wants to do and run the according function to do it
NextTurn	This will rotate the current player to the next player (clockwise).

Player

This will store the data hidden and separate to each individual player.

Attributes		
SocketID	String	Stores the player's SocketID
LobbyID	String	Stores the player's current lobby's ID
playerID	Integer	0-4 Stores the player's id in their lobby
Hand	Array	Stores the player's current hand
Methods		
Constructor	Creates a new player with and empty hand and no LobbyID – done on loading the application and connecting to the server. Takes in SocketID as parameter.	
Bid	These will be specified by the subclasses of player – RealPlayer and Robot Player and they will return an action that said player want to do.	
Trick		

External Storage

Though player profiles aren't a necessary requirement in my application. I might expand to including a player profile which must be maintained even when the game isn't running (hence external). This will be stored on a database and will be as follows:

PlayerProfiles			
Attribute	Type	Description	Validation
playerID	String	Unique Identifier of the player's profile (Primary Key)	Generated by system.
playerName	String	In game username of the player	Input by player, between 1-15 characters, no profanity (checked using an API most likely).
playerEmail	String	The player's email will be used to sign up and log on.	Check against a regex (BELOW).
playerPassword	String	The player's password will be used to sign up and log on.	Check against regex (BELOW).
playerGamesWon	Integer	Incremented when a player wins a game.	Incremented by system.
playerGamesPlayed	Integer	Incremented when a player plays a game.	Incremented by system.
playerAvatar	Integer	The number used to represent which avatar this player uses: (King-0, Queen-1, Pauper-2, Joker-3)	Generated by system.

Validation

All user inputs are either mouse clicks or touch that either do something or don't. Because of this there isn't any explicit validation that is necessary in this code, but can be affected by events, such as reloading, or changing the size of the window. Server-side processing will also check that these client messages originate from the correct player.

For player profiles and lobby join IDs there will be times when players need to input text this must be sanitised to stop SQL injections and will most likely be checked against a regular expression. Below is the list of Regular Expressions.

Regular Expressions:

These are statements that strings can be checked against to see if they are valid. Their purpose is stated above mostly in the player profile database.

Email:

`/^[\\w-]+@([\\w-]+\\.)+[\\w-]{2,4}$/i`

Password (8 characters, 1 upper-case letter, 1 lower-case letter, 1 number, 1 special character [@\$!%?&._-]):

```
/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%?&.\_-])[A-Za-z\d@$!%?&.\_-]{8,}$/
```

Algorithms

Server-Side

Initialise Game

This is called when a game has begun, so instantaneously when a game has begun.

Class: Lobby object

Parameters: N/A

- Create deck of cards
 - o Loop placing pips from 1 (Ace) to 13 (King) 4 times from suit 0 (spades) to 3 (diamonds)
- Shuffle deck
 - o Shuffle deck – loop through all cards and swap them with a random index
- Deal deck
 - o Give one card to each player in turn 13 times
- Order Hands
 - o Order hands firstly by suit then by pip
- Create Player Objects
 - o Create 4 new player objects with their respective hand in them as an attribute
- Select a dealer at random
 - o Gets random number from 0-3
 - o Set current player to this player's SocketID
- Set game state to Bidding.
 - o Set game state to bidding
 - o And run Bidding function

To test this algorithm, I will:

- Create a series of games and check:

Test	Description	Successful Outcome
1	Shuffle a new deck	Console logs a purely random deck on game load – and after 4 pass bids
2	Receive a random hand	Each hand received is unique, random, and different across subsequent and concurrent games
3	Dealer chosen	Dealer is a different random seat

Handle Click

This is called whenever the server receives an “input” message.

Class: N.A

Parameters: Message JSON

- Will validate the msg
 - o Firstly, check the check digit – maybe run some maths – if come back wrong then return – this will be important to stop people creating their own inputs.
 - o Secondly, check the SocketID against the current player’s – if not same then return.
- Run correct function
 - o Checks what state the lobby is: creating lobby, dealing, bidding, trick-taking, score sheet, or done.
 - o Runs said function, parses the msg as parameter.

To test this algorithm, I will:

- I will make a myriad of fake inputs from wrong check digits to impossible moves and wrong turns.
- I will also make correct inputs to check they work too.
- I will set the state to different values and check if they run the correct function.

Test	Description	Successful Outcome
1	Fake Input	Doesn’t allow fake inputs through
2	Real Input	Allows real inputs through and acts upon them
3	State change	Each state will run its own set of functions and instructions

Bidding

This is called when an input is received, during the bidding phase.

Class: N/A

Parameters: LobbyID string, Message JSON

- Sets Lobby’s current deal
 - o If pass and current bid is not null, then: doesn’t change but increment PassCount, if then PassCount is 3 then increment the game’s state and run the Trick function.
 - o If double set the doubled to 1
 - o If redoubled set the doubled to 2
 - o If normal card pressed, then, if possible (so pip>current pip), then set current bid to it.

To test this algorithm, I will:

- Input pass 3 times to test if it can select a final bid
- Test if doubled and redoubled works
- Try playing different bids from erroneous ones to boundary ones and working ones.

Test	Description	Successful Outcome
1	4 passes	Auction ends – deck is reshuffled and hands are dealt correctly
2	3 passes	Final contract becomes the last non-pass bid
3	Double / Redouble	The contract is x or xx when (dbl or rdbl)

Trick

This is called when an input is received during the Trick-Taking phase.

Class: N/A

Parameters: LobbyID string, Message JSON

- Check if the player can play their card
 - o Checks if the card is in their hand
 - o Checks if the card is of the dealer's suit, if not then check if they can play the dealer's suit, if not then let them play anything.
- If can play card, then add to trick list
- If trick list's length equals 4 then end trick
 - o Calculate the winner of the trick and increment tricks won
 - Trump beats non trump suit
 - Higher pip beats lower pip of same suit
 - o Empty trick current
- If TricksPlayed is 13 then move to score sheet
 - o Increment state.

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Valid tricks	Only let through valid tricks – must follow suit, if possible, must be owned by this player, etc
2	Take turn	Player cannot play when it isn't their turn – UI will show an error
3	Dummy can't play	Dummy player cannot play when it is or isn't their turn
4	Double / Redouble	The contract is x or xx when (dbl or rdbl)
5	Dummy controlled by declarer	Declaring player can play the dummy's hand during the dummy's turn
6	Your turn	The game visually indicates if it is your turn and allows you to play a card
7	Winning tricks	After 4 cards are played the correct player wins the trick and the next lead is assigned correctly

Score

This is called when the score state is begins

Class: N/A

Parameters: LobbyID string

- Check games won
 - o Increment the pair's games won if points-below-line in said game ≥ 100
 - o If pair's gamesWon is 2 then increment state and set the winner to the pair that won
- Wait for next deal
 - o Wait for user input
 - o Then sets state to dealing
- Resets game
 - o Resets most data to default values:

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Next Deal	Next Deal Button works correctly – starting the next deal correctly and without error
2	Game lines	Game lines are drawn correctly and consistently

Synchronise State

This is called when a game's state is changed

Class: Game Object

Parameters: LobbyID

- Broadcasts the new game's state to all sockets within the lobby.
 - o If new state is score, then also broadcast the score data.

To test this algorithm, I will:

- Test with all states.
- Test to see that scores are sent correctly

Test	Description	Successful Outcome
1	Correct Data	All correct data, which a user needs, is sent to them
2	Hidden Data	All data which should be hidden is never sent to the user
3	Multiplayer Clients	All users' clients are independent and receive their own information from the server

Client-Side

Draw Screen

This is called every input from the server

Class: N/A

Parameters: message JSON

- This draws the current screen
 - o Will draw the background
 - Loop and place a grid of the Baize pattern from the tile-atlas.
 - o Will draw deck and dealing then animation if dealing
 - o Will draw the hands otherwise
 - o Will draw bid table, whilst during bidding state
 - o Will draw trick card movement, when during trick-taking
 - o Will draw scoresheet
 - o Will draw win screen, when Rubber is done

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Game	Play a game and make sure there are no artefacts

Animate Play

This is called when an animation is to be played (dealing and trick playing)

Class: N/A

Parameters: message JSON

- This will lerp animations
 - o Move something between two points using an easing function to control were.

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Deal	Start multiple games to test the dealing animation each time
2	Tricks	Play many tricks to check if the cards move correctly

Get Click / Input Handling

This is called every mouse click

Class: N/A

Parameters: N/A - P5.js has built-in mouseClicked function and mouseX, mouseY variables.

- Gets the target
 - o Finds whatever has been pressed on
- Send to server
 - o If no target return
 - o Send message to server with SocketID and target

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Non-targets	Click on non-targets like the background and unclickable cards to see if there is no message
2	Targets	Click on targets (buttons and cards) and see if the resultant message is correct

Robot Action

This is called every time a robot opponent has an action to complete

Class: Robot – sub-class of Player

Parameters: N/A

- Runs the appropriate function for the state
 - o Checks what state the game is in – bidding or trick-taking
 - o Gets appropriate data
 - If bid – pass or a random card if dealer
 - If trick – gets random legal card that can be played
 - o Runs appropriate function

To test this algorithm, I will:

Test	Description	Successful Outcome
1	Opponents play	Bots automatically bid and play a legal move
2	Large capacity	Bots can play any bid, pass, dbl or rdbl and can play cards when it's their turn – choosing non-extreme bids and plays

Application Protocol

The server will need to communicate with every client concurrently. How it will do this is via this protocol.

Message Parsing

Client-Server

- Create Lobby
 - o This tells the server to create a lobby and adds the socket to said lobby
- Join Lobby
 - o Sends the LobbyID that the client wants to log into
 - o Adds the socket to said lobby if lobby exists and is not full – replies with any error
- Start Game
 - o This tells the server to start the game

Client-Server-Client

- Input
 - o This will be called on mouseClicked
 - o Will send the target to the server
 - o If something happens (algorithm above) will respond with a broadcast to all in the lobby

Server-Client

- Update State
 - o Sends new state and any necessary information to all clients in lobby.

Validation

- Join lobby
 - o Will take in a string 6-character LobbyID
 - So must be verified if said lobby exists and ID is of correct length and lobby isn't full
- Input
 - o Will take in a SocketID and a target
 - Must check if SocketID is real
 - Must check if target is real
 - Must check if target is targetable – e.g., it's your turn

Synchronisation

- State change
 - o Anytime there is a state change send this to all sockets within the LobbyID
 - o Anytime something happens that must result in a graphical change server sends request to all clients

Development

Iteration Plan

ID	Iteration	MOSCOW	Details
PROOFS OF CONCEPT			
<u>1</u>	Heuristic opponents		The game of Bridge will have bot opponents for when other players are not available. It is important these bots don't play purely random movements, and I must experiment with bots that are randomly controlled vs heuristically-controlled-bots. Also, as this will be experimenting with and learning about bots, I shall also practise using sub-classes as in Bridge most likely I'll use the Player class and the different types will be subclasses – like weak AI, strong AI, human.
<u>2</u>	Networking		The game of Bridge is a multi-user card game and because of this I will require a client-server architecture, and this will be done via Node.js and socket.io as mentioned in the Hardware & Software Requirements . Because of this I need to download and test Node.js onto my Chromebook, test how to make player's take turn, storing data on databases and keeping this safe.
<u>3</u>	Advanced graphics		The Ui for this game is very important so here I will experiment by making a set of art for the cards, laying these out and animating them.
<u>4</u>	Game Logic		The game happens in very specific and distinct phases, which each need their own UI elements and interactions so I will need to test making an example project.
ACTUAL ITERATIONS			
<u>1</u>	Deck of cards	M1	The cards are very important in Bridge – as it is a card game. This means I need to be able to create a deck of cards, shuffle this deck and deal it out to the four players. Creating a deck, shuffling (fisher-yates) and dealing. I also will need to order the cards within the hand.
<u>2</u>	The Game states	M2 M3 M4 M5 M6	The game of Bridge has 4 main states: Dealing (above), bidding, trick-taking, and scoring. The game will require an algorithm for each. Here I will also make a completely random opponent that plays legal moves, such that after this step, an entire rubber of Bridge will be playable.
<u>3</u>	Node.js	S2 S3	Here I need ed to split the game into a very simple client with just a cli and a server which can host a full rubber of Bridge,
<u>4</u>	Advanced Opponents	S1	Playing against purely random opponents isn't fun and doesn't teach much, so here I introduce a heuristic to the opponents, and they can create more informed and smart decisions – more like real players.
<u>5</u>	Multiplayer	S3 C1 C2A	Bridge is supposed to be played by 4 players – I need to make a simple lobby code system for private concurrent lobbies, such that more than 4 players can join a lobby and can each play their seat, the empty seats should be filled by bots.
<u>6</u>	GFX & SFX	M4 S5 S4	The game needs a UI that shows your cards clearly and allows inputs. It should also execute some audio which after talking to my stakeholders it seemed that three files were enough a UI click sound, card swipe sound, and background music. Me and my stakeholders also liked what (FunBridge, 2025) did by including when clicking on a suit that suit remains all that is visible.
<u>7</u>	WAN	-	This is simple but playing the game on multiplayer would benefit from having multiple computers playing rather than multiple places locally.
<u>8</u>	Clean-up	-	Up to this point the code is very messy. It needs to be neatened and made more maintainable.

Proofs of Concept

Iteration 1 – Heuristic ‘AI’

To learn how to make bot opponents I am going to make a small application. My first thought was a No Thanks bot, but, after more thought, this would be too complex to learn from, so, instead, I am going to make a hen and fox application, where a random number of hens are released and the player can either wait or take – if they take, they get the hens the player with the most hens at the end wins. But the caveat is that if 2 players take then the hens run and neither get a hen. If no one takes, then the number of hens gets added to the next round up to a max number of hens.

This project is designed to develop key AI principles relevant to the game of Bridge – including heuristic decision-making, risk evaluation, opponent modelling, and adaptive behaviour. Although the fox and hen game are far simpler than Bridge, it captures core ideas such as turn-based choices under uncertainty, score-maximisation trade-offs, and multi-agent conflict. This also helped me learn how to use subclasses which could become very helpful when handling players in Bridge.

I first made the simple game, and it runs in the console.

I then created a player class which has a number, several hens and a play() class

I then created the first subclass – randomPlayer. This made a 50/50 decision whether to take or wait and after simulating a 1000 round game the results are often very equal ranging from 160-220 which is the random luck which occurs due to this game.

I then made a second subclass – smartPlayer – which has a slight heuristic advantage – It knows how many hens each player has. This then decides to take or wait based on how likely others will take or wait due to the number of hens they have. This also works well against the random bots as after simulating 1,000 games the 3 random bots got between 80 and 90 hens and the smart bot got 350. Below is the code:

```
if (coupedHens < this.minHens) return null;
let takers = 0;
for (let opponent in players) {
    if (opponent.num == this.num) continue;
    if (opponent.hens < this.hens) takers += this.weak;
    else takers += this.strong;
}
if (takers > this.max) return null;
return this.num;
```

The code on the left shows the play() function within the smartPlayer subclass. This has two bits of knowledge:

- List of players – in which it can see how many hens each player has
- coupedHens – in which it can see how many hens are up for grabs.

The other variables all influence the personality of the player – so aggressive, lazy, etc.

My plan was then to make a third subclass – adaptivePlayer. This player will be the same as the smartPlayer but will have more knowledge and this would be the knowledge of all past actions. This had two cons and one pro. The cons are that:

- A) No player should know the actions of other players – so this goes against that rule and gives the AI an unfair advantage
- B) This wouldn't affect the hen and fox game very majorly but would affect the Bridge game so it will be something I attempt in [S4](#) of the success criteria

While randomPlayer provides a good baseline it lacks strategic depth, but given even a little information – as done above – and acting upon this data the results of the rounds are much better, but it presumes all players will act predictably and consistently which isn't always the case and this is solved by the adaptivePlayer which changes their personality based on previous actions.

Though this idea is very heavily abstracted from Bridge, the idea of a simple AI is made a lot simpler by giving scores to certain data and results. And this also shows how important it is to not settle with just a purely random AI especially in Bridge where so much strategy could be done.

Iteration 2 - Local network chat-app

To begin working with networking features, I first needed to set up the necessary software. Since I’m unable to install Node.js directly on the college system (as noted in the [Hardware & Software Requirements](#) section), I configured it on my Chromebook instead, which is running Debian 12 (Bookworm). I followed a method at (NODESOURCE, 2025) install Node.js, which also includes the option to install N|Solid Runtime – a more robust upgrade of Node.js. However, since I’m already familiar with standard Node.js, I opted to stick with that for now.

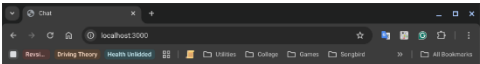
To verify that both Node.js and Socket.IO were functioning correctly, I created a quick proof-of-concept project. The server was set up to respond to the client with a simple ‘Hello world’ header.

To manage dependencies for this project and future ones, I created a reusable Node.js template that includes a package.json file. Below is the script and how I created it.

Package.json:	I created the Dependencies JSON file (package-lock.json) via the terminal code:
<pre>{ "name": "socket-chat-example", "version": "0.0.1", "description": "a test socket.io app", "type": "module", "dependencies": { "express": "^4.21.2" } }</pre>	npm install express@4 This automatically creates package-lock.json and adds the ‘express’ dependency to the dependencies list in the json file.

Now I had the base server I moved on to integrating Socket.IO in (to enable real-time, bidirectional communication).

I first made a Serving HTML file called index.html (see right) - containing a basic form and a little CSS. This acted as the UI. Then, I installed the necessary Socket dependencies via:



Npm install socket.io

This updated the dependency list accordingly.

After connecting the client and server with basic event listeners, the server was able to log user connections and disconnections. Although this is simple functionality, it lays the groundwork for a full multi-user environment. Normally, you would get this boilerplate automatically when using platforms like Repl.it, but since I'm building everything manually in VS Code and would like to know more about how the code functioned, I've implemented it from scratch.

To further develop the chat system, I implemented even emission using the following lines of code:

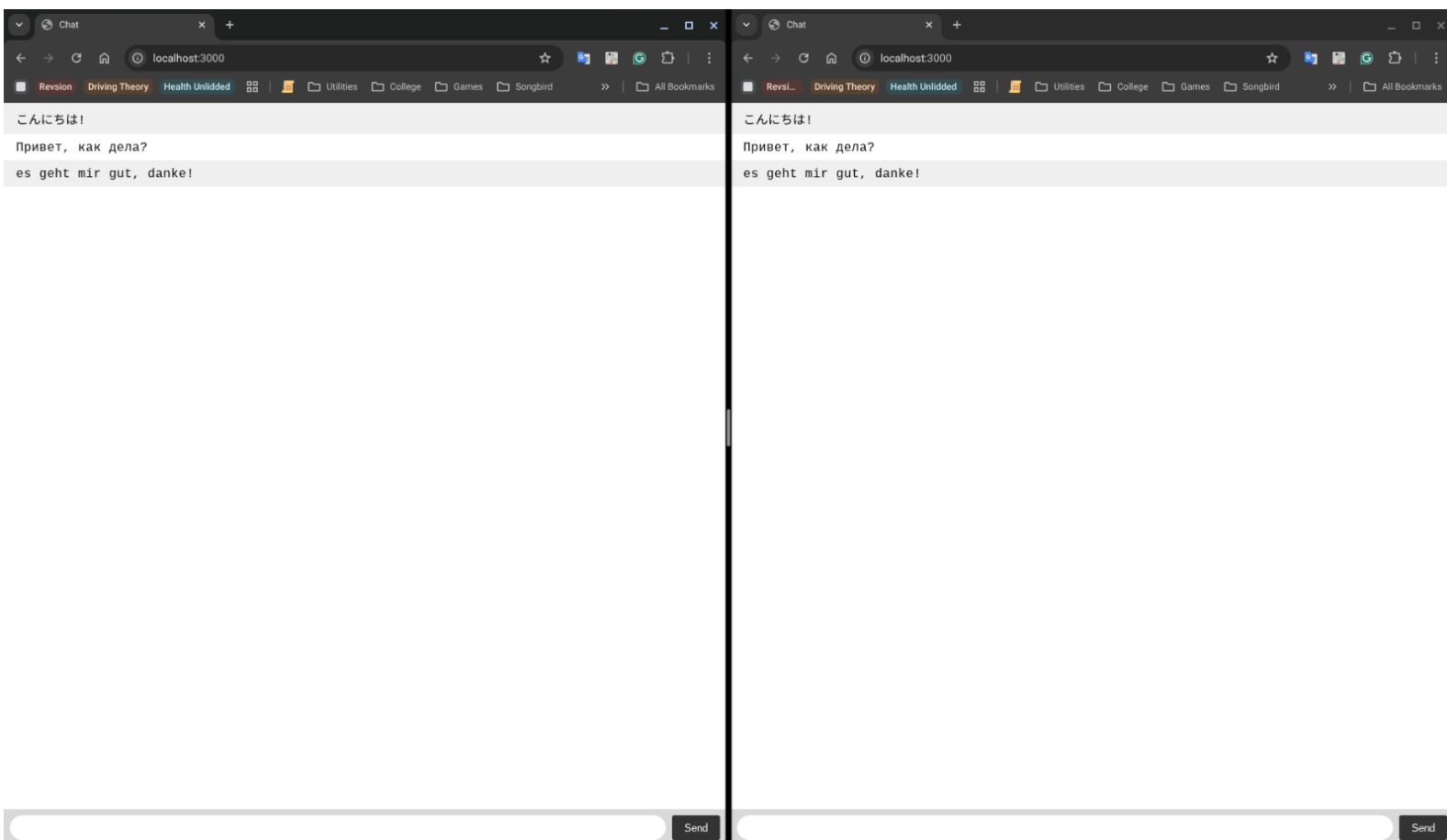


Client-Side	Server-Side
<code>socket.emit('chat message', input.value);</code>	<code>socket.on('chat message', (msg) => { ... });</code>

Currently, I’m using io.emit() to broadcast the message to all connected clients, including the sender. A more refined approach would be to use socket.broadcast.emit() to exclude the sender and I will test this later.

Below is a screenshot of the chat system in action including a fun test of Unicode using multiple languages. They are both clients running through the local server – via separate sockets.

- Japanese: こんにちは (Hello!)
- Russian: Привет, как дела? (Hello, how are you?)
- German: Es geht mir gut, danke! (I’m good, thank you!)



This iteration proved to me that I am fully capable to make the game of Bridge on VS and avoid Repl.it – which can solve some issues on the college’s system as now the firewall is blocking repl.

This practice also reminded me how client-server systems via socket.io can work and thus will allow me to create lobbies and eventually add multiplayer to the game of Bridge.

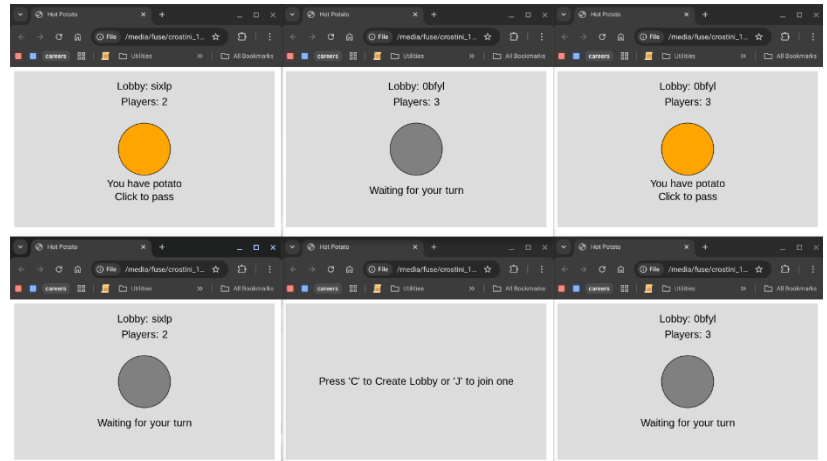
Iteration 2.1 - Lobbies and Concurrent Games

Building on [Iteration 2](#)'s chat app and local server structure, I decided to implement a lobby system – where multiple games can be played concurrently, using Socket.IO's room feature. Now each lobby is a lightweight, server-side grouping of sockets – created dynamically when a client requests entry via `{ socket.join(lobbyId); }`. This allows targeted broadcasts, scoped only to members of a lobby via `{ io.to(lobbyId).emit(msg); }`. I first learnt about this concept through Socket.IO's official documentation on Rooms: (Socket.IO, 2025), which explained how rooms isolate communication between subsets of sockets.

On the server I maintain a games object mapping lobbyId -> { players, turn }, with event handlers for create_lobby, join_lobby, pass_potato, and tidy-up on disconnect. On the client, a P5.js canvas shows the potato and reacts only when it's your turn.

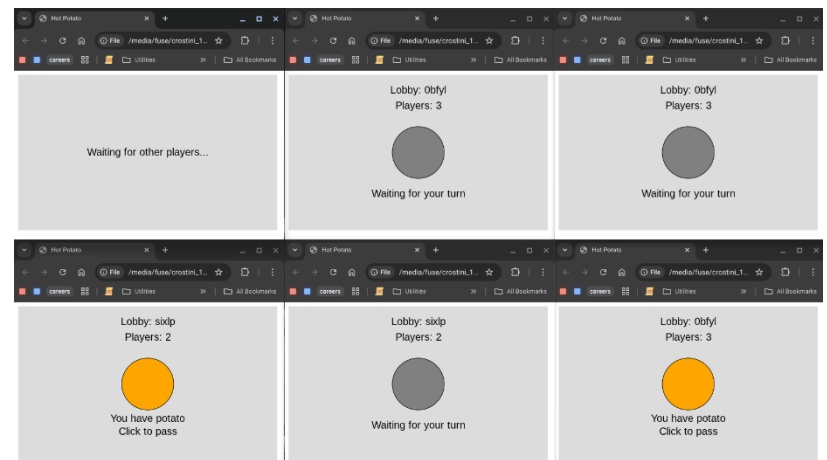
The screenshots to the right show this system at work. To make the concurrency clearer, I tested with six clients simultaneously, laid out in a 3x2 grid.

- Left column: lobby sixlp
- Right column and top-middle user: lobby Obfyl
- Bottom-middle user: not in any lobby



Later, events occurred in sequence:

1. In lobby Obfyl player 2 has passed the potato to player 3
2. The bottom middle player has joined lobby sixlp
3. Player 1 of lobby sixlp has left so the potato is passed onto player 2 (now player 1)
4. The top left player then proceeds to create a new lobby 1jmjv



This iteration proved that I could not only handle multiple concurrent game instances but also manage dynamic lobby creation and clean-up.

Iteration 2.2.1 - Databases

I found two good methods for storing the Player Profiles:

- SQLite – lightweight, a single .db file, no server needed
- MongoDB – document-based, works with JSON objects, free cloud tier available

To compare I'll make a To-do database with both.

	SQLite	MongoDB
Make a table	<code>db.run("CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, description TEXT, done INTEGER)");</code>	<code>Collection = client.db("todoApp").collection("tasks");</code>
Add a task	<code>db.run("INSERT INTO tasks (description, done) VALUES (?, ?)", [description, 0]);</code>	<code>collection.insertOne({description, done:false});</code>
Get all tasks	<code>db.all("SELECT * FROM tasks");</code>	<code>await collection.find().toArray();</code>
Mark task done	<code>db.run("UPDATE tasks SET done = 1 WHERE id = ?", [taskID]);</code>	<code>collection.updateOne({_id: taskID}, {\$set: {done:true}});</code>
Comparison	As you can see above, SQLite can be rather a lot more code than MongoDB, but SQLite follows standard SQL code and thus I can understand it better. I plan to use SQLite as it is local and is completely free. It also creates a .db file which could be useful. In comparison MongoDB is flexible and doesn't need a predefined schema, but I'll know the data i need to store before storing it so I can predefine the schema.	

Iteration 2.2.2 – Sanitisation

Firstly, a profanity checker is simple. I am going to use the Leo-profanity API, as then I won't have to try list every possible profane phrase and alternative that people will try and use. This is going to be important if players can input their own usernames.

The code is simple:

```
if (Filter.check(txt)) {Then run code to do handle a profane input}
```

The next issue is SQL Injections – which can be a problem in SQLite. Looking at the to-do databases from iteration 2.5. If a user typed the following:

```
"test'); DROP TABLE tasks; --"
```

The table would be deleted. This is also a rather simple issue and one I fixed in Iteration 2.5 without even knowing it. The safest way is rather than inserting the description straight into the SQL statement, use parameterised queries (?) to escape the data automatically. Below is an example of inserting a task using parameterised queries.

```
db.run("INSERT INTO tasks (description, done) VALUES (?, ?)", [description, 0]);
```

Iteration 2.3 – Turn-Taking

I then researched about socket.id - which I found out about through <https://socket.io/docs/v3/server-socket-instance/> - and so I could emit messages to just specific players and because of this I could set the button element which passes the potato to disabled and I could show the potato emoji or hide it.

The rest of the coding was quite simple and below is all the interesting stuff:

<pre>io.on('connection', (socket) => { console.log(`a user connected \${socket.id}`); players.push(socket.id); })</pre>	This server-side code registers when a user connects and adds their unique id to the players array. This allows for many synchronous players that the server knows about all of them.
<pre>socket.on('disconnect', () => { console.log(`user disconnected \${socket.id}`); players = players.filter(id => id !== socket.id); })</pre>	This is more server-side code which handles when a user disconnects and just removes their id from the players array, using the built in js array.filter() method.

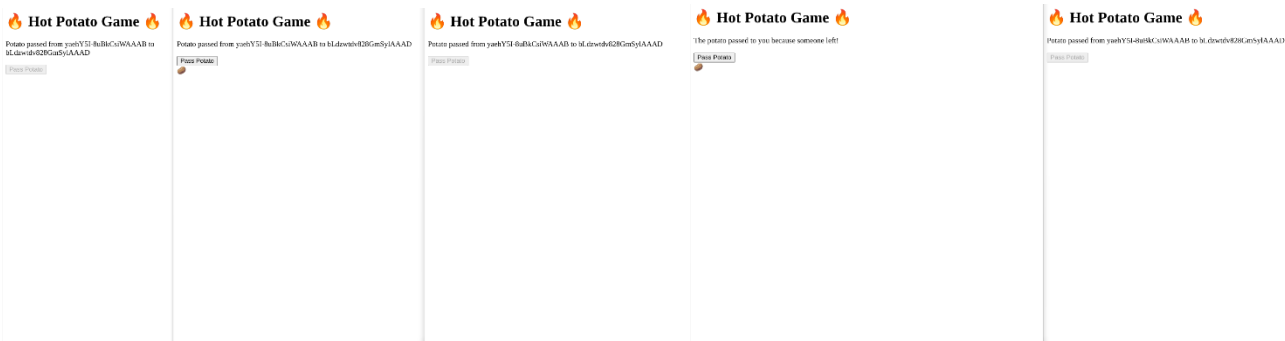
npm install socket.io

I then had the error arise:
Error [ERR_MODULE_NOT_FOUND]: Cannot find package 'socket.io'

Npm init -y

So, I reinstalled socket.io using:

This second line rewrites the package.json so npm knows better of how to manage the project.



The HTML is rather simple. The client just displays the h1, information text, pass button and the potato art. The information text updates when the potato moves – so everyone sees the message saying who it passed from and to and the client that gains the potato may see more information, e.g., if the current holder leaves, they may get the potato.

Iteration 2.4 - WAN

Hostname -I

I then had one problem so up until this point, I have been using a local host:3000 but this game needs to work over a LAN or better the internet. Because of this I wrote some more code.

I ran the command:

This gets the local IP address (e.g., 100.115.92.200) - this isn't the actual IP address it is just a sample one, but it shows the same error I encounter later, I lack knowledge on IP addresses, so I never even noticed it until it didn't work.

Below is the change in the server code:

Before	After
<pre>Server.listen(3000, () => { console.log(`server running at http://localhost:3000`); });</pre>	<pre>const PORT = 3000; const HOST = `0.0.0.0`; // Listens on all IPV4 addresses server.listen(PORT, HOST, () => { console.log(`server running at http://localhost:\${PORT}`); });</pre>

ip a | grep inet.

I then ran the server and opened safari on my phone (to test cross-platform as well at this new host). But this doesn't work. The error I mentioned earlier? It's most likely due to a VM my Chromebook has a lot of security measures so let's find the local Ip address and run that. I ran the command:

This revealed more intel on the Ip address which revealed that this address is in the CGNAT range 100.x.x.x – used by my Chromebook because it runs Debian Linux via a virtual machine and a virtual network. I opened a port forwarding for port 3000 to open a tunnel but this would require access to my Chromebook's ip address. After looking at the wi-fi address I noticed it was in the format 172.32.x.x and here lied a problem I completely forgot about. This will work at home thus I'll test and use this method at home – where network security is lacking – but at college this IP address is a private one likely with many security measures like internal firewalls which would probably block my phone from connecting to my Chromebook and because of this I'll have to use Repl.it from now on at college (or use just a local host) and node.js at home.

Iteration 3.1.1 - Card art

After consulting with my key stakeholder (Kade) and several peers, we reached a consensus on the visual priorities for the playing cards:

- High clarity – legible even at small sizes
- Minimal resource usage – small file sizes to ensure efficiency even with a heavy load.

Initially, I searched online for sprite sheets that fit these requirements. However, most options suffered from poor visual clarity, excessive colours, or restrictive copyright licenses. As a result, I opted to create my own custom sprite sheets using Aseprite – a lightweight, pixel-art-focused program available on Steam. Aseprite is affordable, user-friendly, and excellent for creating compact, high-quality art.

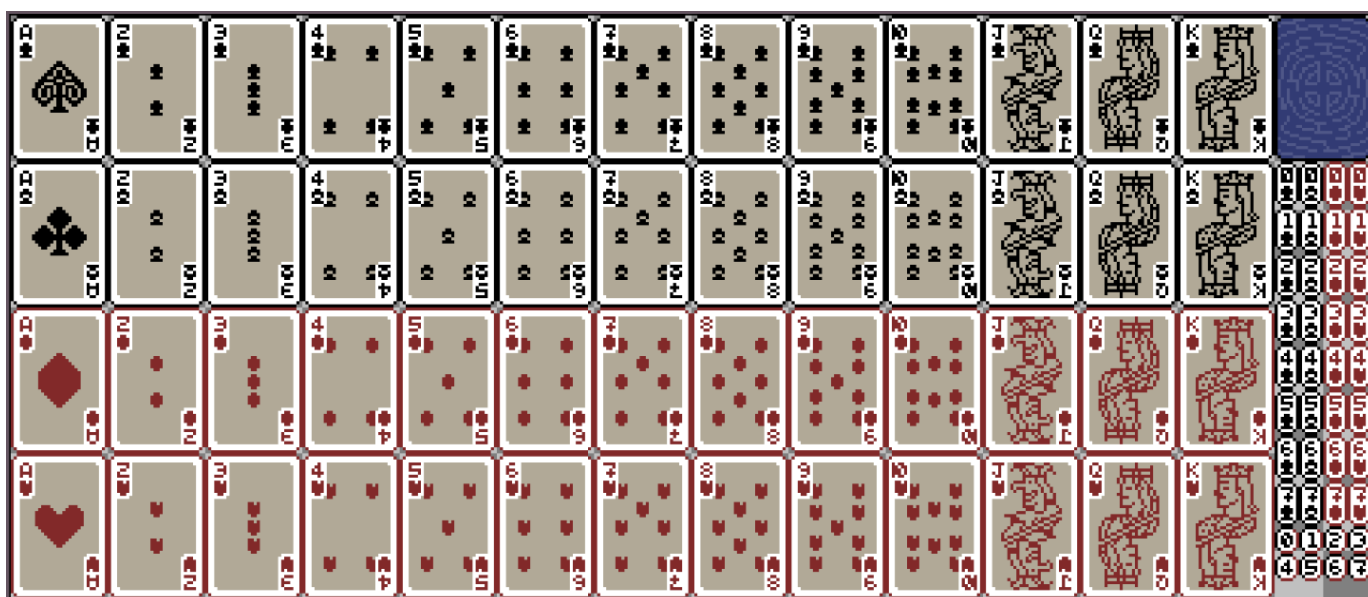
Below is a screenshot of the current image atlas. Note that the image appears slightly blurry here due to compression – this will not affect the final application. Creating the sprite sheet took approximately 3.5 hours.

Design Process and Revisions:

Throughout the development, I encountered several design challenges and made iterative improvements:

1. I originally enclosed the pip and suit symbols in a full border. However, given the small card size (32px x 48px), this layout left little room for the artwork and made the cards feel cramped. I removed the border entirely, but this caused cards numbered 4-10 to become visually obscured. To solve this, I added a subtle background to the suit and number area for better contrast and improved accessibility.
2. I initially included 4 corner indices for readability. However, at this size, they appeared cluttered and once again hard to read once small. I removed the top-right and bottom-left, as this would still let the cards be rotated and read.
3. My first attempts featured highly detailed, multi-coloured designs for the Jack, Queen, and King. However, at low resolution these also lacked clarity and felt inconsistent with the visual style of what would become the rest of the cards and UI, so I simplified the design for a clean, unified look.
4. The original red card back was too like the red suits which undermined visual differentiation – particularly for colour-blind players. I switched to a blue design, which offers strong contrast and is more accessible.

Including the back card created empty space below it in the sprite atlas. Since performance and efficiency are priorities, I didn't want to leave this space unused. Initially I experimented with alternate back designs (e.g., a red sword motif), but after further stakeholder discussions, it was agreed that customisation features were not essential. Instead, I repurposed this area to include bidding cards – these will represent the current contract and serve during the bidding phase of gameplay.



Iteration 3.1.5 - Rendering cards

This phase of iteration was completed at college. Because of this, I used a basic HTML file to connect my JS code and link to the p5.js library. Although I will plan to explore the P5 Visual Studio extension later. Also, for now, I used Five Server, a built-in VS Code extension, to run the project locally (due to the lack of node.js access at college) - necessary because my implementation uses images.

At this point I had:

- Card artwork from [iteration 3.1.0](#),
- Card generation, shuffling and dealing logic from [iteration 4](#)

The challenge now was to display these cards on screen using p5.js.

Loading and Displaying

My first hurdle was loading the image correctly. Initially, nothing appeared. After some searching, I found the preload() function from (W3Schools, 2025), which ensures assets are loaded before the sketch begins.

Then came the of how to draw a specific subsection of an image (since my sprite sheet contains all cards. After consulting (W3Schools, 2025), I learned the full syntax

image(src, x, y, width, height, sx, sy, sw, sh)

The s* parameters define the subsection of the source image to render.

With some trial and error, I adjusted the width and height to match the card ratio (32x48), but I noticed everything looked blurred. This led me to discover noSmooth() at (W3Schools, 2025), which disables anti-aliasing and restoring the crisp pixel-art clarity I needed.

With images rendering correctly, I now needed to make a display function and a fanning function.

Card Display Function

I created the function Display as part of the Card class. At first this only took 4 arguments: x, y, width, and height which worked well at displaying a specific card (subsection found from the self.pip and self.suit), in a required location, but this caused the issue about displaying the back of the card, as more cards will be face-down then face-up, because of this I added the 5th optional argument (visible) which if false would set the subsection coordinates to (13 * cardWidth [14th column], 0 [0th row]).

Fanning a hand

Fanning the cards caused a lot more struggles for me and I had to talk to Kade a lot about different parts which didn't look good. Below are the code (left) and the explanation (right):

```
function fanCards(cards, x, y, xRadius = 125, yRadius = 50,
rotation = 10) {
  const totalAngle = rotation * (cards.length - 1);
  const startAngle = -totalAngle / 2;

  const startOffset = -xRadius / 2;
  const deltaOffset = xRadius / cards.length;

  push();
  translate(x + startOffset, y);
  rotate(startAngle);
  for (let card of cards) {
    card.display(0, -yRadius, cardSize[0] * 2, cardSize[1] *
2);
    rotate(rotation);
```

So, the code originally was very different but different problems caused changes to the code.

1. I first used positive yRadius but this caused the cards to be upside down, which is something Kade had noticed.
2. I also didn't know about push() and pop() which I learnt about on P5's website (W3Schools, 2025) I found out about them because when I made a card follow the mouse cursor in the draw function it seemed to have the translation and rotation effects affecting it.
3. I now had a set of cards going around a perfect arc, but this arc was a perfect circle which didn't look right, so after speaking to Kade I realised this should be a more oval shape, so I had to include a changing x radius of the fan (xRadius) which would translate

<pre> translate(deltaOffset, 0); } pop(); }</pre>	the centre of the circle, each card, so as to generate an oval shape. The result of this code is shown below:
---	---



Iteration 3.2 - Layout of cards

I had implemented a fan layout (fanCards) above in [iteration 3.1](#), which is exclusively for the client – which should always be the bottom player even if the player isn’t south. This is done because it adds visual interest (and immersion, as well as clarity when selecting cards.

Next, I had to display the hands of the other three players, and I found that using a fan layout for all players cluttered the UI and created a disordered looking imbalance. Instead, I used linear, rotated stacks (displayHand) for these players. This evokes the feeling of sitting by a table with real people – something I spotted worked well in some of the [actual iterations](#). This also gave the client a visual distinction from the other players.

The next problem I encountered is that this meant all cards were visible and some must be invisible. For now, I solved this problem via a variable within each card object (visible) which if false the card will be hidden – the back will be drawn. How this function is mentioned above in the [Card Display Function](#) though this has changed since from an argument in display to a variable. This allows much more control over which cards are visible but when I do more work with servers the servers will withhold the cards entirely and just send the number of them so that there are less security issues with cheating.

I also talked to Kade to discuss the clarity of this layout, and she appreciated the fan layout for the players but found the vertical hands were a little difficult to interpret initially.

Although there were no major bugs, I implemented a few minor improvements based on her feedback:

- Vertical spacing – more spacing between cards.
- Offset adjustment – to centre cards properly during rotation and improving alignment.

On the right shows the finished layout. This is on a white background which I do not plan to keep. Furthermore, this is blurred as this is a screenshot. This shows the player – bottom with their cards fanned as well as the dummy – right with their cards stacked. The other two stacks are filled with not visible cards as these hands are not supposed to be visible to this client – hence why their backs are showing. The distance between the cards in the vertical stack is perfect to see the index of the card but not the picture, this was something Kade mentioned wasn’t clear	
--	--

originally. But this makes sure to save screen real estate and still shows the cards in enough required detail.	
---	--

Iteration 3.3 – Animation

The dealing animation was very simple to create, as well. I started with the code from [iteration 5.2](#) as I needed the card layout from [3.2](#) and I wanted to test clicking from [5.2](#).

I started by creating the deal dictionary which stored a Boolean (that we are dealing), the queue, and a timer as well as a constant interval – this is fine for now but the Boolean as a state handler is very rough and can create problems when scaled, like when multiple game states are introduced and because of this I explore this problem in [iteration 8](#).

I then added to the deal.queue to look something like [0,1,2,3, 0,1,2,3, 0,1,2,3, 0,1,2,3,...]. This means that player 0 then 1 then 2 then 3 receives cards – which when drawn looks like players are being dealt cards and once the deal is done then the player and ‘dummy’ are shown.

This is enough for what I’ll need and my stakeholders seemed to like it. They have suggested that I animate the cards flying from deck to the hands and this would use lerp() and easeInOutQuad(). I won’t test this yet, but I will in [Iteration 8](#) – when I test an animation for casting spells in a turn-based combat game.

Iteration 3.4 - Graphical enhancement on clicks

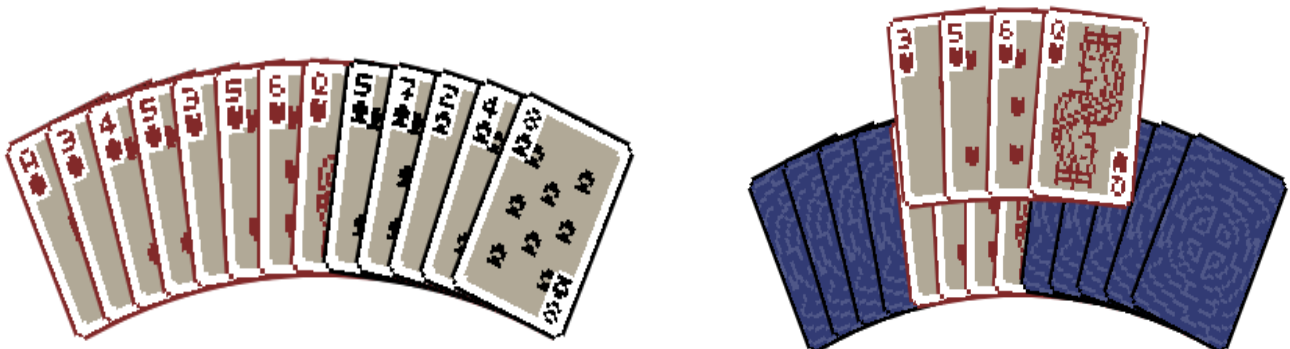
This feature ended up being much simpler to implement than I initially anticipated. At first, I had overcomplicated the process, but after reviewing the code, I realised that a straightforward approach was sufficient.

I started by copying the code from [Iteration 5.1](#), which had the clickable fan of cards, and removed the code responsible for displaying the other players’ hands. The, I modified the mouseClicked() function so that it identifies the selected card, stores its suit, and uses JavaScript’s built-in array.filter() method to create a new array containing only the cards from the player’s hand matching that suit. I then set the visibility of all the other cards to false so only the backs were shown.

In the draw() function, instead of just rendering the entire hand as a fan, the program now draws just the filtered cards after the click. This worked but Kade didn’t appreciate how the other cards seemed to just disappear, and this could confuse people who were newer to the game of Bridge. Because of this I drew a second hand beneath the first this shows the back of all the cards but shows the front of the selected cards. I then changed the xRadius of the filtered hand to be large so that there is more space between the cards – and they are more visible.

Below is a comparison showing the hand in an unselected state (left) and a selected state (right).

This simple graphical enhancement significantly improves user experience by clearly highlighting which cards in the hand are currently selected – so usable, thereby aiding visibility and gameplay clarity, as well as standing as a teaching tool for new players.



Iteration 4 - Game Logic in a Finite State Machine

The game of Bridge has 4 states – bidding, trick-taking, scoring and the results. Each of these states show different UI with different inputs, processes and outputs. I needed to practise making a Finite State Machine, as I lacked experience in making scalable FSMs and this helps me practice using classes and dictionaries (JSON).

This also helped me practice with how to do simple animations using lerp() and calculating easeInOutQuad() – this was done in the cast state to show the player's and enemy's attack.

Below is the plan of the states – these states imitate the states in Bridge with spell select being bidding, cast being trick-taking, score being the scoresheet and end being the results.

Plan - The Game States			
Spell Select	Cast	Score	End
UI 3 buttons – fire, water, earth, selects which spell to cast Wait for player to select a spell Generates random enemy cast Once button selected cast state starts	UI No buttons just a drawing of an emoji that appears at bottom left and goes to middle (lerps) and another appearing top right going to middle too.  Figures new health values Once animation done Score starts	UI Shows your and your opponent's health. Button to go to next round If button clicked, then If health == 0 then End starts If health != 0 then Spell Select starts again	UI Shows the winner Button to retry If button clicked, then Spell Select starts again

I made code using generic methods (a dictionary which stores state, and whether the state has 'begun'). Then in the draw code for each state at the start there is a check to see if the game hasn't begun and this code will run once at the start of the state and will set begun to true. This became very hard to code once at the latter states and I presume that when making an actual Bridge variant this would be too hard to complete, and because of this I needed to write a more 'proper' FSM:

- Use an FSM class
- And in draw function of p5 just run the state's draw function.
- Also, there is a transition function which changes the state and sets 'time in state' to 0. This then runs a function for each state which is what previously would run in the if (!game begun) – which neatens the code and aids scalability.
- I also simplified the nested if statements using mapping – e.g., the winMap() which stores the three casts and what they beat: fire:'earth'

Animation	
<code>this.ease = t => t < 0.5 ? 2 * t ** 2 : -1 + (4 - 2 * t) * t;</code>	The code on the left calculates t (time) – so that it can ease into and out of an animation (easeInOutQuad())
<code>const t = this.ease(this.time); const pos1 = [lerp(0, width / 2, t), lerp(height, height / 2, t)]; const pos2 = [lerp(width, width / 2, t),</code>	This is the actual code to animate the two emojis. The pos1 lerps – using p5's built in lerp function – from the bottom left to the middle and pos 2 lerps from top right to middle.

<pre> lerp(0, height / 2, t)]; text(this.msg[0], pos1[0], pos1[1]); text(this.msg[1], pos2[0], pos2[1]); if (this.time >= 1) this.transition(gameStates.SCORE); </pre>	This then draws the emoji at these positions. If the time is then 1 then the animation is done and the emojis are both at the middle. This means that once the time is or is greater than 1 then the state can transition to the next.
--	--

Actual Iterations

Iteration 1 - A deck of Cards – M1

So, I need to be able to generate and store a deck of cards. This deck needs to be shuffled perfectly (I'll use a Fisher-Yates shuffle). Furthermore, this deck needs to be dealt to 4 players, so I'll have to store 4 hands of 13 fair cards.

First things first, I had to decide how I wanted to store the cards. I have two options:

AS AN ARRAY	OBJECT ORIENTED PROGRAMMING
Here is the simpler at first option. This is because we can store each card as an array [PIP, SUIT]. This isn't a great idea, as there is more that I need the cards to do that OOP would exceed in ->	This is a much better option as I can store PIP and suit as card.PIP or card.suit but I can also add functions in here, for example displayCard(faceDown=..., x=..., y=...)

Ok so now I know for sure that a deck / hand will be an array with 52 CARD class objects. I know had to figure out how to do three things with this.

1. Make the algorithm to generate a deck of cards
2. Make the algorithm to shuffle said card
3. Make the algorithm to deal those cards to the 4 players.

Generating:

This is rather simple, and I did that via a nested for loop. The outer for loop is for the suits so [0,1,2,3]. The inner loop is for the PIPs so 1 (ace) through to 13 (king) and 12 (queen) and 11 (jack). This was simply done via the code below:

```
for (let suit of [0,1,2,3]) {  
  for (let pip = 1; pip < 14; pip++) {  
    deck.push(new Card(suit, pip));  
  }  
}
```

Shuffling

A Fisher-Yates shuffle isn't very tough either. All it does is loop through all cards and replaces that card with a random card in the deck, so 52 random swaps. Below is the code I did this with:

```
for (let idx in deck) {  
  let pos = Math.floor(Math.random() * (deck.length-1));  
  [deck[idx], deck[pos]] = [deck[pos], deck[idx]];  
}
```

Dealing

Now a deck of card is being generated and shuffled, I must deal this, but that is also very simple, as what I found worked well was using splicing the first 13 cards of the deck into a hand. I struggled at first as I forgot how splicing worked but it worked to just set the hand = deck.splice(0, 13) which just sets the hand to the first 13 cards of the deck and removes them from the deck.

Here I tested this code by generating many hands and each were different – seemingly fully different so that is a pass

Ordering

After I did this, I realised (having been told that I forgot by Kade) that these cards should be ordered (by Suit then Pip) as it is a very important quality of life feature included in most existing iterations.

This was rather simple as I used js' built-in array.sort method though this caused the problem with how I didn't know how the compare function worked but after some research I found that (W3Schools, 2025) explained it quite nicely that if result is negative a comes before b, if positive b comes before a, and if 0 no changes are made. I had originally sorted the cards' suits from lowest points to highest, but this caused the red cards to be in the centre and black cards on the edges which became hard to read so I reordered them from spades to hearts.

Test	Desired Outcome	Success
------	-----------------	---------

1	Deck is always 52 cards	TRUE
2	Deck is always differently shuffled	TRUE
3	Each player receives a different hand	TRUE
4	The hands are correctly ordered	TRUE

Finishing this dealing phase completes M1 Dealing

Iteration 2 – The Game States – M2, M3, M4, M5

The game will first function in the console. With the deck of cards produced as before and the following algorithms.

Dealing – M1

Handled prior in iteration 1.

Bidding – M2

I made the bid code mentioned in the [algorithm's](#) sections. This wasn't very hard but most of the code is validation and it seemed I overlooked a lot of the checks.

- First the new value must be greater. And value must be between 1-7.
- Then I remembered suit must be between 0-3.
- Then I remembered that if the value is the same then the suit must be a No Trump – 4 – in which i realised my range for the suit is also incorrect as it must be 0-4.

I also seemed to overlook other parts such as the passCount.

- 3 passes confirm a final deal.
- But if no deal is made 4 passes results in reshuffling the cards.
- I found that i could check if the value of the current bid is 0 then no bid has been made - hence 4 passes can occur.

Below are some screenshots of the console showing this algorithm in function:

4 passes – pass out (reshuffle deck) – shown on left and on the right is if a bid is played and 3 passes confirms that as the bid

```
> game
< ▶ Game {players: Array(4), pairs: Array(2), state: 1, currentDeal: {...}, passCount: 0, ...}
> bid('pass')
pass script.js:115
< undefined
> bid('pass')
pass script.js:115
< undefined
> bid('pass')
pass script.js:115
< undefined
> bid('pass')
pass script.js:115
4 passes reshuffle deck script.js:130
< undefined
> game
< ▶ Game {players: Array(4), pairs: Array(2), state: 1, currentDeal: {...}, passCount: 4, ...}
```

```
> game
< ▶ Game {players: Array(4), pairs: Array(2), state: 1, currentDeal: {...}, passCount: 0, ...}
> bid({pip:3,suit:1})
< ▶ {pip: 3, suit: 1, doubled: 0}
> bid({pip:4,suit:2})
< ▶ {pip: 4, suit: 2, doubled: 0}
> bid('double')
< ▶ {pip: 4, suit: 2, doubled: 1}
> bid('pass')
< 'pass'
> bid('pass')
< 'pass'
> bid('pass')
3 passes current deal is ▶ {pip: 4, suit: 2, doubled: 1} script.js:124
< 'pass'
```

I had to remember to validate the deal with bidding too. To the right is the result and I did this using simple regex tests.

```
> bid({pip:0, suit:5})
< 'Impossible suit'
> bid({pip:9, suit:3})
< 'Impossible pip'
> bid({pip:0, suit:3})
< ▶ {pip: 0, suit: 3, doubled: 0}
> bid({pip:0, suit:3})
< "Can't play same bid"
```

Test	Desired Outcome	Success
1	After 4 passes everything reshuffles	TRUE
2	After 3 passes: the correct declarer and the correct bid are chosen	TRUE
3	Double and redouble are possible and work correctly	TRUE
4	You cannot bid invalid – value less than the current bid, value or suit out of range	TRUE

This marks the end of M2 – Bidding is fully playable

Trick-Taking – M3

Once the bidding is complete Bridge moves into the trick-taking phase.

Firstly, I needed to ensure players couldn't play invalid cards. So, two rules must be enforced:

- The card must exist in the player's hand – that was simple:
 - o `player.hand.findIndex(c => c.pip == play.pip && c.suit == play.suit );`
 - o If this is `-1` then the card doesn't exist in this hand.
- Also, the suit must be the leadSuit, if possible.

If the move is legal then the card is removed from the hand and added to the current trick. Once four are placed the trick is evaluated using the following code:

<pre>function evaluateTrick(trick, leadSuit, trumpSuit) { let best = trick[0]; for (const play of trick) { const card = play.card; //Ignore if bad suit if (card.suit != trumpSuit && card.suit != leadSuit) { continue; } //Trump suit beats not trump suit else if (best.card.suit != trumpSuit && card.suit == trumpSuit) { best = play; } //Higher pip wins else if (best.card.suit == trumpSuit && card.suit == TrumpSuit best.card.suit == leadSuit && best.card.suit == leadSuit) { if (card.pip > best.card.pip) best = play; } } }</pre>	This sets the best card of the trick to whichever card has the highest value. It first checks if the card is of a “bad suit” which means not lead suit or trump suit – as this would garner no value. It then checks if the next card is a trump suit and if the best card is the lead suit which isn't the trump suit, then the trump suit card is better. It then checks if both cards are the lead suit or both are the trump suit (looking at this now I realise i could just check if <code>suit == suit</code>). As then the better card is whatever has a greater pip so there is one more check. There is one more forgotten thing that Ace has pip value of 1 the smallest yet ace is most valuable to a trick, so I need to change the check to the following: <pre>if (best.card.suit == card.suit) { if (card.pip > best.card.pip card.pip == 1) best = play; }</pre>
---	--

Every time four cards are played, a winner is determined and `game.currentPlayer` is set to that winner. I also track the number of tricks completed and once this reaches 13, the deal is over and the scoresheet begins.

Test	Desired Outcome	Success
1	Player is forced to play the lead suit if they can	TRUE
2	Trump suits beat non-trump suits	TRUE
3	Higher pip (Ace being highest) wins	TRUE
4	Players cannot play a card they do not own	TRUE
5	The current trick array updates correctly and the player loses the card from their hand correctly	TRUE

And this is M3 complete as a full round of tricks are playable

Score – M4 & M5

I first made the score variable:

`[[below: [0, 0, 0], above: 0, penalty: 0, bonus: 0]] *2`

Then I made code to add the results of a deal to the score variable, but I realised here how that was storing the score variable was very primitive and would eventually cause problems. Because of this, I first changed how I am storing pairs:

Key	Value
Id	Unique id of the pair
Players	Stores the two player instances of the pair
Score	This stores the same as above but also stores whether the pair is vulnerable or not

With a better way to store pairs I could now set the scores more accurately. I could get the two current scores using the `findIndex()` function built into arrays. Then I calculate the base values – level, trump, doubled, requiredTricks, made?, vulnerable, overtricks, undertricks, etc.

The code then calculates the numbers to add to the score parts.

Then the code calculates whether the game has ended - >100 points-below-the-line and checks if the rubber is complete – a pair has won 2 games. It also changes the games state accordingly.

This code works well and had only one problem with it. I had set the score variable to the declaring pair's score then I handle this score, but this only changes the score locally so I had to then set the pair's score to this variable and had to do the same with the `oppScore` if the declarers didn't make enough tricks.

I also made three helper functions `contractPoints()`, `overtrickPoints()`, and `penaltyPoints()` as these were heavy bits of maths related to each section and I found the code handled better and was easier to maintain, when these were handled separately.

Testing this algorithm thoroughly is very important, as it does so much for the game and it's very important that the scores are calculated correctly. To test this algorithm a test algorithm had to be made, as otherwise setting up the score state via playing a bid and taking a full trick – so all 13 tricks, can be rather tricky and testing all possibilities will take a long time so instead I wrote a three different score algorithms the first runs one test by setting the game values but I need more than one test so I ran a battery of tests via this. I then altered this to test the results and check if they are correct. What I found:

- I had a problem with Spades – 4 overtricks should be 120 points not 80
- I had a problem with Diamonds – 5 overtricks should be 100 points not 150
- I had a problem with Clubs – 4 overtricks should be 120 points not 80
- There were no problems with No Trump or Hearts

Luckily these were all one rather simple fix as I found that the `trickValues` were set up incorrect – `trickValues` is a dictionary to help convert from suit to base points.

I also had problems with the naming of variables. Some variables weren't clearly indicated when they were an index or not, but this also wasn't too big a fix.

After these fixes the following tests were conducted

Test	Desired Outcome	Success
1	The correct section of the correct score dictionary updates	TRUE
2	The score updates to the correct amount based on bid, vulnerability, and tricks won	TRUE
3	Game lines are recognised correctly	TRUE
4	Rubbers end correctly – with the correct pair winning	TRUE

Calculating the score completes M4 and M5 is fully completed via recognising when games and rubbers complete.

Iteration 2.5 – Random Robot Opponents – M6

Making the function to have random opponents was very simple.

At first, I overcomplicated the bidding by trying to find optimal bids, but if a bid has already been played then pass is a fully valid bid. The first bid can be a random number for the value and the trump. Then run the bid function with this action as its argument.

The trick-taking was the same thing and I eventually done it by looking at the player’s hand and using the built-in find functions firstly finding if there is a leading suit and if so if there is a card of said leading suit – if so then select it, if not then find if there is a trump suit if there is a card of said suit, if not then selects the first card of the hand. This guarantees a legal play.

Test	Desired Outcome	Success
1	Bots play on and only on their turn	FALSE

This was fixed easily by changing the while loop’s condition as it was set incorrectly – all players were instances of the player class so instead i checked if their seat ID appeared in a “human” player’s list and if so, break the loop.

2	Bids are always valid	TRUE
3	Tricks are always valid	TRUE

This is the end of M6 – random opponents that can play against you.

This is also the end of the Must section of the game development as from here one can play a full game of Bridge – even if it isn’t very pretty.

Iteration 3 – Node.js – S2 & S3

I next need to create a server but doing this would mean that I needed to get rid of running commands in the console. Because of this, I am going to make some HTML to handle inputs and outputs and then some simple code will analyse your input and run the desired function.

I made the mistake of not checking my code properly using the output function which takes txt as a parameter and adds the txt to the inner HTML of the output division. I had a few console logs where I had used commas to concatenate and well this would be a second or third parameter, but luckily that was a simple fix and with the console no longer needing to be used – I can now transfer to using a server to run all of the code.

Also, I forgot to handle unexpected inputs – yes, this system is temporary, but I still need the inputs to be accurate and Kade suggested to use an alert with the phrase (You're funny).

I also had some issues with recursion. A lot of the code is recursive – calling the robot action and running a bid / trick. But this meant that the outputs were weirdly ordered as more deep outputs would occur first, so I had to redo when robot action is called. I did originally try fix this via prepending rather than appending which mostly worked but had some issues hence I had to change the rest of the code – but this also means I am starting the creation of the server with a stronger foundation.

I also changed the emojis to be    , which appear slightly larger than the numbers and are red with heart and diamonds – this was a very simple change which helps visibility.

To make the server I had to create some more folders and files. Below is the hierarchy (with Src and Public being two folders):

File Tree:

Src	Client.js	Any files within the Src folder are served to the client – so is code that is run on the client. The js file handles inputs and outputs and the html handles graphics.
	Index.html	
Public	Game.js	These are the files ran by the server.
Server.js	These are the server's files – this is the actual server, and all code will be run in files from either the public folder or on the client in the src folder.	
Package.json		
Package-lock.json		

This is very simplistic but meant that all of the functions of the game were executed by the server and not locally and all game data could be stored server-side rather than client-side which is important such that one cannot cheat. It also lays a better foundation for when this project will be converted to multiplayer.

This right now has a very simplistic application protocol being action and remote action between the two possible broadcast messages – they have the following data: id, actionStr, fromLocal. This broadcasts what needs to be done, to whom and whether it was from that client, or not – this is important as it can speed up the code by running some code locally – though later I remove this due to security concerns.

At first, I changed the player inputs to emit an “action” message and I realised I didn't do this for the robots – so I handle them differently this will later cause an issue with multiplayer and if I need to change the core functions or change all inputs at once – the computer needs to treat the robot opponents as if they were human.

Also at first, I made the server run the code to handle multiple clients, but the server wasn't authoritative, so I eventually changed this by executing most of the code on the server. I also had the issue that my code handled outputs, but I wanted to separate the UI onto the client and the server handle the game data.

I had a few more rather random issues, that don't all holey relate to the server, that I found through testing:

Issue	Fix
Next() not working – but should start next deal	When a contract is finished, I had a guard clause to stop the player doing any action, but this had to change to be stop doing any action but next()
If player is the dummy their hand appears twice	Adding a guard clause to a repeated call such that it only shows the dummy's hand if it isn't the players otherwise don't show both just the players.
Only 3 players play each bid and each trick	Firstly, I had called a loop which loops between players 1-3 but adding the dummy added a break in this loop as this could be played by player if so, robot is player 1 and 3 and 0 and 2 must be played by human so I added a recursion to this loop for after said player played. Then it still didn't work but I found this was because all four players play it was just the output was moving too fast and thus, they never had a chance to output their action and instead went straight onto the next trick.
The hands (player and dummy) not appearing	On the bid the player's hand are shown for sure by using a first turn variable which checks if the hand is displayed first or robots are to act first and then displays the hand after that.

To neaten my code, I created multiple scripts, which will access each other via exporting some public functions but not all and then importing the required functions using the following code (esm style):

<pre>Import { Game } from `./Public/Game.js` // This code imports the game class to any script</pre>
--

File Tree with multiple scripts:

Src - folder	client.js	This gets and handles the inputs
	draw.js	This displays the outputs – p5.js
	index.html	This handles the game UI and html tags
Public - folder	game.js	This stores the code that runs the game loop.
	bidding.js	This stores the code to handle the bidding phase.
	tricks.js	This stores the code to handle the trick-taking phase.
	score.js	This stores the code to handle the scoring phase.
Server.js	These are the server's files – this is the actual server, and most of the code will be run in files from either the public folder or on the client in the src folder.	
Package.json		
Package-lock.json		

This is how the hierarchy looks currently, though is destined to change as the requirements and feature list of this project grow.

This mostly completes S2 & S3 – S2 still requires multiplayer which will be done later

Iteration 4 – Advanced Opponents – S1

Currently the robot opponents play random moves that are legal, thus aren't very intelligent and are rather boring to play against – not very good for someone wanting to learn how to play Bridge. So, I need to introduce some heuristics. (Wikipedia, 2025) describes heuristics as any approach to problem solving that employs a pragmatic method that is not fully optimized, perfected, or rationalized, but is nevertheless "good enough" as an approximation or attribute substitution. I am not going to use any advanced conventions though I will later mention them.

To make these opponents work I first must figure out how their logic will work:

Bidding:

First part to a successful bidding decision is:

Hand Evaluation:

First the HCP (High Card Points) value of the hand must be calculated:

Card	Points
Ace	4
King	3
Queen	2
Jack	1

Then the distributional points can be calculated

Type	Points	Explanation
Void	3	Having none of a suit
Single	2	Having 1 of a suit
Double	1	Having 2 of a suit

These are important as the distribution of the cards within a hand affect how powerful a trump or no trump could be and how powerful crossruffing would be.

Then the total points are calculated – sum of the HCP and distributional points and affected by difficulty and personality.

Bidding Heuristics:

Total score range	Likely Bid
Opening	
0-10	Pass
15-17	If balanced then open 1 no trump
13-19	Open 1 in longest suit
6-10	If there's more than 6 cards in a suit – open 2 longest suit
20+	Bid 2 in longest suit
Responding	
6-9	If fit then single raise
10+	If fit then jump raise
5+	If fit and supportive then single raise
14+	If not fit and is balanced then bid no trump
Over-calling	
15+	If aggressive then single raise longest suit
If none	
Then bid pass	

Add simple rules as well:

- Prefer Majors (♠♥) over Minors (♣♦) – more points
- Respond to partner's bid using the same logic – e.g., Raise with support, bid new suit if strong, pass if weak

Doubles:

- Takeout double (opponent opens) if hand has 12+ HCP, short in opponent's suit and support for others – useful if good hand but no clear trump able suit
- Penalty double (all others) if no safe bid of own and opponents have strong holding.

Redoubles:

- Do if confident – strong hand, partner makes double – but be wary as very risky.

If I have time at the end of the project, I can introduce a bidding convention. Conventions are an agreement about an artificial call or a set of related artificial calls according to (Wikipedia, 2025).

*Trick Taking:***What the AI will know:**

- Its own hand
- The cards played
- Current contract
- The dummy – if visible

When leading:

- Lead top of the sequence (KQJx – play K)
- Lead low from unguarded high (Kxxx – play low)
- Lead trump if opponents are crossruffing or dummy is short suited
- Lead partner's suit if known (found from their bidding open)

When following suit:

- If can win – play lowest winning card
- If can't win – waste lowest card
- If partner can win – waste low card
- Track which high cards are dead – (seen)

When can't follow suit:

- If trumping helps win trick use lowest trump
- Otherwise discard weakest card of weakest suit

Personality & Difficulty:

I plan to make each robot opponent feel different and I'll do this by:

Personality Sliders:

Use 0-1 for each trait:

Trait	Description
Aggression	Likelihood to overbid / take risky plays
Supportiveness	How much trust in partner's bid and play cooperatively
Risk Aversion	How likely to choose safe plays over bold sacrifices

At first these will be randomised though I could include sliders in a settings or lobby setup menu.

Difficulty levels:

Difficulty	Requirement
Random	Plays random legal moves – from before – easier than any heuristic choice
Easy	Uses HCP for bidding & basic play rules
Medium	Adds card-tracking
Hard	Adds probabilistic estimations & Bridge Conventions

When this is complete this will complete S1

Coding:

First part of the coding was setting a framework, and I did this by creating a new script in Src called Player.js. This stores the player class which is as follows:

Attributes	
ID	Will store the player's id – to check if they are current player
Hand	Will store this player's hand.
Methods	
Bid	Return a code as to what the player will bid – set up in sub-classes
Trick	Return a code as to what the player will play – set up in sub-classes

It also stores the sub-classes:

- RealPlayer – will return null when bid / trick called
- RandomPlayer – will return random legal play – this will be included as a difficulty of opponent
- HeuristicPlayer – will return chosen legal play – chosen by probabilities and information – this will be easy-hard difficulties of opponents and thus will have changeable data including 4 more attributes:
 - o difficulty – 1->3 – easy to hard
 - o aggression – 0->1
 - o supportiveness – 0->1
 - o riskAversion – 0->1

At first, I used switch case statements to calculate HCP and distributional points – I then changed these to maps as this simplifies the code and makes the code easier to change.

To evaluate the hand this is the code (without comments):

<pre>let hcp = 0; let suitLength = [0, 0, 0, 0]; let distributionalPoints = 0; const hcpMap = { 1:4, 13:3, 12:2, 11:1 }; const distributionalMap = { 0:3, 1:2, 2:1 }; for (let card of this.hand) { hcp += hcpMap[card.pip]; suitLength[card.suit]++; } for (let length of suitLength) { distributionalPoints += distributionalMap[length]; } const totalPoints = hcp + distributionalPoints; const longestSuit = suitLength.indexOf(Math.max(...suitLength)); const isBalanced = [suitLength.sort((a,b) => b-a).join(',')].some(shape => ['4,3,3,3','4,4,3,2','5,3,3,2'].includes(shape));</pre>	First default values are set – all to 0 as these will be calculated in the for loops. Secondly the two maps are made which simplifies code and simplify how the math can be calculated and changed, e.g., if I wanted to give more weight to voids, I could increase the distributionalMap [0]. This is currently sorted in descending points order. Then it loops through all of the cards in the hand And increments the hcp value based on that of the hcpMap. Then it calculates the length of each suit by adding to the suit lengths Then, it loops through the suits and adds to the distributionalPoints based on the suit lengths found in the above for loop. Finally, it calculates values to be used in parsing. IsBalanced is a long line but in short calculates whether the suits are of similar length.
---	---

Coding the rest of the bid method was rather simple:

3. I coded the opener: a simple if tree checking that the totalPoints are within a certain range along with other factors such as if the hands are balanced – then play NT if not then play longest suit, etc.
4. Then I coded the responder: (where they respond to their partner's bid). This requires some context – some more heuristics (awareness) such as their pair, the dealer's pair, their partner's id, whether their partner is

the last declarer. Then checks if their hand is fit with their partner's bid. Then raise if they are fit to if not then don't.

5. Finally, if another player is declarer, then the player might have to overcall. If hand is strong raise if very strong then jump raise otherwise just pass. I had an issue at the start where this caused an infinite loop as I never checked if the bid is above 7 and so I made an if next level = 7 then pass but this still meant the bid was always 7 so below is how I fixed it:
6. I calculated an estimate for the projected tricks but my first method undercalculated and caused 4 passes each deal so I changed it to: (which sets a default of 6 and plusses a third of hcp and 0.3 per card if more than 4 cards):

$\text{Const projected tricks} = (6 + \text{totalPoints} / 3) + (\text{suitLength}[\text{suit}] - 4) * 0.3$

I then made a lambda expression to calculate whether a target level is greater than this projected tricks – this will make sure that no bid greater than can be met is made.

7. I then implemented difficulty and personality. Difficulty was simple and for the bids is just a noise value which increases its randomness according to the difficulty – easier => more random. This is then added to the totalPoints.
The personality took a bit more work as there's more this can affect in bidding – most of the if statements that raise can check if a random value is greater than the aggression, which means more aggressive players will raise. I also added a safety margin to add to the lambda expression, so aggressive players have smaller margins and risk aware players have much larger margins so they're less likely to bid high then aggressive players.

Issues – The code snippets have been shortened to show important bits; there were a few issues - as I expected

Issue	Source	Fix
Bots always pass	Total points = NaN, because distributional points = NaN. Distributional points were added to via the map – which only stores keys 0, 1, 2 so if there are any suits where there are more than 2 cards this would be undefined	Changed the distributional points to be: DistPoints += distMap[length] ?? 0;
After testing this there wasn't any issue anymore and this taught me to always handle missing dictionary keys and test all normal data.		
Bots opening on wrong suit	A bot would open with a suit that's short so the suit length array must be mis-ordered. When I find out whether the hand isBalanced I use the .sort() function which would be fine, but this mutates the original array, so spades are always the longest suit.	This was changed to a non-destructive method: Sorted = [...len].sort((a,b)=>b-a)
This was tested via simple console logs, and this fixed the issue, and this made me cautious to look out for mutating methods and use the ... literal.		
Infinite Bids	Game soft-locks during bidding with the console showing an infinite number of bids	Added two guard clauses: Level >= 7 -> return `pass` Bid <= previous -> return `pass`
After simulating strong hands against each other these stopped at 7 but there were still too high bids – so this I later fixed by calculating the expected number of bids and overhauling these guard clauses. This reminded me to make sure to check boundary conditions.		

Most of the issues afterwards were down to the fact that this code is mostly maths and a load of if statements, so were both easily predictable and hard to track down issues. In a later Iteration this will be neatened.

Coding trick-taking took a lot longer as the logic to a trick is even less of an exact science.

I first calculated what was known such as what suit trump is, which is lead, is this player a leader. I then coded for the following situations:

- Leading: finds lead suit choice – longest suit. If there are less than 3 cards in the trump suit and this is the longest suit, then change the choice to the next longest suit (originally had a problem where i didn't check if there were any other cards which were playable but this was fixed with a simple guard clause). It then plays the strongest hand of said suit. (If i upgraded this I would use the partner's hand if they were dummy to have a say in this code).
This later added partner preference and disliked suits based on code mentioned below and these can also change the choice.
- Following suit: this first calculates whether the player can win – so if they have a card of the lead suit which is greater than all cards played prior and whether any trump cards have been played. It then calculates whether it even wants to win - so if the partner isn't winning and some maths relating to personality. Then if it wants to win and can win then it plays the lowest winning card otherwise it just discards the worst possible card.
- Trumping: this first calculates the cards which are playable and are larger than the current trump cards that are played. Later I also added a check to see if the partner has ruffed (played a trump). It also calculates how bold the player is based on their personality.
It then choses a card – the strongest trump card if bold and is first to trump but weakest if partner has ruffed and or is winning.
- Discarding: If no cards are playable that could garner a win, then discard the weakest card of the weakest suit.
I then changed this to if the player is supportive then play a signal card – one of a good suit – and high in pip to tell their partner their preferences otherwise just play a bad card of a bad suit.
- Conceding. I felt like the robots acted in a very un-human-like way, which my stakeholders expressed their concerns about how it felt unnerving. I first added a “morale” based on aggression and risk awareness and whether they are winning or losing and how badly they might be losing. If there is only one trick left and the player has low morale – no hope of a win – they'll concede.
If the player has low morale but many tricks left – 3 or less then they will enter a “soft play” mode where they will play lazily – make mistakes and play often not the best card.
I also added a “fatigue” which accumulates from playing and partly affects their personality over the course of a contract. Though this is a very minor change (it affects the traits by a maximum of 0.13) this can change the feel of the players by a lot.

Issues – The code snippets have been shortened to show important bits; there were a more issues - as I expected

Issue	Source	Fix
Lead suit mis-selection	Most of the time the bot will try lead with their longest suit – their strongest suit – but this doesn't update say there is 6 cards in this suit when there are 0 the bot will still try to play it.	This was simple and just meant that I changed the suit lengths when i removed a card from said suit.
This was tested with voids and worked – this reminded me how important it is to change data over time as it does change.		
Song card comparison	Two issues caused this bug. Firstly, the pips were stored as strings at some point so checks would be lexical ordering rather than numeric ordering hence `10` < `2` is true. The second issue is that aces had the pip of 1 which is the least so aces were viewed as the weakest card.	Converted all pips to numeric values before use so parse int Used the rank value function that returns a value = the pip or 14 if ace
Checked sequence when playing and Ace always now led in front of 10s and 7s...		
Signal overlap	When a supportive bot tried to signal a good suit, it sometimes picked a card marked as a dislike previously causing erratic behaviour and problems with choices	Added checks to exclude suits marked as disliked from signalling a like.
Behaviour became consistent as there were no more contradictory signals.		

Risk Aversion mis-scaling	Risk averse bots were more likely to play dangerously despite their aggression factor.	Changed checks from:
		Random < riskAversion
		to:
		Random > riskAversion
Testing many different personalities resulted in expected results.		
Soft play indexing error	During soft play the robot caused an error. This was because I was using Math.random() which works well but returns decimals	Using a Math.floor() function on the Math.random when getting a random card in the hand.
After de-moralising my opponents and playing many soft play tricks this issue didn't occur again.		

As this is only a heuristic opponent, and not a true AI, there will be several limitations and simplifications in how these players function and probably won't always play the best move. Below are some examples of what I might add if I have time at the end of the project but aren't necessary for a heuristic opponent (as heuristic means "good enough" and the bot only needs to be good enough to teach someone how to play and get someone better at the game – I could be coding a bot for years otherwise):

- No bidding conventions - bids are purely from hand's points and personality rather than standardised agreements
- No inference from bidding history – The bot doesn't react and infer moves from opponent's prior actions – and thus it doesn't try to understand the opponent's communications
- No planning – The bot selects each card now, during the current trick, based on current short-term data but doesn't react for future tricks – thus it also doesn't consider deceptive plays, or ducking
- Limited use of personality - there are only 4 factors that affect how different players would play a hand – they are enough to seem different but are very linear and limited so don't create complex emergent personalities or learn adaptively (fatigue is a very small change and preferably the entire personality would shift based on how their opponents have played)

Completing this completes S3

Iteration 5 – Multiplayer – S3

First part to creating multiplayer is creating lobbies – so players can create lobbies and join them – though for a start this will be a one player lobby. There will be lots of possible lobbies played at once – concurrent games.

I have though changed some things for example I have created another script into Src folder called lobbies.js which stores the Lobby class which is as follows:

Attributes	
game	This stores the game instance
players	This stores the socket.id of each player in said lobby
id	This stores the id of said lobby
started	This stores whether the rubber in said lobby has begun

Once these concurrent lobbies worked well, I then changed some code as before this I had msg as part of the response socket command which told the client what to output but I need a better more disassociated canvas when I convert this to a purer multiplayer, luckily this was also very simple – just sending data about the game back to the user. And the user's client.js can interpret this data and send this data to draw.js, where it is displayed (later draw.js will handle p5 code and actual graphics for now the graphics are an output field and an input field.). Below is the final separation of the responsibilities of the server vs the client:

Responsibility	Server	Client
Game logic	✓	
State tracking	✓	
UI, text, drawing, animations		✓
User Input		✓
Sending commands	✓	✓
Rendering results		✓

New application Protocol:

- Game:input
 - o From client to the server
 - o Sends the target that the player wants to interact with such that the server can understand them and commit a function according to what the player wants.
- Game:update
 - o From server to client
 - o {type, payload, state}
 - o Type:
 - Message, bid, trickPlayed, or score: these help the client understand how to understand their commands.
 - o Payload:
 - Information such as player, bid, card, clear?, etc
 - o State:
 - More information about the game, such as much data that can be sent to said player that shouldn't be hidden such as their hand, the dummy and their hand, the declarer, current player, game state, etc
 - This was later changed to just send any new data or data that is necessary for the client to have so less data was sent over the internet which i found to be an issue – I have a very poor Wi-Fi connection at home.

The code to receive this was later simplified using an event handler:

<pre>Const eventHandlers = { message: ({ text, clear }) => { if (clear) clearOutput(); output(text) }, bid: ui.showBid, trickPlayed: ui.showTrick, score: ui.showScore</pre>	This is the event handler, which will be used to simplify
---	---

};	
handler = eventHandlers[event.type];	This is how the client could figure out what to do based on said function.

This worked but introduced lots of errors and the code that has been created up to this point was very poor quality. Because of this, I rewrote server.js, client.js, and draw.js. The changes:

Server.js:

- Separated the code from each action – system code, action code, and output code
- I add JSDoc comments to each function to make them legible
- I changed what was outputted to the client via game:update

Client.js:

- Had the event handler – controlling three event types: `system`, `action`, `data`, `error`
- I also added a data JSON object to the client.js to store data such as the player's hand and what id they play
- Then I update the ui in draw.js

Draw.js:

- Stores an output buffer
- Controls order of outputs to: title, player hand?, dummy hand?, currentDeal?, output buffer?, player's turn?

This tangent being finished means that I had a better foundation for creating an actual multiplayer game, that won't cause lots of errors.

Although there were many errors, this was fast and simple. I firstly made a player map which mapped socket ids to seat ids and added these to the playable array and that was mostly it. Players were already together in a lobby. And data was already being sent to all the sockets separately and correctly. There were a lot of issues though especially where the number 0 being the player was hard baked into the code, and the following is a list of the issues plus their solutions:

I then had the issue that private data was only being sent to the player which did some action, so it loops through the players array and gets the socket via io.sockets.sockets.get(socketId) and outputs the player's data to all sockets using this.

I also had an issue where the dummy and declarer didn't always switch and that was because when i did switch them so that the player played the dummy's hand it checked if the declarer is 2 and the dummy is 0 which worked well when it was guaranteed to be the player is 0 and the bots are all else but now i had to change this to check instead if they are instance of heuristic player vs real player.

I then had a latency issue as I had mentioned before my Wi-Fi is poor so I predicted visual outcomes in the client then changed them to the server's outcomes – this will be more of a fix required when I introduce GFX and animations.

I then had an issue with hidden data. Initially Game:update exposed too much information, so I stripped it back to only the required or changed data that the client should know about being sent.

When a player disconnects mid-play, the lobby just froze so i had to replace them with a robot containing the same information - hand and had to run the robot loop.

There were also many smaller issues like syntax issues or ones that could be fixed by simple word replacement thus many aren't included here. There were still a few things to deal with multiplayer left but I decided best was to do these after GFX such as spectator support, matchmaking menus and settings, and reconnects.

This being done finalises S3 though there are still many changes to make later

Iteration 6 – GFX & SFX – M4 & S4 & S5

I decided the next best step was to create a good User Interface. So far, I've had a text field input and text field output which works but for more complex things such as setting up lobbies this will be limiting and this was hard for my stakeholders to understand and as this is also an app for people to learn Bridge, I figured this would be useful.

I only made one change to the server and that was the robot loop I added a second delay between actions using a promise and making the function asynchronous. The rest was done in client-side scripts.

I first deleted draw.js and most of index.html leaving a mostly blank canvas. I then imported both p5 and p5's audio addon. But this had a few issues

P5 issues:

- I have been using ESM for all imports and handling libraries, and I prefer ESM over global or normal, as it reduces issues and error, but ESM has been included in many P5 instances but only the recent ones. That wasn't an issue, and I used an import map for both P5 and socket and used those in the code.
- When I started coding, I realised I'd need the sound addon but there was no ESM compatible version that I could find and there was no version which were compatible with the main files which were ESM and if the sound file isn't ESM none of it can be ESM so I had to eventually convert to a global approach.

HTML

The HTML file uses 2 different parts – the body and the header

- The body contains the canvas and only the canvas – this is drawn on using p5.js

This is altered using CSS to: remove margins, padding, scrolling, and set the background colour and foreground colour to be nicer on the eyes – for if it takes a while to load. It also positions the canvas to the top left and full screen.

- The head is the section at the top of your browser and contains metadata

This holds meta tags to set the character set to UTF 8, and to name and set sizes of the viewport to the screen size

This also holds a <title> tag which shows the title Bridge.

Finally, this contains links – mostly to js scripts (p5.js, classes, and other scripts), but also to a favicon icon, linked via:

```
<link rel="icon" type="image/png" href="./Art/favicon.png" />
```

Art/favicon.png - 

I created this image using (favicon.io). Favicon.ico generates a series of files:

- Android-chrome-192x192,
- Android-chrome-512x512
- Apple-touch-icon
- Favicon
- Favicon-16x16
- **Favicon-32x32**
- And a site manifest.

I deleted all of them using just the 32x32 (highlighted above) as it works on most devices, whilst still looking good and being lightweight.

JS:

Here I could start coding using p5.js to draw onto the canvas and create the UI and UX.

First was to create the menu screen. This required a Create Lobby Button, Join Lobby Button, a Settings Button, game title and watermark, though this was very simple to code, it did require 3 buttons, and I knew I'd need to handle more buttons in the settings screen, the lobby creation screen, etc, so I had to figure out the best way to display a button.

Buttons:

- There are 3 options for buttons, and I chose probably the least conventional method, but it helped my brain understand it and I have the most control over how it looks and functions:

1. HTML

2. P5

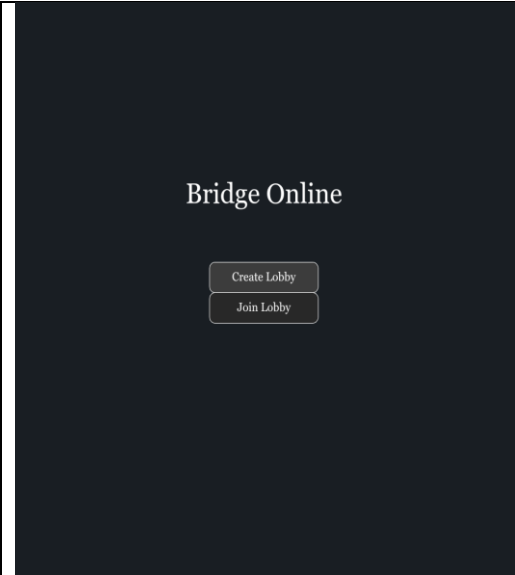
3. My own class. I chose this and it is as follows.

Button

Attributes	
Key	Purpose
Label	The label displayed on the button
x	Generic rectangle attributes (x,y) is centre position
y	
w	
h	
onClick	Callback function to run on button click
anim	0 by default increments when hovering – affects animation amount (0-1)
Methods	
IsHovered	Return if the mouse position is within the boundaries of the rectangle.
Update	Changes anim based on whether the button is hovered
Draw	Draws rectangle and text, colours and scale change based on the anim number
Click	Checks if the button is hovered then animates a click and calls the call-back function

I tested by:

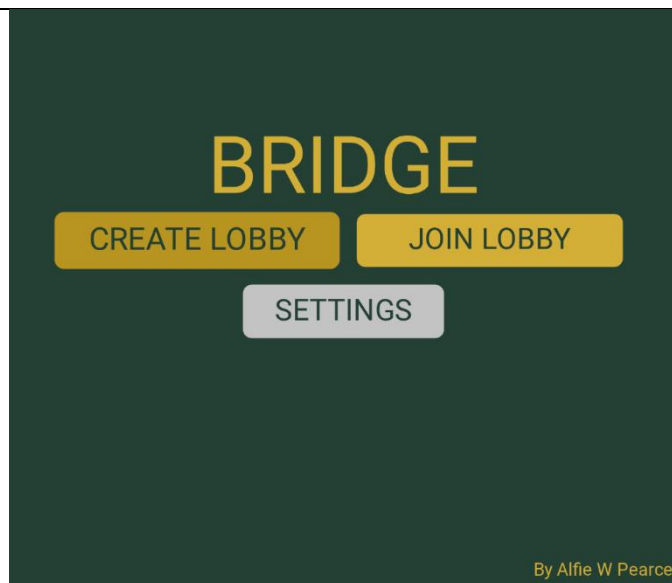
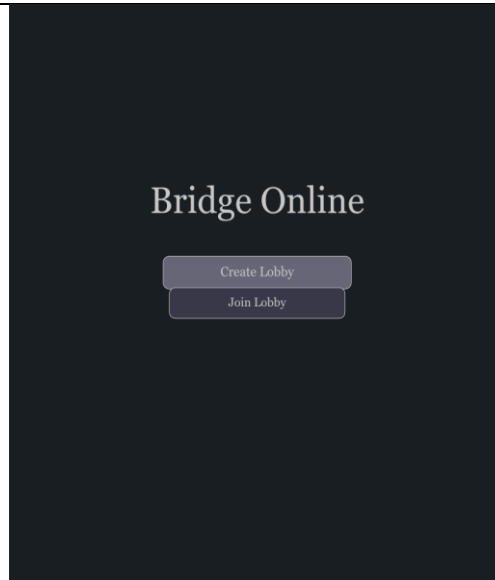
Test	Desired Outcome	Success
1	Creating lots of buttons in different locations and different sizes	TRUE
2	Checked if they always called their onclick function correctly	TRUE

	On the left is my initial UI design - it is simple, but it was a proof of concept really. You can see here that the create lobby button is being hovered by the mouse hence it is brighter.
--	---

On the right shows the first iteration based on stakeholder feedback.

They complained that:

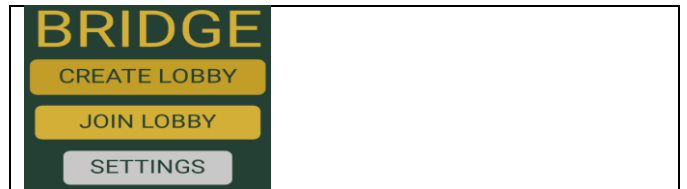
- Everything was too small
- Hovering should grow up a button



On the left shows the final main menu screen.

After showing more stakeholders the previous screen there were still some issues:

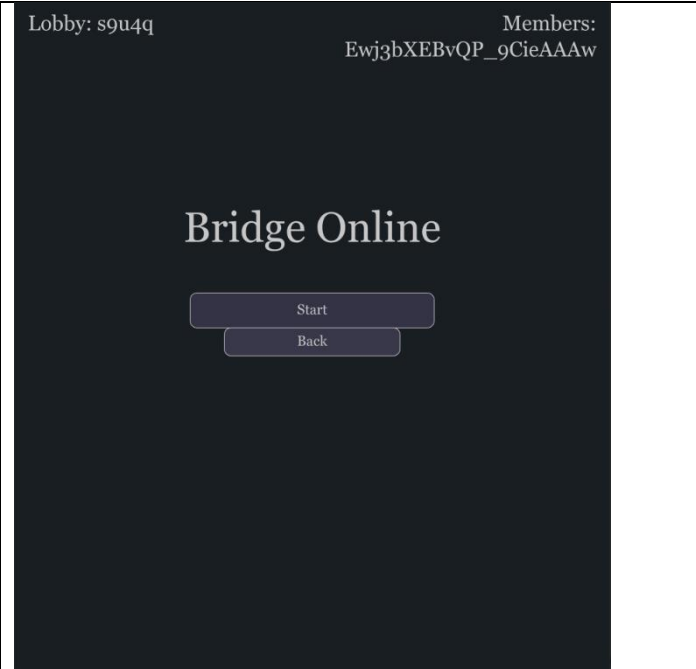
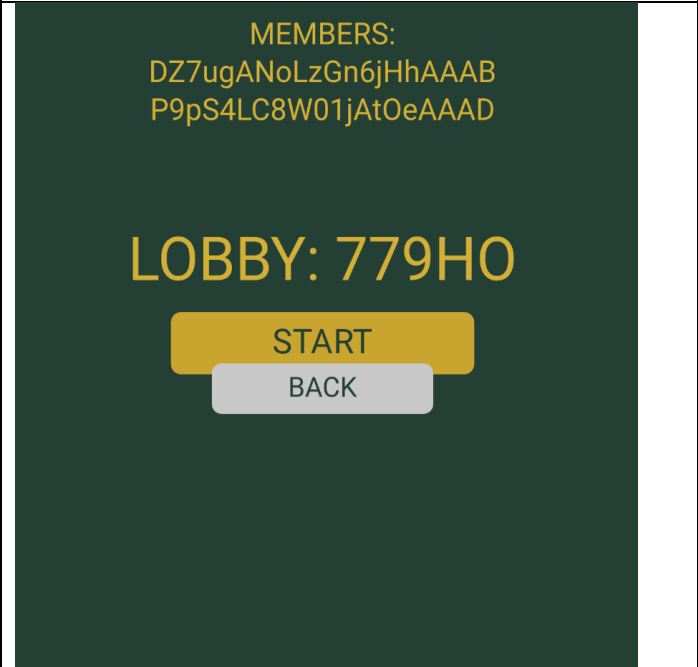
- The colouring didn't match the game of Bridge, so a green baize-like colour has been used for the background, and a golden colour has been used for the foreground elements.
- A watermark on the menu would be a plus
- A settings button and screen would be useful
- Text was hard to read – I changed to Roboto and to full caps
- Buttons were small – I made them even larger.
- On narrow screens this layout of buttons didn't work so I stacked them as follows if the screen is narrow:



Test	Desired Outcome	Success
3	All three buttons were hover able and clickable	TRUE
4	They console log when they are clicked – for now	TRUE

With the button class and the menu, the lobby creation screen was very straightforward and took a similar route.

The lobby creation screen is a start of C1

On the right is my proof-of-concept design for the lobby creation screen. It must at a minimum allow starting a lobby, leaving a lobby, showing members and the lobby id – though originally had just a start and back button.	
	On the left shows my final minimum lobby creation screen, though this also has gone through many changes: - Colours matching the menu screen - No watermark is necessary here - The title isn't necessary here - you know its Bridge - Having members at top right and lobby id at top left was a problem when narrow window so instead used the lobby id as the title and the member list at the top centre. - Text and buttons have also been changed to match the menu screen. There are some features still to change – I need to add the ability to select where each member sits, I need to add sectators and some lobby settings - such as bot difficulties and personality sliders.

Test	Desired Outcome	Success
5	Players can create a join a lobby via the previous screen	TRUE
6	Once players are in a lobby this screen appears	TRUE
7	Players can leave the lobby or start the game via the buttons	TRUE
8	Lobby id updates correctly	TRUE
9	Members list updates correctly if a player leaves or joins the lobby	TRUE

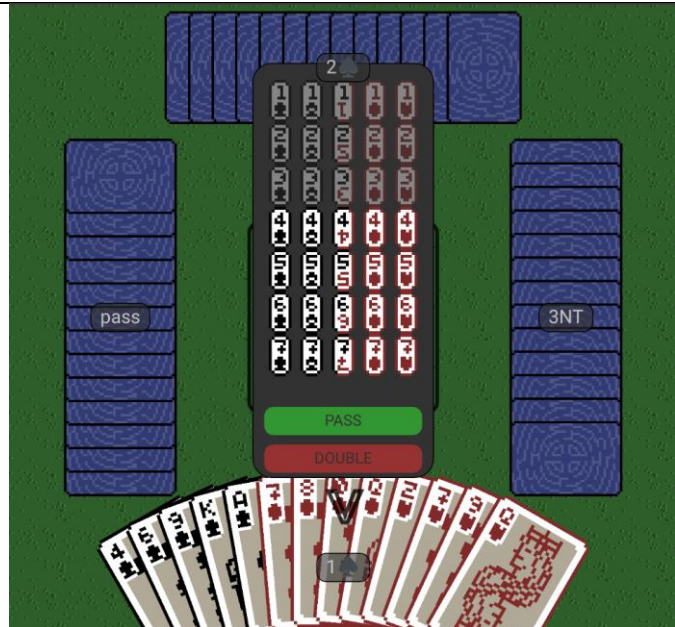
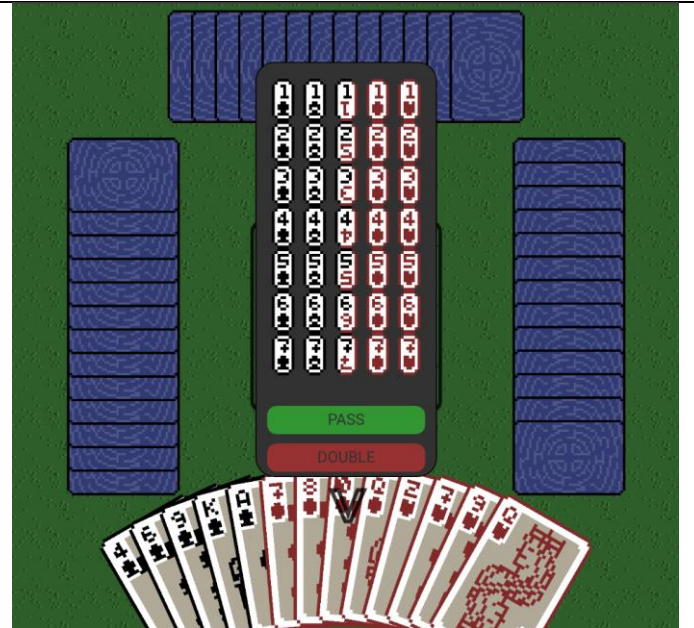
The first stage of gameplay is the auction phase. This needs to show your own hand as well as the table of possible bids.

This is the final iteration of the screen to show the auction phase; some changes I made are:

- Bigger player cards to all others
- The panel being higher than the centre point
- You can't bid with a level=0 – so I had to remove that line

Problems that the stakeholders want fixed:

- Pass and Double/Redouble buttons being too low – they were not changed when the panel was moved – a simple fix
- The arrow to indicate who's turn it is, isn't very clear, but for now is good enough.



This shows after a couple turns of bidding

- Levels 1-3 can't be played, hence are blacked out.
- It shows every player's most recent bid.

Test	Desired Outcome	Success
10	Bidding works correctly – tests from the bidding phase of the prior iteration	TRUE
11	Your hand always appears correctly – and to the south	TRUE
12	Past bids are correct	TRUE
13	The turn indicator points to the correct player	TRUE

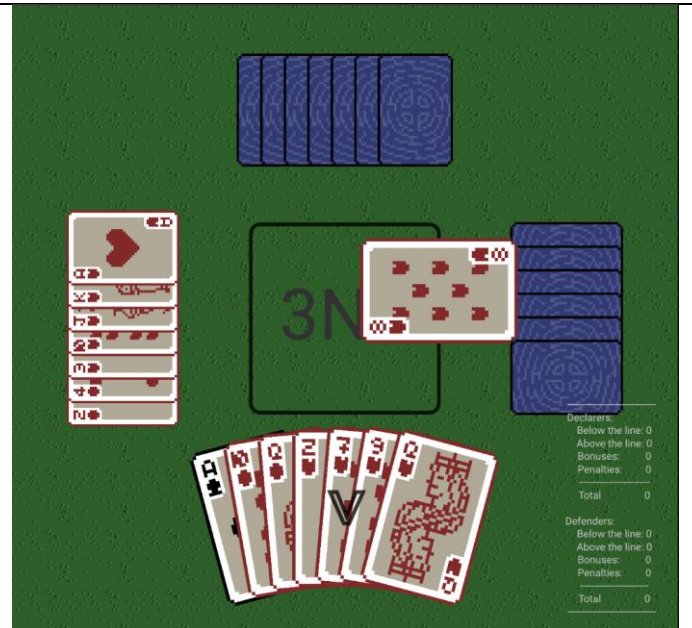
Trick-taking is the second stage of gameplay. This needs to show your hand, the dummy's hand, and the cards that are played by each in a trick.

This shows the screen part way through the trick. My stakeholders requested a few changes:

- Showing the cards in the centre
- Playing a form of animation to lerp them there
- Show the scoresheet in the bottom right – will change when I make the final scoresheet, so yes, it's hard to read but that will be fixed.

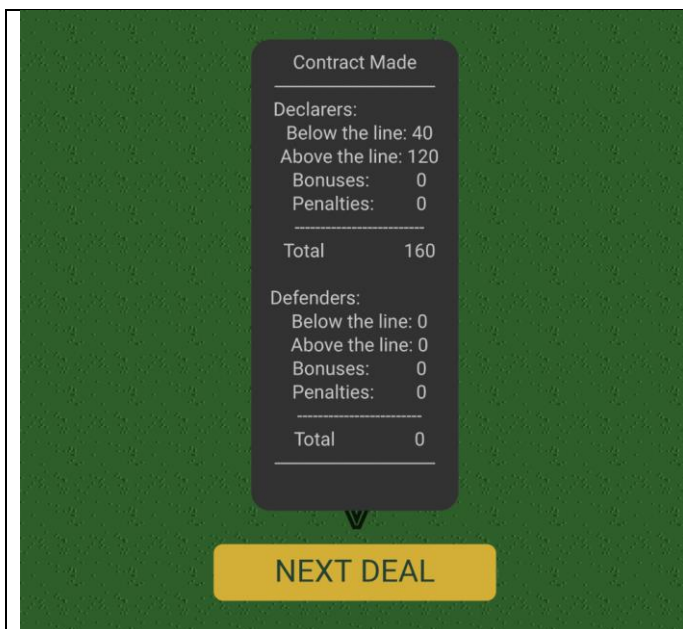
They have also requested:

- A way to hide the scoresheet – it takes up too much screen real estate
- A way for the bid to be always visible
- The centre should show names, avatars, the deal, the amount of tricks won / number of tricks required by both pairs.
- A clearer and neater indicator of who is playing.
- A concede button – when they give up or when the game runs on too long.



Test	Desired Outcome	Success
14	Dummy's hand appears correctly	TRUE
15	Cards are correctly placed in the centre	TRUE
16	The trick rules are followed – tests mentioned in a prior iteration	TRUE
17	The score is updated correctly – tests mentioned in a prior iteration	TRUE

The scoresheet is an important thing which must be shown at the end of every trick and should always be visible on the screen.



This shows the score sheet right now. All it shows is the summation of each section of the score – this was only chosen as this is currently how the scores are handled.

My stakeholders would prefer a more traditional Rubber Bridge scoresheet.

It also has a next deal button so people can start the next deal when they are ready.

This screenshot shows an issue – the turn indicator should be hidden when it's showing the scoresheet.

Test	Desired Outcome	Success
18	Scoresheet appears after every deal concludes	TRUE
19	Scoresheet updates correctly for every client	TRUE
20	The next deal button works correctly	TRUE

Displaying the hands

The hands and central layout are made, yet my stakeholders still hold many complaints about it and there are some things I feel like needs to change as well:

- Show deal throughout the whole trick as well the progress of completing the contract by both pairs
- Show the hand more visibly – less overlap
- Handle resizing better – have a minimum size and change card scale of opponents if the size is too small, likewise show the borders of the table if the size is too large.
- Show Avatars and Names – can use random for now
- Place cards in the centre more neatly
- Indicate who's turn it is more neatly
- Have a tutorial button

I first added a header. The point of this header was to show some information which may be obscured or feel weird to be displayed onto the table. The final requirements of this header will be having the current deal, how many more tricks each pair must win to be able to win this contract and having two buttons: concede, and tutorial. My header started being large – taking up too much screen real estate, so I rotated this about 10 degrees, but my stakeholders complained that this was too much and didn't match the art of the rest of the game, so I rotated it flat and shrunk it. I also moved the current deal from the centre where it would be obscured to the centre middle of this.

I then added a border if the screen gets too large, so the cards don't spread out so far.

I then shrunk the opponents' cards as they were too large.

I then replaced the central console with just the card placed into the middle, removing what I had previously.

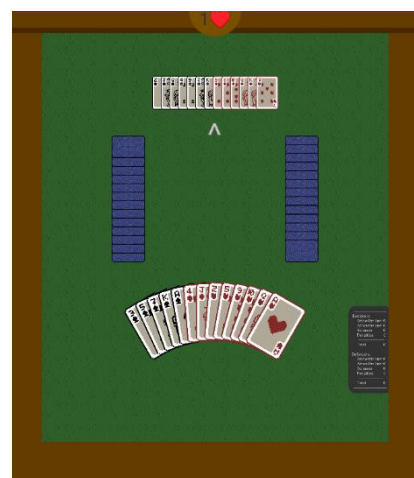
I also added a background to the scoresheet – though this will eventually show a more generic Rubber Bridge scoresheet.

Test	Desired Outcome	Success
1	The correct deal appears in the header	TRUE
2	Changing the size of the screen scales everything correctly and never hides or overlaps anything	TRUE



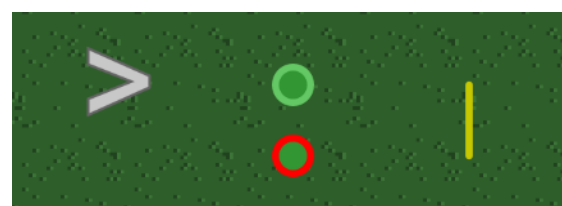
To the left is my first iteration of redoing this. Showing the change to the header, centre, scoresheet and opponent cards.

On the right shows the table's borders – this screen has been scaled up using the developer tools' responsive emulator.



Next my stakeholders have asked for the header to show progress of the current trick as well as showing whose turn it is clearer – for now I am going to make a simple circle for each player’s avatar, and this will be highlighted to show your turn.

To the right, are the three examples I showed my stakeholders – A) is what I currently have which is an arrow to point at them, they didn’t like how much space this took up and how easy this was to miss; C) is a line to highlight your cards but this would be tricky with a fan and my stakeholders didn’t like it, so finally B) the one my stakeholders prefer is your avatar (green circle) has an outline which changes if it’s your turn.



Centre Console

To make the central console I first made a new folder and script to experiment with.

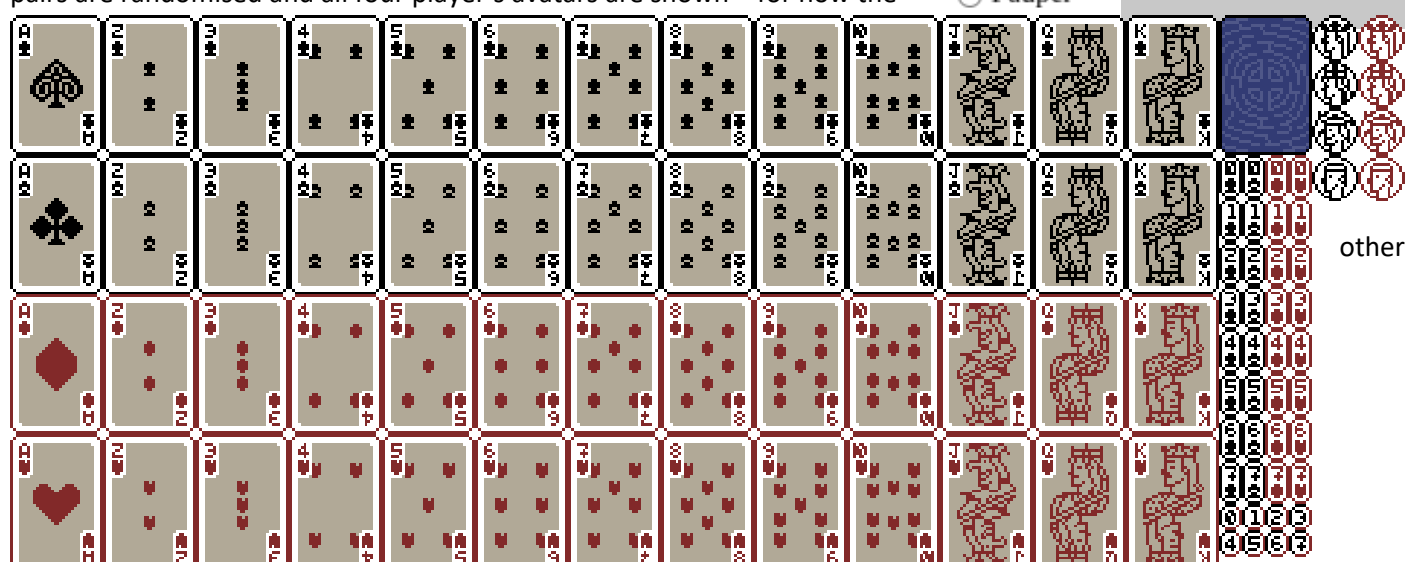
Avatars

First, I needed to make player avatars as I found this was helpful in multiple existing implementations. My stakeholders talked to me and i made a list of requirements for these:

- Don’t have racial characteristics
- Do include gender representation
- Do allow lots of customisation capacity
- Do keep them simple and following the same art style

I chose a simple male and female face design – rendered in a minimal line art – based on the cards’ face cards’ design I made on Aseprite. This eventually was changed to also include a joker and a pauper, just to improve customisation capacity. This was chosen because firstly, it matches the style of the cards, and it removes racial characteristics – reducing the risk of bias or stereotyping, while still providing gender representation to address potential concerns about inclusivity. Each partnership is assigned a distinct colour – black for one pair and red for the other – which visually reinforces team identity and matches the colours used on the card faces. This maintains immersion without requiring user-uploaded images, improving safety and keeping the aesthetic consistent across the game, my stakeholder Kade had the idea of converting this pair’s colour and avatars to your favicon icon and I’ll do this if I have time at the end.

These were also drawn using Aseprite and below is the updated sprite atlas – in which to the right is the avatars from top to bottom being king, queen, joker, pauper, and this picture is very blurred as this is an enlarged PNG file. I added these to the menu code and to the right is both when the iteration is in paused and played state (above and below respectively), where when you are in pause menu you can select the gender and when you press play the colour of the pairs are randomised and all four player’s avatars are shown – for now the



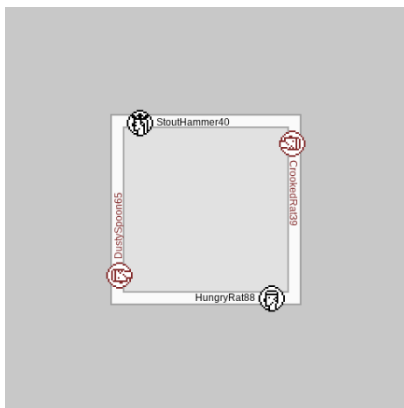
three players have random choice between the 4 avatars.

Names

My First thought was to use a text field for the names, but this caused a security risk for the game would be multiplayer and this would require sanitisation. Because of this I looked at other examples predominantly Kahoot with its adjective then noun being for names. Below is the dictionary which controls the name generator:

<pre>const namePools = { King: { adj: ["Grand", "Iron", "Golden", "Stout", "Crimson"], noun: ["Crown", "Sceptre", "Throne", "Banner", "Hammer"] }, Queen: { adj: ["Silver", "Velvet", "Scarlet", "Silent", "Emerald"], noun: ["Rose", "Lace", "Moon", "Crown", "Mirror"] }, Pauper: { adj: ["Dusty", "Hungry", "Patchy", "Crooked", "Rusty"], noun: ["Sock", "Boot", "Bow", "Spoon", "Rat"] }, Joker: { adj: ["Mad", "Wild", "Sneaky", "Twisty", "Baffled"], noun: ["Banana", "Card", "Dice", "Goose", "Duck"] } };</pre>	This has a name pool per your avatar (character). It then randomly chooses an adjective from the list. And randomly chooses a noun from the list. Finally, it chooses a random number from 10 to 99 – as I noticed most name generators use numbers, and they'd be more unique as there is a rather high chance that two players can get the same name otherwise.
---	---

I then positioned the name next to the avatar and placed this around a central area where the tricks will be played. This is visible below and to the left:



Then, I added these the graphics of the rest of the game I placed them just off from the player's hand and above and to the right is a screenshot to show where these are positioned. Making this work on multiplayer just required storing the avatars and names in a dictionary in the server – which is made on game start and sent to the player.

I then added a yellow (matching the menu screen) border to display around these for when it is said player's turn.

Test	Desired Outcome	Success
3	Display the avatars and names	TRUE
4	Rotate the avatars and names correctly according to each user's seat	TRUE
5	Display the current player highlighter correctly	TRUE

To test score handling and displaying the score I made a new project just to input data and draw the resultant score card.

Originally, I had the penalty shown on the other pair's side as the result of a 'we' penalty is added to the 'they' points, but Kade mentioned how this looked confusing.

- though these are only used in this side project, and the main one has these calculations already built in.

75 | DEVELOPMENT – ACTUAL ITERATIONS

- If level = 7 then the slam bonus of 1,000 is added or 1,500 if vulnerable

Test	Desired Outcome	Success
1	Score updates correctly as according to the rules above	TRUE
2	Game lines are drawn in correct positions	TRUE
3	Rubbers ending is handled correctly and the correct user wins	TRUE

This builds on the prior part of M4 as the scorecard now can be shown in a way that my stakeholders' like.

Introducing this to my coursework was very simple I just had to, on the client: change the draw score sheet function to copy that which was done on this other project; on the server: making a new score object, which contains some metadata and a rounds array. The meta data holds data such as whether the rubber is complete and what the totals are. The rounds store who the declarer is and what above, below and penalty points are received during each round.

Iteration 6.2 – Audio – S5

Background Music

During one of the times that I played Bridge with my stakeholders, we found that there can be awkward silent pauses while players think through their next move. We discussed adding a timer, but I felt this would undermine the casual social nature of the game – something essential for beginners learning Bridge – so a timer isn't a viable option, for now,

Another suggestion was to add subtle background music, since we often listen to music when playing physical games of Bridge.

I began by composing a short piano riff, experimenting with melodies in D minor before shifting to Dorian D for a brighter feel. Once I had a full track, I arranged and notated it in flat.io (sheet music shown on the right).



Bridge

The backing track to the game Bridge
Score created with the free version of Flat - <https://flat.io>

Alfie Pearce

2

Score created with the free version of Flat - <https://flat.io>



Loading the audio file in p5.js was straightforward – the process is very similar to loading an image file:

```
SoundFile = loadSound(`/Audio/Bridge.wav`);
```

Playing it was equally simple with:

```
SoundFile.loop();  
SoundFile.amp(0.4); //Quietens the backing music
```

However, I hit a limitation when I placed the *loop()* call inside P5's *setup()* function. Due to modern browser security rules, audio cannot auto-play on page load. Attempting to do so triggers the following warning:

p5.sound.min.js:2 The AudioContext was not allowed to start. It must be resumed (or created) after a user gesture on the page.

This means playback must be triggered by user interaction, such as a mouse click or keypress. My solution was to instead start the music when you start your game. I might later make lobby music - so music playing on the lobby creation screen – but not the main menu as this will still require some form of interaction.

Sound Effects

The next part of audio that I had to handle was haptic feedback and immersion – and sound effects are one of the easiest ways to handle both. I decided on two sound effects:

1. The haptic click - This is a sound which provides haptic feedback when the user clicks a button. I chose the *Walkman Button* from ([pixabay](#)) created by *soundsforstory*.
2. The card sound - This is a sound which mostly provides immersive feedback and is used during animations (like dealing) and for extra feedback during gameplay like when a trick is played. This is the *Card Sounds* also from ([pixabay](#)) created by *henrygillard (Freesound)*.

Loading these sound files was identical to how loaded the background music. Playing them, however, was another story. I noticed a delay between the user's clicks and the sound playing. My first theory was that the audio file has a silent lead-in, so I experimented with changing the *cuestart* value to skip ahead in the file. But no matter how I tweaked it, the delay persisted – so I began suspecting latency.

I tried preloading the sound effect and restarting rather than reloading it each time, which helped but not enough and the delay was still noticeable. Because of this I decided to bypass P5.js for sound effects entirely, using the Web Audio API directly. I found (Perfect Timing and Latency, n.d.) to be an excellent reference.

Interestingly, the Web Audio API is what P5.js uses under the hood, so theoretically, there should be no difference in latency. That's when I realised my real mistake: my *cuestart* values were far too small – multiple powers of ten out – and on a one-second audio file, fractions of a second matter a lot. Changing *cuestart* to skip further into the file instantly solved the problem. So, I converted back to P5, and the results play smoothly, with minimal code, and taught an important lesson – that the simplest solution (maths) is often correct, and often it is the numbers that are wrong.

This did have a problem during testing though, where if another player plays a card the swipe sound didn't play – I forgot to include this. But here I had a debate to ask my stakeholders: would they prefer only your card swipe sound play or all cards' swipe sounds play. They had originally said all cards should play the sound but after a prototype they found it was too much, so it only plays when it's your card swipe.

Test	Desired Outcome	Success
1	Click sound plays immediately after clicking any button	TRUE
2	Card swipe sound plays immediately after this user clicks on a card	TRUE
3	Background music loops correctly whilst playing a game	TRUE

Iteration 7 - WAN

I already had most of the backbone to handle cross-platform, but I now needed this server to run through a URL which isn't a local host.

I also had an issue that this VS Code is being run on a Linux Virtual Machine, which means that the IP address it runs through is only accessible on the VM. To avoid this, I couldn't use an IP URL so instead I used ngrok

To download ngrok locally was a simple node command: `npm install ngrok`

Then I made an account and added this authtoken to the ngrok. To add this token, I just needed to run a simple command: `ngrok config add-authtoken <TOKEN>`

Then to run this I use: `npx ngrok http 3000`

This routes <http://localhost:3000> through another URL like <https://example.ngrok.app>, which can be run on other hardware – which has access to the internet. My Chromebook cannot run this URL though as there are lots of safety setups on a Chromebook, so I must run localhost on my Chromebook, which isn't a problem as ngrok just routes traffic to and from localhost.

To test this, I started the server and started ngrok. From there, I loaded the URL on my phone and tablet and ran the localhost on my Chromebook twice to play a game with these 4 clients, and the full game went as expected.

Iteration 8 – Clean-up

Up to now the code has been added to and removed from many times. Throughout there are deprecated functions, variables and verbose code. The next step was logically to clean up all the code. I have done this a few times throughout the creation of the project such as cleaning up the Server.js script before creating a GUI though this will explain how this step happened every time and show the sort of issues, I came across throughout.

The main things I did was:

- Creating dictionaries and new scripts to separate functions and variables in a more maintainable way.
- Creating comments – normal single line comment and JSDoc comments to explain each function.
- Creating whitespace – often the code lacked any causing it to be hard to read.
- Reducing verbosity – often the code was verbose or included functions which weren't necessary or code which was repeated and often there were lists of if statements and other such bits of code which could be simplified. Often long bits of code could be replaced with js array functions like `sort()` or p5 functions like `map()`.
- Simplified data structures – a lot of the main data such as data on the client and lobbies and game on the server had repeated data values, so these have been simplified – e.g., there was `lobby.players`, `lobby.playerMap`, `lobby.game.players`, `lobby.game.playable`.

A specific Example:

Bidding could parse an input in the format of either a card (object), ``pass`` (string), or ``double`` (string). This was handled before via a nested if statement but this was not only hard to read but also seemed rather verbose so instead, I made an individual function for each of these possibilities and placed them within a `bidType` object, such that these could be called via `bidType.object` – this reduces global namespace clutter.

Further Enhancements:

Earlier in this project I had downloaded Prettier. This was useful as it helped me maintain a more consistent coding format. For this a new JSON file was made called `.prettierrc`, which controlled how the formatting should be, such as single quotes or double quotes, semi-colons, and brackets around parameters of arrow functions.

With this I turned on format on save and made a script in node's package.json called `format` which run the command: `"npx prettier --write ."`.

After doing these changes, I still felt like my code could be neatened. To do this I downloaded two further extensions: code metrics and eslint:

Code metrics is simple it calculates the complexity of a function, method, if-statement and warns if it is above certain thresholds which led to better, neater functions that were more maintainable and easier to explain using JSDoc comments.

Eslint reports on problematic patterns, bugs and style issues, this is very useful at pointing out repeated code and unused code. This does have the issue though that by default it doesn't work with prettier – they try to fight, so I had to run: `"npm install --save-dev eslint-config-prettier"` and add it to the extends tag in the `.eslintrc` file. With this I made a second in node's package.json called `lint` which run the command: `"npx eslint . --fix"`.

I also wrote a third and final script, which sped up the cleaning processes, called `clean` and it runs both above: `"npm run format && npm run lint"`. So now I could clean and review a lot of code at once using the command `"npm run clean"`.

This shouldn't cause any issues with the code, but I did repeat every test mentioned above and received the same positive results

Testing

Throughout the development of my implementation of the game of Bridge, I ran many tests to make sure that each feature works. Now that the coding is complete, I must test the functionality and the robustness of my project.

Evidence of each test's success will be viewed in the final video – of which timestamps will be mentioned. Some tests may require small tweaks to the code – e.g., `console.log(result)`

C1 – Lobby Setup & C2A – Private Lobbies

Objective not fully attempted. due to time constraints – Players may create basic lobbies and join with a code, but deeper configuration options are unfinished.

When a lobby event occurs (join/leave/delete), all connected clients are expected to update, ensuring a synchronised lobby list and seat availability.

Test ID	Description	Successful Outcome	Success & Timestamp
C1.1	Create Lobby	Player can create a lobby – This can be joined by other players such that they can play a game together	0:06
C1.2	Leave Lobby	Player can leave a lobby – removing them from the list and all clients get an update	0:08
C1.3	Delete Lobby	When there are no more players the lobby is deleted	0:15
C2A.1	Unique Lobby	Lobby Code is always unique	0:11
C2A.2	Join Lobby	Player can join the lobby using the lobby code – sending a message to all other clients in this lobby	0:22
C2A.3	Max Players	Only 4 players can join a lobby – as there are only 4 seats	0:27
C2A.4	Leave Game	If player leaves mid-game player's seat is filled by a bot – all clients will update	0:42

M1 – Dealing

The system must create and shuffle a standard 52-card deck, then deal 13 cards to each player, in a completely fair method so no one gets an unfair advantage.

Test ID	Description	Successful Outcome	Success & Timestamp
M1.1	Shuffle a new deck	Console logs a purely random deck on game load – and after 4 pass bids	0:50
M1.2	Receive a random hand	Each hand received is unique, random, and different across games	0:50

0:50

M2 – Bidding

There must be an auction phase in which players must be able to bid, pass, double, or redouble culminating in a final contract.

Test ID	Description	Successful Outcome	Success & Timestamp
M2.1	4 passes	Auction ends – deck is reshuffled and hands are dealt correctly	1:05
M2.2	3 passes	Final contract becomes the last non-pass bid	1:18
M2.3	Double / Redouble	The contract is x or xx when (dbl or rdbl)	1:11

M3 - Trick-Taking

Players take turns playing cards, following Bridge conventions, so unable to place illegal cards.

After each 4-card trick a winner is identified and leads the next trick. After all the 13 tricks are played, the scorecard is updated and shown.

Test ID	Description	Successful Outcome	Success & Timestamp
Can't play			
M3.1	Take turn	Player cannot play when it isn't their turn – UI will show an error	1:25
M3.2.1	Dummy can't play	Dummy player cannot play when it is or isn't their turn	1:32
Can play			
M3.2.2	Dummy controlled by declarer	Declaring player can play the dummy's hand during the dummy's turn	1:37
M3.3	Your turn	The game visually indicates if it is your turn and allows you to play a card	1:37
M3.4	Winning tricks	After 4 cards are played the correct player wins the trick and the next lead is assigned correctly	1:43

M4 – Scorecard & M5 - Games & Rubbers

There must be a scorecard UI displaying current scores: overtricks, bonuses, undertricks, and separating above / below the line scores.

Test ID	Description	Successful Outcome	Success & Timestamp
M4.1	Correct score	The correct resultant score parts appear on the scorecard	1:51
M4.2	Scorecard appears	The scorecard appears immediately after concluding a contract	1:48
M4.3	Scorecard updates	The scorecard in the bottom right updates after a contract is done	1:51
M5.1	Game lines	Game lines are drawn correctly and consistently	1:58

M6 – Random Opponents & S1 – Heuristic `AI`

If real players are unavailable, the game needs to fill the seats with random opponents (the quality of them does not matter here) – furthered in S1

Test ID	Description	Successful Outcome	Success & Timestamp
M6.1	Opponents play	Bots automatically bid and play a legal move	2:02
S1.1	Large capacity	Bots can play any bid, pass, dbl or rdbl and can play cards when it's their turn – choosing non-extreme bids and plays	2:02

S2 – Server & S3 - Client

The server should control core logic (e.g., dealing, scoring and handling hidden information), enabling synchronous multiplayer games. This should allow multiple people to play in the same game.

The client should provide communicate with the server allowing people, from across the world, to access the same game if they have internet.

Each user will have their own view with ability to see their hand but preventing access to hidden details such as opponents' cards.

Test ID	Description	Successful Outcome	Success & Timestamp
S2.1	Multiplayer	Multiple users can join any given lobby and play a game of Bridge	2:23
S2.2	Hidden data	Data that should be hidden shouldn't even be sent to the client – which will store all data in app.data so can be found through the console	2:38
S3.1	Cross-platform	Players anywhere on any device see different real-time views – only being able to see own hand, and can interact with them	2:23

S4 – Audio

UI should include immersion-helping and feedback-helping sound effects as well as background music

Test ID	Description	Successful Outcome	Success & Timestamp
S4.1	Click	Every UI interaction causes the click sound	3:07
S4.2	Card	Every Card interaction causes the card swipe sound	3:11
S4.3	Background	While playing the game the main music loops	3:18

S5 – Animation

Objective not fully attempted. I ran out of time – but I have got toast animations

Test ID	Description	Successful Outcome	Success & Timestamp
S5.1	Error	Red toast animations appear and disappear on error	2:58
S5.2	Information	Blue toast animations appear and disappear on information such as player joining / leaving	2:51
S5.3	Multiple	Stacking lots of toasts on top of each other should work	2:58

C2B – Public Lobby

Objective not attempted – Only private lobbies exist.

C3 – Tutorial

Objective not attempted

C4 – User Profiles

Objective not attempted

C5 – Friend List & Quick Invite

Objective not attempted

C6 – Rubber Records

Objective not attempted

W1 – AR/VR

Objective not attempted

W2 – AI

Objective not attempted

W3 – Variants

Objective not attempted

Stakeholder Testing

I know the game works as intended and is robust (the tests above prove that), but it is very important to gain external opinions and feedback as other people will have different assessments of how well this project works. I will ask my stakeholders to answer questions for this. Firstly, this will be Kade, then a series of classmates probably 2 who study Computer Science and 1 who does not. The following page will show the questionnaire that I will give them.

Game Testing

Thank you for your involvement in the testing of my variation of Bridge. Your feedback will be important, as it will influence every decision I make going forward, so your honesty and your time will be greatly appreciated by me! I will be with you for your first time playing this, and once I have loaded the server you may use the following link to test the game and play against your friends if you wish:

<https://unmarketable-elliott-nondangerous.ngrok-free.dev>

User Interface

You will be greeted by the menu screen – you can either create a lobby or join one using a lobby-id (supplied by someone who created one).

What do you think of the overall UI?	
How intuitive was the lobby process?	

Any other general comments?	
-----------------------------	--

Game Play

You know the rules of Bridge, for you have played it before. As you play think about the following questions:

Was it clear who's who and whose turn it is?	
How was the gameplay?	
How clear is the scorecard?	
How was the audio? Did you feel like it was helpful or immersive	

Any other general comments?	
-----------------------------	--

Summary

Unfortunately, I have run out of time for the development of this game. So:

What features do you believe that the game needs?	
Were there any features you found to be especially fun / useful?	
Was the game fun to play?	

Stakeholder Results

User Interface

UI	1. I think the UI and UX felt clean & like I was up a table. 2. The header is too large considering it didn't show much info, but what is shown there is good. 3. The UI was intuitive, but the art could be simplified – as such, sometimes, it did confuse me. 4. Art is cool, especially the card art, but spades and clubs are too similar.
Lobby	1. I think creating & joining the lobby was very simple, but the lobby screen was too minimal. 2. It was not clear that the code was needed to join a lobby – I need more to inform me. 3. Everything was clear and easy to use, but why is there a cap of people, why not spectators? 4. The lobby setup screen wasn't a setup screen, but the overall process was simple.
Other	1. N/A 2. The Baize pattern needs to be toggleable – it confused me with too much detail on screen. 3. I appreciate the cards' borders and pip references but the detail on the cards was too much. 4. The buttons were good – good haptic feedback – I like how, with lag, I know that I pressed it.

Game Play

Turns	1. The indicator of who's turn it was very clear & I like the avatars that are highlighted. 2. The toast was a good indicator of it not being your turn, but maybe there should be more. 3. It wasn't clear who's turn it was – I had to get someone to show me the avatar highlights. 4. I liked the avatars and names, but I kept on pressing, not on my turn, when bidding.
Gameplay	1. The game loop was clear & simple to use; I really liked the dummy swapping as well. 2. During the game, there could have been some tutorial to talk about the specific UI, but it did feel like I was at a table playing Bridge like I use to, so all in all the gameplay was good. 3. The Rubber took too long to play, so there should be the possibility to concede a game. 4. The games felt fast paced and I liked the card placements in the centre of the 'table'
Scorecard	1. The scorecard was clear & it updated nicely, but penalties were written too high on the card. 2. I appreciated how it was like a normal Rubber scorecard, but what about 3 Rubbers? 3. The scorecard could be simplified maybe points-below-line per game and points-above-line. 4. I liked the scorecard, but could you click the card, during play, causing it to appear full-size?
Audio	1. The sound effects were nice, and the music was moody, I'd enjoy listening to just that. 2. I think the card swipe should play on all players' play but the music and haptics were nice. 3. I appreciate the haptic sounds, but the music was too much – a settings screen is essential. 4. I love that music.
Other	1. N/A 2. There should have been a way to see the active suit better – maybe click and that suit moves. 3. There should be a settings feature for music and sound effects, and new gameplay features. 4. Maybe there should be a way to change the time for bots to move – 0-2s

Summary

Needed features	1. After clicking on a scorecard, during play, the scorecard should expand, as it is hard to read. 2. The game needs a tutorial. 3. There should be a settings screen. 4. The game needs a friend and user profile system.
Good Features	1. I loved the music and sound effects & I really liked the dummy switching mechanic. 2. That AI was good, because I struggled against them. 3. The header showing how many tricks each pair has won compared to the amount they need. 4. The avatars and names were a nice touch – especially that it matched the cards' art-style.
Fun	1. The game was great, but I felt more excited by the music 2. I loved how much it felt like I was playing a normal game of Rubber again with real people. 3. I felt it lacked a lot of necessary toggleable features, but the art was cool. 4. Very fun and thrilling, just hard to see sometimes

Evaluation

Success Criteria

Essential Gameplay – M1/2/3/4/5/6

All the criteria in this section were met successfully. This is shown in the tests: M1.1-2, M2.1-3, M3.1-4, M4.1-3, M5.1, M6.1. The stakeholders feel that the “UI and UX [are] clean” and that they “[were] actually up a table”, saying that the game is “great” and “felt fast paced”. Overall, there seems to be mostly positive comments about the core gameplay, but some negative comments did crop up. “The [game] took too long to play” and “there could have been [a] tutorial”, as well as some missing quality-of-life features. Because of these, I can add some simple features:

- A concede button (a button in the header, which calls the concede function – already implemented in game.js),
- A UI explanation overview mode (a button in the header, which shows an overlay that explains the UI elements),
- Some actions for viewing a specific suit (click a card and said suit moves forward and becomes more visible – messed around with in early iterations).
- Settings (button in main menu and button in header and during lobby-creation).

Lobbies – S2/3 | C1/2A/2B

This was mostly completed, because there is a server and client, which can communicate (shown in tests: S2.1-2, S3); there is a private lobby (shown in tests: C1.1-3, C2A.1-4); but there is no public lobby or lobby settings feature, so a lot of features - that should be necessary with lobbies – haven’t actually been implemented. My stakeholders agree that “the lobby screen [is] too minimal” and “not clear” enough and it wasn’t really a “setup screen”. But they have also said that it was “simple” and intuitive. To improve this, I need to add:

- A settings screen (mentioned above and in the future developments),
- A bit more information (explain everything more clearly, where text is shown),
- A spectator’s system (let more players join and they can see all hands but not play any)
- A public lobby system (a list, maybe ordered by geographical distance, shows public lobby ids – which the player can click to join {join(thisId)}).

Smart Opponents – M6 | S1

This was completed fully, as indicated in tests M6.1 and S1.1. Though these opponents are intelligent, they still lack a lot of depth, which I would add if I had more later. The stakeholders mentioned that they felt “like human”, so the opponent wasn’t bad, but it still lacked some depth. To fix this, I should add some bidding conventions, as well as reactions to the dummy’s hand and more reactions to their partner, so overall introduce more heuristic factors to influence the decisions of the bot – using more if statements.

Server & Client – S2/3

The server is fully functional, and the clients are handled separately, as shown in tests S2.1-2 & S3.1. My stakeholders appreciate how “fast” and “clear” their client UI is, and Kade has told me before that she “[likes] how some of the data is hidden, [so even if someone can code,] they can’t access this hidden data, like my hand”. I believe some of the server and client details could be further separated – such as the client storing less data and the server caring less about rendering – maybe instead just send all data (that the user is allowed to see – maybe a public data dictionary, a private data dictionary per client and send these merged to the correct client) at the end of each turn / round / etc.

UI & UX – M4 | S4/5

This was mostly completed, for audio was complete, as mentioned in tests S4.1-3 and the UI works – This is shown in almost all tests but tests S5.1-3 showed the toast animations working which were my only animations – I didn't do more animations (such as dealing cards and throwing a card into the pile for playing a trick). I would add these later, but these would require large changes to the client.

My stakeholders have mentioned that the UI is “simple”, “clean”, “intuitive” and “clear”. This sounds all great but there are many holes in the UI. It can be “not clear [when joining] a lobby” and there needs to be “a settings screen” and the header needs some features too: “a concede button” and “a tutorial button” and overall is “too large” for its current purpose (taking up too much screen real-estate) – but it was appreciated. I would:

- Redo the header (making it shorter),
- Create a settings screen and a lobby setup screen
- Add some features like concede and tutorial,
- Make some things clearer (differentiate clubs from spades more and simplify the art and add some more text).

User Profiles – C4/5/6

This feature was not attempted but would have been greatly appreciated by my stakeholders if I had included and have mentioned that the game “needs a friend and user profile system” and as such my next feature, I would make, is a user profile feature. These need to have a username, password, normal avatar, friends, and records of how many rubbers played and won.

Tutorial & Help – C3

This feature is not attempted but would have been greatly useful. A rather large point of this project was to teach people how to play Bridge – which is why I made the servers with any number from 1-4 players and the seats will always be filled, but there is no better teaching aid than a tutorial – so I should add a tutorial button to the header as some of my stakeholders have mentioned.

Complex Additional Features – W1/2/3

These have not been attempted in the slightest and would vastly change the application and its purpose. Therefore, these criteria are and will remain completely unmet.

Assessment of Usability Features

The game was designed to be as easy to use as possible. As such all the user inputs are simple, bold and large buttons. My stakeholders did mention that these were “easy” and “intuitive” to use. There are 4 screens to the game, and I got feedback on each one during testing: Menu, Login, the game and the scorecard.

The Menu

This presented the user with a title, watermark and 3 buttons: Create Lobby, Join Lobby, and Settings. Players had no issues navigating this screen, though many have expressed concerns about how there should be “a settings screen” - as currently the button does nothing. This should be addressed by implementing a functional settings screen, rather than removing the button, as the expectation of customisation is reasonable.

Joining a lobby caused some initial confusion, as the source of the lobby ID was not clearly communicated. Once this was understood, the interaction was considered “simple”. To improve usability, I would add contextual guidance – such as helper text or tooltip explaining where the lobby code is generated – reducing reliance on external explanation.

The Lobby

This shows the members list, the lobby code and 2 buttons (start game and back). The “back button [should have been] renamed to leave lobby” and there should have been more options to setup on this screen. Even with this my stakeholders mentioned this was “simple” and “clear”, but this may have been “too simple”, and I need to add more settings on this screen – such as a public lobby setting, and I should add some tooltip animations. Future improvements would include:

- Lobby settings (e.g., public/private lobbies)
- Clearer host indicators
- Lightweight tooltip animations explaining each option and button.

The Game

The game’s UI was often mentioned to be “simple”, “clean” and “intuitive”. My stakeholder appreciated the “dummy swapping” and “[feeling] like [one] was actually up a table,” “against friends.” The game screen seems to be very successful especially in how simple this application is to use. Some constructive feedback was also mentioned:

- “The art could be simplified,” “spades and clubs are too similar” – some of the art was possibly too complex and in too small a space, so hard to see.
- “There should be a settings feature” – a lot of features were either liked or disliked, so to be toggleable would be beneficial.

The Scorecard

This seems to be good, being described as: “simple”, “clean”, and “easy to read”; the scorecard just lacks a few features:

- “The scorecard [...] is hard to read” – during the gameplay, the scorecard appears in the bottom right, but there should be a feature to expand this to full screen.
- “What about 3 Rubbers” – during production of this application, I seemed to forget that most of the time when players played Contract Bridge, they didn’t just play one Rubber, so I should the ability to play multiple Rubbers ~3.
- “Could be simplified” – Another possible setting: ability to show just points below line in current game and in the points above the line.
- “The penalties were written too high” – they did interfere with the We/They text, so I updated this.

Overall, the stakeholder comments were overwhelmingly positive, with most criticism focusing on neither confusion nor difficulty but on lacking features that would further enhance usability. This suggests the core UI and UX design is sound, and that future development should focus on extension and refinement rather than redesign.

Robustness

There are three aspects to the robustness of the game These are data entries (inputs), turns, and client counts (number of users).

Data Entries

All User Inputs are either a button or text input.

- Text inputs only appear once – in entering the join id. This input is trimmed and set to lower case. Then it is used in only one place – and that is when checking it against current lobbies. Because of this, no matter what you enter this field – you have a simple Boolean – either you can join a given lobby or no lobbies of said id exist. This has been proven to both work and be robust in test C1.2.
- All other user inputs are buttons and handled through mouse clicks or touch events. This is tested and proved to work in almost every test.

Turns

During the game, you cannot play when it isn't your turn. This is proven to work in tests M3.1-3. This is just a simple check and is tested many times as it is important that this rule isn't broken.

Client Count

If there are lots of players, the server may slow or even stop. The state of the game cannot be affected by this.

Limitations

Despite the projects strengths there are three notable limitations of this project as it stands:

No User Profiles

There are no user profiles which would be a database that stores (usernames, avatars, win amount, win streak, play amount, etc). This is a large feature, as without it, there is: no possibility of leader boards; a lack of customisation capacity; and no friend systems so inviting people must be done through them reading the code externally. These are all features that my stakeholders have requested, and it is a rather simple addition, so, in the future, this could be added using a databasing library, like SQLite. This will require a registration / login page to create an account, and this would naturally justify the need for a settings screen, which is mentioned below.

No Settings / Setup

A lot of features are wanted or hated, but without a settings screen, user control is limited. Stakeholders also mentioned the importance of customisation. A settings screen for the lobby setup would also be very helpful – and this could allow the user to set the lobby to be either public or private and configure other parameters before play begins.

No Tutorial

This is arguably the most significant limitation of this project, as one of the primary aims of this implementation of Bridge is to teach new people how to play. While AI players and flexible players counts help support this goal, they do not replace a guided tutorial. A structured tutorial or interactive UI guide would significantly improve the educational value of the application and was explicitly requested by stakeholders.

Maintenance Issues

Currently the game list on the server stores all active games. This works well and this deletes old, deprecated games. This is important as otherwise this list will get too long. This still has that issue though as if there are many games this list can become very long, and this can slow or even crash a server. This can be fixed though by distributing the clients (load balancing) through multiple servers – which can be done innately in ngrok using the tag: '--pooling-enabled'

There is a lot of code, across many scripts and even with lots of scripts, esm, comments and aptly named functions and variables – this can be very hard to read and understand, purely because how much there is. This will be a problem only really if someone had to update the code in the future for whatever reason. Though, saying this, there

are lots of comments and I could add many more JSDoc comments that would make it easier to read – and the code is split up into many aptly named scripts and functions, so it could be a lot harder to read.

Another possible issue is external libraries. This implementation uses ngrok, node, socket, express & p5 which each one could fail or become deprecated, for whatever reason, so need to be updated.

If I include User Profiles later, that will be a further maintenance issue. Firstly, the database must be maintained – so preferably stored in the long term on a series of multiple servers. Furthermore, this could cause a few GDPR issues:

- According to the (GDPR Art. 5, 2016) (1.e) states that data about someone can only be stored for “no longer than is necessary for the purposes for which the personal data are processed”. This means that if the account is not accessed for a long-enough amount of time, like a year, then the data will be deleted – maybe just have some code which checks once a month if someone has been inactive from more than 24 months if so then warn them and a week later delete their record.
- The data subject also must have the ability to delete all data about them, according to Art. 17 of the GDPR (Right to erasure (‘right to be forgotten’). This is a very simple delete account button which will just delete the record.
- Thirdly, all data stored about the data subject must be able to be requested, by the subject, according to Art.15 of the GDPR (Right of access by the data subject).
- Fourthly, all data must be kept up to date and accurate according to (GDPR Art. 5, 2016) (1.d). This and the latter issue can easily be averted by having a “my details” section where you can change any data when you wish to.

Future Developments

The core and first feature I would implement is the User Profiles. I would create a flat-file database using SQLite. I would then add a registration option to the main menu – in which people will register with an email and a password. From here the menu will show the username in a typeable text box (which any input will be sanitised and check for crude language using an API like Leo-Profanity). Also, the four types of avatars will be selectable and that will become the user’s default – king, queen, joker, pauper. This will be selectable with 4 buttons side-by-side in the menu, which is shown in the UI development section. This will require additional messages to be sent between the clients and the server but should be fine to follow the current standard procedure.

The second feature that I would develop is the tutorial as this project requires a bit more to be able to teach someone how to play the game. I would add a button to the header, like TUTORIAL and this button will show an overlay which will explain each element of the UI and what it does and what you should put in – e.g., explaining how one should play an auction.

The final feature that I would develop is the lobby setup system. This will allow me to implement public lobbies and set-up friend inviting, seat position, and spectator mechanics, as well as the ability to set specific stats for the bots.

This concludes my project write up! Thank you for reading and goodnight!

Bibliography

- 24/7 Games LLC. (n.d.). 247. Retrieved from Bridge: <https://www.247bridge.com/>
- American Contract Bridge League. (2014). *Contact Bridge - Wikipedia*. Wikipedia. Retrieved from https://en.wikipedia.org/wiki/Contract_bridge
- Bridge Base On Line. (n.d.). *Bridge Base*. Retrieved from Just Play Bridge: <https://www.bridgebase.com/>
- Downey, T. (2025, November 2). *A Brief History of Bridge*. Retrieved from haroldschogger: <https://www.haroldschogger.com/history.htm>
- FunBridge. (2025). *Minibridge*. Retrieved from Fun Bridge: <https://play.funbridge.com/minibridge>
- GOTO Games. (2025). *FunBridge / How to play bridge*. Retrieved from <https://www.funbridge.com/how-to-play-bridge>
- No Fear Bridge. (2013). *RUBBER BRIDGE SCORING*. Retrieved from https://www.nofearbridge.co.uk/acol/rubber_scoring.pdf
- NODESOURCE. (2025, 05). *nodesource/distributions*. Retrieved from GitHub: <https://github.com/nodesource/distributions>
- Perfect Timing and Latency*. (n.d.). Retrieved from Web Audio API: https://webaudioapi.com/book/Web_Audio_API_Boris_Smus_html/ch02.html
- Rauðás Games. (n.d.). *Cardgames.io*. Retrieved from Bridge: <https://cardgames.io/bridge/>
- shawnomancy . (2025). *Pirate Bridge: Nullo Bids*. Retrieved from <https://shawnomancy.com/gaming/pirate-bridge/nullo/>
- Socket.IO. (2025, 08 22). *Rooms*. Retrieved from Socket.IO: <https://socket.io/docs/v3/rooms/>
- The Bridge World Magazine. (2025). *LESSON 2: Bridge Scoring*. Retrieved from <https://www.bridgeworld.com/pages/learn/beginners/lesson2.html>
- The Official Encyclopedia of Bridge. (2001). *Wikipedia Chicago (Bridge card game)*. Retrieved from [https://en.wikipedia.org/wiki/Chicago_\(bridge_card_game\)](https://en.wikipedia.org/wiki/Chicago_(bridge_card_game))
- W3Schools. (2025). *image*. Retrieved from p5: <https://p5js.org/reference/p5/image/>
- W3Schools. (2025). *noSmooth*. Retrieved from p5: <https://p5js.org/reference/p5/noSmooth/>
- W3Schools. (2025). *preload*. Retrieved from p5: <https://p5js.org/reference/p5/preload/>
- W3Schools. (2025). *push*. Retrieved from p5: <https://p5js.org/reference/p5/push/>
- W3Schools. (2025). *W3Schools*. Retrieved from JavaScript Array Sort: https://www.w3schools.com/js/js_array_sort.asp
- Wikipedia. (2025, September 2). *Bridge convention*. Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Bridge_convention
- Wikipedia. (2025, 09 1). *Display aspect ratio*. Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Display_aspect_ratio
- Wikipedia. (2025, October 4). *Heuristic*. Retrieved from Wikipedia: <https://en.wikipedia.org/wiki/Heuristic>

Appendices

Final Hierarchy – scripts and files

node_modules/

...

.prettierrc

```
{
  "semi": true,
  "singleQuote": false,
  "jsxSingleQuote": false,
  "quoteProps": "as-needed",
  "trailingComma": "es5",
  "tabWidth": 8,
  "useTabs": true,
  "printWidth": 1000,
  "bracketSpacing": true,
  "bracketSameLine": false,
  "arrowParens": "always",
  "proseWrap": "preserve",
  "endOfLine": "lf",
  "htmlWhitespaceSensitivity": "css",
  "vueIndentScriptAndStyle": false,
  "embeddedLanguageFormatting": "auto",
  "singleAttributePerLine": false,
  "plugins": ["prettier-plugin-organize-imports", "prettier-plugin-packagejson"]
}
```

Eslint.config.js

```
import js from "@eslint/js";
```

```

import prettier from "eslint-config-prettier";
import { defineConfig } from "eslint/config";
import globals from "globals";

export default defineConfig([
  js.configs.recommended,
  prettier,
  {
    files: ["**/*.js,mjs,cjs"],
    plugins: { js },
    extends: ["js/recommended"],
    languageOptions: {
      globals: {
        ...globals.browser,
        ...globals.node,
        ...globals.p5,
      },
    },
    rules: {
      "no-unused-vars": "warn",
      "no-undef": "off",
    },
  },
]);

```

Package.json

```

{
  "name": "bridge",
  "version": "1.0.0",
  "description": "Bridge CS Coursework Game",
  "keywords": [],
  "license": "ISC",

```



```

"author": "Alfie Pearce",
"type": "module",
"main": "server.js",
"scripts": {
  "clean": "npm run lint && npm run format",
  "format": "prettier --write .",
  "lint": "eslint --fix",
  "start": "node server.js"
},
"dependencies": {
  "express": "^4.18.2",
  "ngrok": "^5.0.0-beta.2",
  "socket.io": "^4.7.2"
},
"devDependencies": {
  "@eslint/js": "^9.39.1",
  "eslint": "^9.39.1",
  "eslint-config-prettier": "^10.1.8",
  "globals": "^16.5.0",
  "prettier": "^3.6.2",
  "prettier-plugin-jsdoc": "^1.5.0",
  "prettier-plugin-organize-imports": "^4.3.0",
  "prettier-plugin-packagejson": "^2.5.19"
}
}

```

Package-lock.json

...

```

Server.js
/*
    server.js
    Main server script - sets up server and controls application protocol
*/

// Imports

//Server Imports && Libraries
import express from "express";
import { createServer } from "http";
import path from "path";
import { Server } from "socket.io";
import { fileURLToPath } from "url";

//Multi-Script Imports
import { Game } from "../Src/game.js";
import { Lobby } from "../Src/lobbies.js";
import { HeuristicPlayer, RealPlayer } from "../Src/player.js";

//Server Setup

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const httpServer = createServer(app);
const io = new Server(httpServer);

//serve static ( client ) files
app.use(express.static(path.join(__dirname, "Public")));

```

```

//Global State

//Stores all active lobbies on the server
//Each lobby has a unique id and a game instance once started - look at Src/lobbies.js
const games = {}; // { id:Lobby-instance }

//Helper Functions

//Finds a lobby that the player (id) belongs to
const findLobbyById = (id) => Object.values(games).find((l) => l.players.includes(id)) || null;
//Parse command and arguments: e.g. `join(1234) -> cmd=`join`, args=[`1234`]
const splitActionStr = (plr, actionStr) => {
  const [cmd, argStr = ``] = actionStr.split(`(`);
  return {
    cmd,
    args: argStr
      .replace(`)` , ``)
      .split(`,`)
      .map((a) => a.trim())
      .filter(Boolean),
  };
};

//Socket Logic
io.on("connection", (socket) => {
  console.log(`${socket.id} connected`);

  socket.on("game:input", (actionStr) => {
    if (!actionStr) return;

    //Calculates what room this socket is supposed to be in

```

```

let lobby = findLobbyById(socket.id);
let newLobby = null;
let targetLobby = newLobby;
let success = false;
let result;

const { cmd, args } = splitActionStr(lobby?.playerMap?.[socket.id] ?? socket.id, actionStr);
switch (cmd) {
  case `create`:
    newLobby = system.createLobby();
    //Fall through -> join
  case `join`:
    targetLobby = newLobby || games[args?.[0]?.toUpperCase?.()];
    lobby = system.joinLobby(socket, targetLobby, lobby, newLobby?.id || args?.[0]);
    if (!lobby) return;

    output.emit(socket, lobby, `data`, { lobbyId: lobby.id });
    output.broadcast(lobby, `system`, { msg: `${socket.id}: joined` });
    output.broadcast(lobby, `data`, { members: lobby.players });
    break;

  case `leave`:
    system.disconnectSocket(socket);
    output.emit(socket, null, `data`, { lobbyId: null });
    break;

  case `next`:
    result = lobby.game?.nextDeal();
    if (result?.success) {
      output.broadcast(lobby, `system`, result);
      output.start(lobby);
      action.robotLoop(lobby);
    }
  }
}

```

```

        } else output.error(socket, lobby, result?.msg || `Invalid next deal`);
        break;

    case `start`:
        system.startGame(lobby, socket);
        //Output empty scoresheet
        if (lobby?.game?.score) output.broadcast(lobby, `data`, { score: lobby.game.score });
        //Fall through to handle robot player actions
    default: //A player action affecting game
        success = action.play(socket, lobby, cmd, args);
        if (success === true) action.robotLoop(lobby);
    }
});

socket.on("disconnect", () => {
    console.log(`${socket.id} disconnected`);
    system.disconnectSocket(socket);
});

});

//System Functions
const system = {
    /**
     * Creates a new lobby and add it to the global list
     *
     * @returns { Lobby} Reference to the lobby created
     */
    createLobby: () => {
        //Create Lobby
        const lobby = new Lobby(); //Creates Lobby
        games[lobby.id] = lobby; //Pushes to lobby list
    }
};

```

```

    console.log(`Lobby ${lobby.id}: created`);

    return lobby;
  },

  /**
   * Joins a lobby if it exists
   *
   * @param { Socket} socket The player's socket
   * @param { Lobby} lobby The Player's chosen lobby id
   * @returns { Lobby | null} The lobby if there is no error
   */
  joinLobby: (socket, lobby, oldLobby, id) => {
    if (!lobby) return output.error(socket, lobby, `Lobby ${id} doesn't exist`); //No lobby found
    if (lobby.game) return output.error(socket, lobby, `You can not join an already started game`); //Lobby
    has started
    if (lobby.players.length > 3) return output.error(socket, lobby, `Lobby ${id} is already full`); //Lobby
    is full

    //Disconnect player from any prior lobby
    if (oldLobby) system.disconnectSocket(socket); //Disconnect from old lobby

    //Make join code case insensitive
    id = id.toUpperCase();

    //Adds player to lobby
    games[id].players.push(socket.id);
    socket.join(id); //Handles SocketIO's rooms

    console.log(`Lobby ${id}: ${socket.id} joined`);
    return games[id];
  },

```

```

/**
 * Deletes lobby if it exists and is empty
 *
 * @param { Lobby } lobby The lobby to delete
 * @returns { true|null }
 */
deleteLobby: (lobby) => {
  if (lobby.players.length > 0) return false; //Lobby not empty

  //Delete lobby
  delete games[lobby.id];
  console.log(`Lobby ${lobby.id}: deleted`);

  return true;
},

/**
 * Disconnects the socket from their lobby if there is one and deletes it if empty
 *
 * @param {Socket} socket The Player's socket connection
 */
disconnectSocket: (socket) => {
  const lobby = findLobbyById(socket.id);
  if (!lobby) return;

  //Remove Player
  lobby.players = lobby.players.filter((p) => p !== socket.id);
  const seatId = lobby.playerMap[socket.id];
  delete lobby.playerMap[socket.id];

  //Delete empty lobby

```

```

    if (system.deleteLobby(lobby)) return;

    output.broadcast(lobby, `system`, { msg: `${socket.id}: left` });
    output.broadcast(lobby, `data`, { members: lobby.players });
    console.log(`Lobby ${lobby.id}: ${socket.id} left`);

    //Replace with bot if mid game
    if (lobby.game && lobby.game.dummy && seatId == lobby.game.dummy) return;
    else if (lobby.game && seatId != undefined) {
        const oldHand = lobby.game.players[seatId].hand;
        const bot = new HeuristicPlayer(seatId, 2, Math.random(), Math.random(), Math.random());
        bot.hand = [...oldHand];

        lobby.game.players[seatId] = bot;
        lobby.game.playable = lobby.game.playable.filter((p) => p !== seatId);

        output.broadcast(lobby, `system`, { msg: `Seat ${seatId + 1} replaced by new bot` });
        action.robotLoop(lobby);
    }
},

/**
 * Starts a lobby if it exists and isn't already started
 *
 * @param {Lobby} lobby The lobby to start
 * @returns {Game | null} Returns the game created if a game can be started
 */
startGame: (lobby, socket) => {
    if (!lobby) return output.error(socket, lobby, `You need to be in a lobby`); //Requires a lobby to start
    a game
    if (lobby.started) return output.error(socket, lobby, `Game already started`); //Game is already started

```



```

    //Setup lobby's player map
    lobby.players.forEach((socketId, idx) => (lobby.playerMap[socketId] = idx));
    output.broadcast(lobby, `data`, { playerMap: lobby.playerMap });

    //Start game
    lobby.started = true;
    lobby.game = new Game(lobby.playerMap); //Makes a new game instance

    console.log(`Lobby ${lobby.id}: started`);
    output.broadcast(lobby, `system`, { msg: `Game started`, clear: true });

    //Setup player names and avatars
    lobby.game.newPlayerAvatars();
    output.broadcast(lobby, `data`, { playerAvatars: lobby.game.playerAvatars });

    output.start(lobby);

    return lobby.game;
  },
};

//Action Functions
const action = {
  /**
   * Gets the action for 0-3 robots' turns and plays them
   *
   * @param {Lobby} lobby The robot's lobby
   */
  robotLoop: async (lobby) => {
    //Repeats for each of the players
    while (!(lobby.game.players[lobby.game.currentPlayer] instanceof RealPlayer)) {
      await new Promise((resolve) => setTimeout(resolve, 1000)); //Waits a second between turns
    }
  }
};

```

```

        const move = lobby.game.robotAction(); //Get robot's action
        if (!move) break; //No move returned

        const { cmd, args } = splitActionStr(`Robot`, move);
        action.play(null, lobby, cmd, args);
    }
},

/**
 * Plays the player's turn if possible then plays as many robot turns as it can
 *
 * @param {Socket | null} socket The player's socket
 * @param {Lobby} lobby The player's lobby
 * @param {string} cmd The command the player wants to play
 * @param {string[]} args The arguments parsed into the command such as suit,pip
 * @returns {boolean | null} Whether the action was a success or not
 */
play: (socket, lobby, cmd, args) => {
    //Validation
    if (cmd == `start`) return true; //Player isn't doing an action when they start
    if (!lobby) return output.error(socket, lobby, `You need to be in a lobby`); //Player needs to be in a lobby
    if (!lobby.started) return output.error(socket, lobby, `You need to start game`); //Player needs to start game

    const playerId = socket ? lobby.playerMap[socket.id] : null;

    if (socket && lobby.game.state == 2 && playerId === lobby.game.dummy) return output.emit(socket, lobby, `error`, null, `Dummy cannot play during this phase`); //Dummy cannot play during this phase
    if (socket && playerId !== lobby.game.currentPlayer && playerId !== lobby.dummyPlayedBy) return output.error(socket, lobby, `Not your turn`); //Not this socket's turn

```

```

//Stores some stuff about the game before the turn
const prevState = lobby.game.state;
const prevPlayer = lobby.game.currentPlayer;

//Process Player Action
const response = lobby.game.processActionFromNetwork(cmd, args);

if (response.game) lobby.game = response.game;

//Send scores
output.broadcast(lobby, `data`, { score: lobby.game.score, tricksWon: lobby.game.tricksWon });

//handle phase transition
const biddingEnded = prevState == 1 && lobby.game.state == 2;
if (biddingEnded) {
  //Reveal Dummy hand
  output.broadcast(lobby, `data`, {
    dummyPlr: lobby.game.dummy,
    dummyHand: lobby.game.players[lobby.game.dummy].hand,
    currentDeal: lobby.game.currentDeal,
  });
  //Send each player their updated hand
  Object.entries(lobby.playerMap)
    .map(([socketId, playerId]) => [io.sockets.sockets.get(socketId), playerId])
    .filter(([socket]) => socket)
    .forEach(([socket, playerId]) => output.emit(socket, lobby, `data`, { currentHand:
      lobby.game.players[playerId].hand }));

  //Set dummy handler
  lobby.dummyPlayedBy = (lobby.game.dummy + 2) % 4;
  const humanDummy = lobby.game.playable.includes(lobby.dummyPlayedBy);
  if (humanDummy) {

```

```

        //Dummy played by human
        lobby.game.playable.push(lobby.game.dummy);
    }
}
const trickTakingEnded = prevState == 2 && lobby.game.state == 3;
if (trickTakingEnded) {
    output.broadcast(lobby, `data`, {
        handLengths: [0, 0, 0, 0],
        currentTrick: [],
        dummyPlr: null,
    });
}

//Output Updates
if (lobby.game.state == 2) {
    output.broadcast(lobby, `data`, {
        declarer: lobby.game.declarer,
        currentTrick: lobby.game.currentTrick,
        handOwner: prevPlayer,
        cardCount: lobby.game.players[prevPlayer].hand.length,
    });
}

//Handle Invalid Action
if (!response.success) {
    if (socket)
        output.error(socket, lobby, response.msg); //emit error
    else console.error(`error robot move unacceptable: ${response.msg}`); //Error robot messed up
    return false;
}

//Broadcast Action Summary

```

```

output.broadcast(lobby, `action`, { bid: response?.bid ?? null, scoreTxt: response?.scoreTxt ?? ``, msg:
response.msg, clear: response.clear }); //emit message

//Output Dummy's hand if it changed
if (prevPlayer == lobby.game.dummy) output.broadcast(lobby, `data`, { dummyHand:
lobby.game.players[lobby.game.dummy].hand });

//Output Player's hand
if (socket && lobby.playerMap[socket.id] == prevPlayer) {
    output.emit(socket, lobby, `data`, { currentHand: lobby.game.players[prevPlayer].hand });
}

return true;
},
};

//Server (Socket) output Functions
const output = {
    /**
     * Broadcasts a message to an entire lobby
     *
     * @param {Lobby} lobby The lobby to broadcast to
     * @param {string} type The type of message to broadcast
     * @param {JSON} payload The contents to broadcast to the lobby
     */
    broadcast: (lobby, type, payload) => {
        io.to(lobby.id).emit(`game:update`, {
            type,
            phase: lobby.game?.state ?? -1,
            currentPlayer: lobby.game?.currentPlayer ?? 0,
            payload,
        });
    };
};

```

```

},

/**
 * Sends a secret message to just this socket containing secret information or an error
 *
 * @param {Socket} socket The player's socket connection to emit to
 * @param {string} type The type of emission such as error or hand
 * @param {JSON} [payload={}] The content to be sent. Default is `{}`
 * @param {string} [errorMsg=''] The error - if there is one. Default is ``
 */
emit: (socket, lobby, type, payload = {}, errorMsg = '') => {
  socket.emit(`game:update`, { type, payload, errorMsg, currentPlayer: lobby?.game?.currentPlayer ?? 0 });
},

error: (socket, lobby, msg) => output.emit(socket, lobby, `error`, null, msg),

start: (lobby) => {
  lobby.game.playable = [...Array(Object.keys(lobby.playerMap).length).keys()];

  let payload = {
    phase: 1,
    handLengths: [13, 13, 13, 13],
    currentTrick: [],
    bid: [null, null, null, null],
    dummyPlr: null,
  };

  //Send each player their data
  for (const [socketId, playerId] of Object.entries(lobby.playerMap)) {
    const playerHand = lobby.game.players[playerId].hand;
    const playerSocket = io.sockets.sockets.get(socketId);
    if (!playerSocket) continue;
    payload.colour = lobby.game.colour[playerId % 2];
  }
}

```

```
        console.log(lobby.game.colour, playerId, payload.colour);
        payload.currentHand = playerHand;
        output.emit(playerSocket, lobby, `data`, payload);
    }
},
};

//Start Server
const PORT = process.env.PORT || 3000;

httpServer.listen(PORT, () => {
    console.log(`Server running at http://localhost:\${PORT}`);
});

//Run using npm start or node server.js
//Run on wan by then running npx ngrok http 3000
```

Src/ - server-side scripts

Src/lobbies.js

```
/*
    lobbies.js
    Contains the class for a lobby
    exports { Lobby }
*/

/** Represents a 1-4 player lobby */
class Lobby {
    constructor() {
        //the game class
        this.game = null;
        //the list of player sockets
        this.players = [];
        //the map for player sockets to seat ids
        this.playerMap = {};
        //whom the dummy is played by
        this.dummyPlayedBy = -1;
        //the lobby's unique id
        this.id = Math.random().toString(36).slice(2, 7).toUpperCase();
        //whether the game is started
        this.started = false;
    }
}

export { Lobby };
```

Src/player.js

```
/*
    player.js
    Contains the player class and its variants
*/
```



```

    exports { RealPlayer, RandomPlayer, HeuristicPlayer }
*/

//Imports
import { rankValue } from "../game.js";

//Parent class
class Player {
    constructor(id) {
        this.id = id;
        this.hand = [];
    }

    bid() {
        throw new Error(`bid() not implemented for base Player`);
    }

    trick() {
        throw new Error(`trick() not implemented for base Player`);
    }

    observeSignal() {
        throw new Error(`observeSignal() not implemented for base Player`);
    }
}

//Sub class - for multiplayer and real players
class RealPlayer extends Player {
    constructor(id) {
        super(id);
    }
}

```

```

//Bid returns action said player wants to do
//Real players will return null as the server needs to wait for their action
bid() {
    return null;
}

//Trick returns action said player wants to do
//Real players will return null as the server needs to wait for their action
trick() {
    return null;
}
}

//Difficulty -1 - totally random but legal plays
class RandomPlayer extends Player {
    constructor(id) {
        super(id);
    }

    //Bid returns action said player wants to do
    //Random players will return to pass if they aren't the dealer - if they are then they'll make a random bid
    bid(game) {
        //Checks if there is a prior bid and if so will pass
        if (game.currentDeal.trump != null) return `bid(pass)`;

        //There is no prior deal thus will open
        const trump = Math.floor(Math.random() * 5); // 0-4
        const num = Math.floor(Math.random() * 7) + 1; // 1-7

        //Will return their deal
        return `bid(${trump},${num})`;
    }
}

```

```

//Trick returns action said player wants to do
//Random players will find any card they can legally play and return it
trick(game) {
    const legalCard =
        this.hand.find((c) => c.suit == game.leadSuit) || // lead suit
        this.hand.find((c) => c.suit == game.currentDeal.trump) || // trump suit
        this.hand[0]; // other
    //Return their legal card
    return `trick(${legalCard.suit},${legalCard.pip})`;
}

}

//Difficulty 0-2 - not the best robot opponent but makes a good enough guess
class HeuristicPlayer extends Player {
    constructor(id, difficulty = 2, aggression = 0.5, supportiveness = 0.5, riskAversion = 0.5) {
        super(id);
        //Controls how the player behaves and their general skill
        this.difficulty = difficulty;
        this.personality = { aggression, supportiveness, riskAversion };
        this.signalMemory = {}; // {playerId: {suit: +1 (like) / -1 (dislike)}}
        this.projectedTricks = 0;
        this.hasConceded = false;
    }

    //Bid returns action said player wants to do
    //Heuristic players will return the best bid that they can according to their hand
    bid(game) {
        //Analyse Hand
        const { hcp, suitLength, distributionalPoints } = this.evaluateHand();
        const totalPoints = hcp + distributionalPoints;
    }
}

```

```

const longestSuit = suitLength.indexOf(Math.max(...suitLength));
const shapeSignature = [...suitLength].sort((a, b) => b - a).join(`,`);
const balancedShapes = [`4,3,3,3`, `4,4,3,2`, `5,3,3,2`];
const isBalanced = balancedShapes.includes(shapeSignature);

//Bidding context
const partnerId = (this.id + 2) % 4;
const partnerIsDeclarer = game.declarer == partnerId;

const { num: currentLevel, trump: currentTrump, doubled } = game.currentDeal;
const noBidsYet = currentTrump == null;

//Difficulty & personality adjustments
// - beginners have more uncertain estimations - more noise
const noise = this.noiseForDifficulty();

//Personality nudges
const { aggression: aggr, riskAversion: risk, supportiveness: support } = this.personality;
const adjustedPoints = totalPoints + noise + (aggr - risk) * 2;

//Use trump index for projected tricks
const trumpIndexForEstimate = noBidsYet || currentTrump == 4 ? longestSuit : currentTrump;

//Simple trick projection & winnability
//Rough estimate - 6 default plus HCP/3 plus 0.3 per card baring 4 cards - then rounded to 1dp
this.projectedTricks = this.estimateTricks(adjustedPoints, suitLength[trumpIndexForEstimate]);

const safeFor = (target) => {
  const margin = risk * 0.6 - aggr * 0.4; //Risk increases margin, aggression reduces it
  return this.projectedTricks + margin >= target + 6.5 && target < 8;
};

```

```

//Bidding Logic

//Opener
if (noBidsYet) {
    if (adjustedPoints < 10) return `bid(pass)`;

    //Balanced 15-17 -> 1NT
    if (isBalanced && hcp > 14 && hcp < 18) return `bid(4,1)`;

    //Strong suit 13-19pts
    if (adjustedPoints > 12 && adjustedPoints < 20) return `bid(${longestSuit},1)`;

    //Preemptive weak 6-10pts & 6+ card suit
    if (hcp < 11 && suitLength[longestSuit] > 5 && Math.random() < 0.5 + aggr * 0.5) return
    `bid(${longestSuit},2)`;

    //Very strong 20+pts -> placeholder
    if (adjustedPoints > 19) return `bid(${longestSuit},2)`;

    return `bid(pass)`;
}

///Consider double/redouble
if (!partnerIsDeclarer) {
    const opponentsLikelyMake = this.projectedTricks >= currentLevel + 6 - 0.8;
    if (!opponentsLikelyMake && adjustedPoints > 15 && Math.random() < 0.2 + aggr * 0.6 &&
        game.currentDeal.doubled == 0) return `bid(double)`;
} else if (game.currentDeal.doubled == 1 && adjustedPoints > 12) {
    if (Math.random() < 0.2 + aggr * 0.5) return `bid(double)`;
}

```

```

//Responder
if (partnerIsDeclarer) {
    //check fit
    const fit = suitLength[currentTrump] > 2;

    if (fit) {
        //Simple raise
        if (adjustedPoints > 5 && adjustedPoints < 10 && safeFor(currentLevel + 1)) return
        `bid(${currentTrump},${currentLevel + 1})`;

        //Jump raise
        if (adjustedPoints > 9 && safeFor(currentLevel + 2)) return
        `bid(${currentTrump},${currentLevel + 2})`;

        //If supportive but marginal points - could raise
        if (support > 0.7 && adjustedPoints > 4 && Math.random() < support * 0.4 &&
            safeFor(currentLevel + 1)) return `bid(${currentTrump},${currentLevel + 1})`;
    }

    //NT
    if (!fit && isBalanced && adjustedPoints > 13 && safeFor(currentLevel)) return
    `bid(4,${currentLevel})`;

return `bid(pass)`;
}

//Over-caller
if (!partnerIsDeclarer) {
    //Overcall if strong and long suit
    if (adjustedPoints > 14 && suitLength[longestSuit] > 4) {
        //If aggressive stretch to overcall even if marginal
    }
}

```

```

        if (aggr > 0.6 && Math.random() < aggr * 0.5 && safeFor(currentLevel + 1)) return
        `bid(${longestSuit},${currentLevel + 1})`;
    }
}
return `bid(pass)`;
}

//Trick returns action said player wants to do
//Heuristic players will return the best card that they can legally play based on their hand and prior cards
played
trick(game) {
    //Suit awareness
    const trumpSuit = game.currentDeal.trump;
    const leadSuit = game.leadSuit;
    const isLeader = leadSuit == null;

    //Hand awareness
    const handSorted = [...this.hand].sort((a, b) => rankValue(b.pip) - rankValue(a.pip));
    const cardsBySuit = {
        follow: handSorted.filter((c) => c.suit == leadSuit),
        trump: handSorted.filter((c) => c.suit == trumpSuit),
        off: handSorted.filter((c) => c.suit != leadSuit && c.suit != trumpSuit),
    };

    this.suitLength = this.evaluateHand().suitLength;

    //Self awareness
    const persona = this.adjustPersonality(game);

    if (isLeader) return this.leadCard();

    // Following suit

```

```

    if (cardsBySuit.follow.length > 0) {
        return this.followSuit(game, cardsBySuit, persona);
    }

    // Ruffing / trumping
    if (cardsBySuit.trump.length > 0 && trumpSuit !== 4) {
        return this.ruff(game, cardsBySuit, persona);
    }

    // Discard
    return this.discard(cardsBySuit);
}

//General Helper
evaluateHand() {
    let hcp = 0;
    let suitLength = [0, 0, 0, 0];
    let distributionalPoints = 0;

    //Point maps
    const hcpMap = { 1: 4, 13: 3, 12: 2, 11: 1 }; //A,K,Q,J
    const distributionalMap = { 0: 3, 1: 2, 2: 1 }; //Void, Singleton, Doubleton

    //Count points

    //Loops through cards of hand
    for (let card of this.hand) {
        //Calculate HCP value of hand
        hcp += hcpMap[card.pip] || 0;

        //Calculate length of each suit
        suitLength[card.suit]++;
    }
}

```



```

    }

    //Loops through the suits
    for (let length of suitLength) {
        //Calculates the distributional points
        distributionalPoints += distributionalMap[length] || 0;
    }

    return { hcp, suitLength, distributionalPoints };
}

//BIDDING helpers
noiseForDifficulty() {
    let noise = 0;
    if (this.difficulty == 0) noise = Math.random() * 4 - 2; //-2 -> 2
    if (this.difficulty == 1) noise = Math.random() * 2 - 1; //-1 -> 1
    return noise;
}

estimateTricks(adjustedPoints, suitLength) {
    return Math.round((6 + adjustedPoints / 3 + (suitLength - 4) * 0.3) * 10) / 10;
}

//TRICK
adjustPersonality(game) {
    let { aggression: aggr, riskAversion: risk } = this.personality;
    const progress = game.tricksAmount / 13;
    const fatigue = progress * 0.01; //Raise aggression and lower caution in time
    aggr = Math.min(1, aggr + fatigue);
    risk = Math.max(0, risk - fatigue);
    return { aggr, risk };
}

//Trick Functions

```

```

leadCard() {
  const suitLen = this.suitLength;

  let suitChoice = suitLen.indexOf(Math.max(...suitLen));

  //Fall back if empty
  let options = this.hand.filter((c) => c.suit == suitChoice);
  if (options.length == 0) options = this.hand;

  const card = this.hasConceded ? options[0] : options.at(-1);
  return `trick(${card.suit},${card.pip})`;
}

followSuit(game, cards, persona) {
  const { follow } = cards;
  const lead = game.leadSuit;

  const highestLead = this.highestOnTable(game, lead);
  const canWin = follow.some((c) => rankValue(c.pip) > rankValue(highestLead));

  const wants = Math.random() < 0.3 + persona.aggr * 0.5;
  const card = this.hasConceded ? follow[0] : canWin && wants ? follow.find((c) => rankValue(c.pip) >
rankValue(highestLead)) : follow.at(-1);

  return `trick(${lead},${card.pip})`;
}

ruff(game, cards, persona) {
  const { trump } = game.currentDeal;
  const { trump: trumpCards } = cards;

  const highestTrump = this.highestOnTable(game, trump);
  const overTrumps = trumpCards.filter((c) => rankValue(c.pip) > rankValue(highestTrump));

```

```

    const bold = Math.random() < 0.4 + persona.aggr * 0.4 - persona.risk * 0.3;

    const chosen = this.hasConceded ? trumpCards[0] : overTrumps.length > 0 ? overTrumps.at(-1) : bold ?
    trumpCards[0] : trumpCards.at(-1);

    return `trick(${chosen.suit},${chosen.pip})`;
  }
  discard(cards) {
    const { off } = cards;
    const playable = off.length > 0 ? off : this.hand;
    const chosen = this.hasConceded ? playable[0] : playable.at(-1);
    return `trick(${chosen.suit},${chosen.pip})`;
  }

  highestOnTable(game, suit) {
    return game.currentTrick
      .filter((c) => c.card.sui == suit)
      .reduce((best, cur) => (rankValue(cur.card.pip) > rankValue(best.card.pip) ? cur : best), {
        card: { pip: 0 },
      }).card.pip;
  }
}

export { HeuristicPlayer, RandomPlayer, RealPlayer };

```

Src/game.js

```

/*
  game.js
  Contains the class for a game
  exports { Game }

```

```

*/

//Imports
import { bid } from "./bidding.js";
import { HeuristicPlayer, RealPlayer } from "./player.js";
import { addScore, createScoreObject } from "./score.js";
import { trick } from "./tricks.js";

//Global variables
const suits = [ `â™™ i`,` â™™y i`,` â™™| i`,` â™™£ i`,` `NT`];
const doubleTxt = [ ``, `x`, `xx`];

// Lambda expressions
var nextTurn = (i) => (i + 1) % 4;
var rankValue = (pip) => (pip == 1 ? 14 : pip);

// Classes
class Card {
  constructor(suit, pip) {
    this.suit = suit;
    this.pip = pip;
  }
  toString() {
    const pips = [null, `A`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `10`, `J`, `Q`, `K`];
    return `${pips[this.pip]}${suits[this.suit]}`;
  }
}

class Game {
  constructor(playerMap) {
    // Game Data
    const totalSeats = 4;

```

```

this.players = Array.from({ length: totalSeats }, (_, seatId) => {
    //Check if this seat belongs to a real player
    const isReal = Object.values(playerMap).includes(seatId);

    if (isReal) {
        //Person
        return new RealPlayer(seatId);
    } else {
        //Bot
        return new HeuristicPlayer(seatId, 2, Math.random(), Math.random(), Math.random());
    }
});
this.playerAvatars = new Array(4).fill({ name: '', avatar: -1 });

this.gameStates = { deal: 0, bid: 1, trick: 2, score: 3 };
this.gameOver = false;

// Player Data
this.dealer = null;

//Set new deal
this.initialise();

//Create new score object
this.score = createScoreObject();

//Set final data
const redTeam = Math.round(Math.random());
this.colour = redTeam == 0 ? [1, 0] : [0, 1];
this.pairs = [
    { id: 0, players: [this.players[0], this.players[2]], score: this.createScore(), vulnerable:
        false },

```

```

        { id: 1, players: [this.players[1], this.players[3]], score: this.createScore(), vulnerable:
          false },
      ];
    }

```

```

initialise() {
  //Create deck and set hands
  const hands = this.dealCards();
  this.players.forEach((p, i) => {
    p.hand = hands[i];
  });

  //Player Data
  if (this.dealer == null) this.dealer = Math.floor(Math.random() * 4);
  else this.dealer = nextTurn(this.dealer);

  this.declarer = null;
  this.dummy = null;
  this.currentPlayer = this.dealer;

  this.playable = [];

  //Bid Data
  this.currentDeal = { num: 0, trump: null, doubled: 0 };

  //Trick Data
  this.currentTrick = [];
  this.leadSuit = null;
  this.tricksAmount = 0;
  this.tricksWon = [0, 0];
  this.passCount = 0;

```

```

    //Game Data
    this.state = this.gameStates.bid;
    this.currentGame = 0;
}

dealCards() {
    //Create Deck of 52 cards
    const deck = [];
    for (let suit = 0; suit < 4; suit++) {
        for (let pip = 1; pip <= 13; pip++) {
            deck.push(new Card(suit, pip));
        }
    }
    //Fisher-Yates Shuffle
    for (let idx = 51; idx > 0; idx--) {
        let pos = Math.floor(Math.random() * (idx + 1));
        [deck[idx], deck[pos]] = [deck[pos], deck[idx]];
    }
    //Deal to 4 hands and order
    const hands = [];
    for (let idx = 0; idx < 4; idx++) {
        const hand = deck.splice(0, 13).sort((a, b) => {
            return a.suit == b.suit ? rankValue(a.pip) - rankValue(b.pip) : a.suit - b.suit;
        });
        hands.push(hand);
    }
    //Return 4 hands
    return hands;
}

//Parse input from network
processActionFromNetwork(cmd, args) {

```

```

let action = "";
if (cmd == `bid`) {
    if (args.length == 2) action = { trump: Number(args[0]), num: Number(args[1]) };
    else action = args[0];
    return bid(this, action);
} else if (cmd == `trick`) {
    action = { suit: Number(args[0]), pip: Number(args[1]) };
    return trick(this, action);
} else if (cmd == `concede`) {
    return concede(this, this.currentPlayer);
}
return { success: false, msg: `Unknown remote action: ${cmd}(${args.join(',')})` };
}

//Handle change of deal
nextDeal() {
    if (this.state != this.gameStates.score || this.gameOver) return { success: false, msg: `You can't start
a deal until this one is complete` };
    this.state = this.gameStates.bid;
    this.initialise();
    return { success: true, msg: `New deal started. Begin bidding!`, clear: true, game: this };
}

//Robot Methods
robotAction() {
    const plr = this.players[this.currentPlayer];
    if (plr instanceof RealPlayer) return null; // player is human

    switch (this.state) {
        case this.gameStates.bid:
            return plr.bid(this);
        case this.gameStates.trick:

```



```

        return plr.trick(this);
    }

    return null;
}

//Serialisable output
exportState() {
    return {
        state: this.state,
        currentPlayer: this.currentPlayer,
        me: this.playable,
        declarer: this.declarer,
        dummy: this.dummy,
        deal: this.currentDeal,
        tricksWon: this.tricksWon,
        hands: this.players.map((p) =>
            p.hand.map((c) => ({
                suit: c.suit,
                pip: c.pip,
                name: c.toString(),
            })))
    ),
    scores: this.pairs.map((p) => p.score),
};
}

//Player setup
newPlayerAvatars() {
    this.playerAvatars = [];
    for (let idx = 0; idx < 4; idx++) {
        let player = { name: ``, avatar: -1 };

```

```

        player.avatar = Math.floor(Math.random() * 4);

        const pool = namePool[player.avatar];
        const adj = pool.adj[Math.floor(Math.random() * pool.adj.length)];
        const noun = pool.noun[Math.floor(Math.random() * pool.noun.length)];
        const num = Math.floor(Math.random() * 99);
        player.name = `${adj}${noun}${num}`;

        this.playerAvatars.push(player);
    }
}

//Helper Methods
getPairIndex(plrId) {
    return [0, 2].includes(plrId) ? 0 : 1;
}

createScore() {
    return { below: [0, 0, 0], above: 0, penalty: 0, bonus: 0 };
}
}

function concede(game, player) {
    game.gameOver = true;
    //Identify who concedes and who benefits
    const concedingPair = game.getPairIndex(player);
    const winningPair = 1 - concedingPair;

    //Tally remaining tricks as won by opponents
    const remainingTricks = 13 - game.tricksAmount;
    game.tricksWon[winningPair] += remainingTricks;
}

```

```

//Finalise score
const scoring = addScore(game, game.currentDeal, game.tricksWon[game.getPairIndex(game.declarer)]);

//Change game state
scoring.game.state = game.gameStates.score;

return {
  success: true,
  msg: `Pair ${winningPair + 1} has claimed the remaining ${remainingTricks} tricks and won the score by
  concession.\n${scoring.txt}`,
  game: scoring.game,
  clear: true,
};
}

const namePool = {
  0: {
    //King
    adj: ["Grand", "Iron", "Golden", "Stout", "Crimson"],
    noun: ["Crown", "Sceptre", "Throne", "Banner", "Hammer"],
  },
  1: {
    //Queen
    adj: ["Silver", "Velvet", "Scarlet", "Silent", "Emerald"],
    noun: ["Rose", "Lace", "Moon", "Crown", "Mirror"],
  },
  2: {
    //Pauper
    adj: ["Dusty", "Hungry", "Patchy", "Crooked", "Rusty"],
    noun: ["Sock", "Boot", "Bowl", "Spoon", "Rat"],
  },
  3: {

```

```

        //Joker
        adj: ["Mad", "Wild", "Sneaky", "Twisty", "Baffled"],
        noun: ["Banana", "Card", "Dice", "Goose", "Duck"],
    },
};

```

```

export { doubleTxt, Game, nextTurn, rankValue, suits };

```

Src/bidding.js

```

/*
bidding.js
Contains the logic for a bid
exports { bid }
*/

//Imports
import { doubleTxt, nextTurn, suits } from "../game.js";
import { RealPlayer } from "../player.js";

//Lambda expressions
var returnCard = (card) => `${card.num}${suits[card.trump]}${card.doubled ? doubleTxt[card.doubled] : ``}`;

/**
 * Runs the bid validation and resultant algorithm
 *
 * @param {Game} game The game that the socket belongs to
 * @param {args} input The input that the player wants to bid
 * @returns Result of the bid
 */
function bid(game, input) {

```

```

//Validation
if (game.state !== game.gameStates.bid) return { success: false, msg: `Wrong state` };

//If the input is a card
if (typeof input === `object`) {
    return bidType.object(game, input);

//If the input is a pass
} else if (input === `pass`) {
    return bidType.pass(game);

//If the input is a double or a redouble
} else if ((input === `double` || input === `redouble`) && game.currentDeal.trump !== null) {
    return bidType.double(game);

//If the input is invalid
} else {
    return { success: false, msg: `Invalid bid` };
}
}

/** Handles the different types of bid */
const bidType = {
    /** Handles bidding a card */
    object: (game, input) => {
        //Validation
        if (!Number.isInteger(input.num) || input.num < 1 || input.num > 7) return { success: false, msg:
            `Invalid num` };
        if (!Number.isInteger(input.trump) || input.trump < 0 || input.trump > 4) return { success: false, msg:
            `Invalid Trump` };
        if (input.num < game.currentDeal.num || (input.num === game.currentDeal.num && input.trump !== 4 &&
            game.currentDeal.trump !== 4)) {

```

```

        return { success: false, msg: `Bid must be higher then current or trump NO TRUMP` };
    }
    //Valid card

    //Sets the deal
    game.currentDeal = { num: input.num, trump: input.trump, doubled: game.currentDeal.doubled };
    game.declarer = game.currentPlayer;
    //Resets the pass counter
    game.passCount = 0;

    const prevPlayer = game.currentPlayer + 1; //the previous player to display it
    //Calculates who the next player is
    game.currentPlayer = nextTurn(game.currentPlayer);

    return { success: true, bid: game.currentDeal, msg: `Player ${prevPlayer} plays:
    ${returnCard(game.currentDeal)}`, game };
},

/** Handles bidding a pass */
pass: (game) => {
    //Incremenets the pass counter
    game.passCount++;

    //If there is a deal and 3 passes have happened then end bidding
    if (game.passCount == 3 && game.currentDeal.trump != null) {
        //Start trick-taking
        game.state = game.gameStates.trick;

        //Whoever won the bid plays first
        game.currentPlayer = game.declarer;

        //Calculates who the dummy && declarer are

```

```

game.dummy = (game.declarer + 2) % 4;
const dummy = game.players[game.dummy];
const declarer = game.players[game.declarer];
//If the dummy is a real player and the declarer is a bot the switch hands
if (dummy instanceof RealPlayer && !(declarer instanceof RealPlayer)) {
    //Switch hands
    [game.players[game.declarer].hand, game.players[game.dummy].hand] =
    [game.players[game.dummy].hand, game.players[game.declarer].hand];
    //Switch dummy and declarer indexes
    [game.dummy, game.declarer] = [game.declarer, game.dummy];
}

return { success: true, bid: `pass`, msg: `\\nBidding finished. Moving to Trick Taking`, clear: true,
game, dummy: game.currentPlayer !== game.dummy };

//If there is no deal and there are 4 passes then redeal
} else if (game.passCount == 4 && game.currentDeal.trump == null) {
    const prevPlayer = game.currentPlayer + 1; //the previous player to display it
    //Restart the game
    game.initialise();
    return { success: true, bid: `pass`, msg: `Player ${prevPlayer} passes\\nFour passes - re-deal`,
game };
}

const prevPlayer = game.currentPlayer + 1; //the previous player to display it
//Calculates who the next player is
game.currentPlayer = nextTurn(game.currentPlayer);

return { success: true, bid: `pass`, msg: `Player ${prevPlayer} passes`, game };
},

/** Handles bidding a double or a redouble */

```

```

double: (game) => {
  //Increments the double but clamps at 2
  game.currentDeal.doubled = Math.min(2, game.currentDeal.doubled + 1);
  //Resets the pass counter
  game.passCount = 0;

  const prevPlayer = game.currentPlayer + 1; //the previous player to display it
  //Calculates who the next player is
  game.currentPlayer = nextTurn(game.currentPlayer);

  return { success: true, bid: game.currentDeal.doubled == 2 ? `redouble` : `double`, msg: `Player
    ${prevPlayer}: ${game.currentDeal.doubled == 2 ? `redoubles` : `doubles`} the bid`, game };
},
};

export { bid };

Src/trick.js
/*
  tricks.js
  Contains the logic for a trick
  exports { trick }
*/

//Imports
import { nextTurn, rankValue } from "../game.js";
import { addScore } from "../score.js";

function trick(game, play) {
  if (game.state != game.gameStates.trick) return { success: false, msg: `Wrong state` };
  const player = game.players[game.currentPlayer];

```



```

//Vallidate card ownership
const cardIdx = player.hand.findIndex((c) => c.pip == play.pip && c.suit == play.suit);
if (cardIdx == -1) return { success: false, msg: `You don't have that card` };

//Enforce following suit
if (game.leadSuit != null) {
  const canFollow = player.hand.some((c) => c.suit == game.leadSuit);
  if (canFollow && play.suit != game.leadSuit) return { success: false, msg: `You must follow suit if possible` };
}

//Play card
const [card] = player.hand.splice(cardIdx, 1);
game.currentTrick.push({ player: game.currentPlayer, card });

//Set lead suit if first card
if (game.leadSuit == null) game.leadSuit = card.suit;

//Trick complete
if (game.currentTrick.length == 4) {
  const winner = evaluateTrick(game.currentTrick, game.leadSuit, game.currentDeal.trump);
  let txt = `Player ${winner + 1} wins the trick\n`;

  //Score trick for pairs
  const declaringPairIdx = game.getPairIndex(game.declarer);
  const defendingPairIdx = 1 - declaringPairIdx;

  const pairWon = game.pairs[declaringPairIdx].players.some((p) => p.id == winner);
  if (pairWon) game.tricksWon[declaringPairIdx]++;
  else game.tricksWon[defendingPairIdx]++;
}

```

```

    //Set next turn
    game.currentPlayer = winner;
    game.currentTrick = [];
    game.leadSuit = null;

    //Advance trick count
    game.tricksAmount++;

    //Deal complete
    if (game.tricksAmount == 13) {
        game.state = game.gameStates.score;
        const result = addScore(game, game.currentDeal, game.tricksWon[declaringPairIdx], game.declarer);
        txt = result.txt;
        game = result.game;
    }
    return { success: true, scoreTxt: txt, msg: txt, clear: true, game, dummy: game.tricksAmount != 13 &&
        game.currentPlayer != game.dummy };
}

//Trick not complete
const prevPlayer = game.currentPlayer + 1;
game.currentPlayer = nextTurn(game.currentPlayer);
return { success: true, msg: `Player ${prevPlayer} plays ${card.toString()}`, game };
}

function evaluateTrick(trick, leadSuit, trumpSuit) {
    let best = trick[0];
    for (const play of trick) {
        const card = play.card;

        //Ignore irrelevant suits
        if (card.suit != trumpSuit && card.suit != leadSuit) continue;

```

```

        //Trump beats everything
        if (best.card.suit !== trumpSuit && card.suit === trumpSuit) {
            best = play;
            continue;
        }

        //Higher card of same suit
        if (best.card.suit === card.suit && rankValue(card.pip) > rankValue(best.card.pip)) {
            best = play;
        }
    }

    return best.player;
}

export { trick };

Src/score.js
/*
    score.js
    Contains the logic for scoring
    exports { addScore, createScoreObject }
*/

//Global Variables
const trickValues = { 0: 30, 1: 30, 2: 20, 3: 30, 4: 30 };
let pointsThisGame = 0;

//Initialise a new score object
function createScoreObject() {

```

```

    pointsThisGame = 0;
    return {
        metadata: {
            rubberOver: false,
            gameLines: [],
            gamesWon: [0, 0],
        },
        rounds: [],
    };
}

/**
 * Calculate and apply the score for a completed contract
 *
 * @param {Game} game - The current game object
 * @param {object} deal - Contains num, trump, doubled
 * @param {number} tricks - Tricks made by the declarer
 * @returns {{ txt: string; game: object }}
 */
function addScore(game, deal, tricks) {
    const declaringPair = game.getPairIndex(game.declarer);
    const defendingPair = 1 - declaringPair;

    const score = game.pairs[declaringPair].score;
    const oppScore = game.pairs[defendingPair].score;

    const { num: level, trump, doubled } = deal;
    const required = 6 + level;
    const made = tricks >= required;
    const vulnerable = game.pairs[declaringPair].vulnerable;

    let txt = ``;

```

```

//Prepare a round object for the score sheet
const round = {
  declarer: game.declarer,
  contractMade: made,
  pointsUnder: 0,
  pointsAbove: 0,
  pointsPenalty: 0,
};

if (made) {
  const overtricks = tricks - required;

  //Base contract points
  const basePoints = contractPoints(level, trump, doubled);
  score.below[game.currentGame] += basePoints;
  round.pointsUnder = basePoints;

  //Overtricks and bonuses
  let overPoints = 0;
  let bonuses = 0;
  if (overtricks > 0) {
    overPoints = overtrickPoints(overtricks, trump, doubled, vulnerable);
    score.above += overPoints;
    round.pointsAbove = overPoints;
  }

  //Small / grand slam bonuses
  if (doubled > 0) bonuses += 50 * doubled;

  if (level == 6) bonuses += vulnerable ? 750 : 500;
  if (level == 7) bonuses += vulnerable ? 1500 : 1000;
}

```

```

score.bonus += bonuses;

//Game completion check
pointsThisGame += score.below[game.currentGame];
if (pointsThisGame >= 100) {
    pointsThisGame = 0;
    game.currentGame++;
    game.pairs[declaringPair].vulnerable = true;

    //Add line
    game.score.metadata.gameLines.push(game.score.rounds.length + 1);

    //Mark game won
    game.score.metadata.gamesWon[game.declarer % 2]++;
}

//Rubber completion check
const gamesWon = score.below.filter((v) => v >= 100).length;
if (gamesWon >= 2) {
    game.gameOver = true;
    const rubberBonus = 700 + (vulnerable ? 500 : 0);
    score.bonus += rubberBonus;
    game.score.metadata.rubberOver = true;

    txt += `\\nDeclarer pair wins the rubber! Bonus awarded.\\n`;
}

game.pairs[declaringPair].score = score;
} else {
    //Contract failed
    const undertricks = required - tricks;
    const penaltyPointsValue = penaltyPoints(undertricks, doubled, vulnerable);

```

```

        oppScore.penalty += penaltyPointsValue;
        round.pointsPenalty = penaltyPointsValue;

        game.pairs[defendingPair].score = oppScore;
    }

    game.score.rounds.push(round);

    return { txt, game };
}

//Base contract points - below-the-line
function contractPoints(level, trump, doubled) {
    let base = trump == 4 ? 40 + (level - 1) * 30 : trickValues[trump] * level;
    if (doubled > 0) base *= 2 * doubled;
    return base;
}

//Points for overtricks
function overtrickPoints(overtricks, trump, doubled, vulnerable) {
    if (doubled > 0) return overtricks * (vulnerable ? 2 : 1) * 100 * doubled;
    return overtricks * trickValues[trump];
}

//Points for failed contracts
function penaltyPoints(undertricks, doubled, vulnerable) {
    const base = doubled == 0 ? 50 : doubled == 1 ? (vulnerable ? 200 : 100) : vulnerable ? 400 : 200;
    return base * undertricks;
}

export { addScore, createScoreObject };

```


Public/ - Client-side (served on connection)

Public/index.html

```
<!--
    Index.html
    Contains code to affect the header, create the viewport, remove the margins, and link the servers
-->
<!doctype html>
<html lang="en">
<head>
    <!-- Sets meta data -->
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0, user-scalable=no"
    />

    <!-- Sets the title -->
    <title>Bridge</title>

    <!-- Sets Favicon Icon -->
    <link rel="icon" type="image/png" href="./Art/favicon.png" />

    <!-- P5.js -- Visuals && Audio -- -->
    <script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.11.10/p5.min.js" integrity="sha512-
    7DACDM+XO+/7cocfzNIXh/3rSXC7MzTtr2elSVJLYcXXHPL1/lAyFSKU1F1JBPPCvBGOLNYJCqLlzFeJzc6dLg=="
    crossorigin="anonymous" referrerpolicy="no-referrer"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.11.10/addons/p5.sound.min.js" integrity="sha512-
    WzkwpdWEMAY/W8WvP9KS2/VI6zkgejR4/KTxTl4qHx0utqeyVE0JY+S1DlMuxDChC7x0oXtk/ESji6a0lP/Tdg=="
    crossorigin="anonymous" referrerpolicy="no-referrer"></script>

    <!-- Socket -- Server Connection -- -->
    <script src="/socket.io/socket.io.js"></script>
```

```

<!-- Classes -->
<script src="buttons.js"></script>
<script src="cards.js"></script>
<!-- Drawing the Game -->
<script src="drawGame.js"></script>
<!-- Client -- Main file -- -->
<script src="client.js"></script>
</head>

<!-- CSS -- only to make canvas 'nicer' -->
<style>
/* Removes margin, padding of everything */
/* Sets to full screen and removes overflow */
/* Sets default colour say the canvas is struggling to load */
html,
body {
    margin: 0;
    padding: 0;
    height: 100%;
    width: 100%;
    overflow: hidden;
    touch-action: none;
    background-color: rgb(50, 50, 50);
    color: rgb(200, 200, 200);
}

/* Makes sure that the canvas is always full screen */
canvas {
    display: block;
    position: fixed;
    top: 0;
    left: 0;

```

```

    }
</style>
</html>

```

Public/client.js – due to my chrome book's clipboard some formatting doesn't work

```

/*
    client.js
    The main client-side script handles inputs, data storage and the communication between scripts
*/

//Global Variables
//Card Display variables
const STATE = Object.freeze({ MENU: 0, LOBBY_CREATE: 1, GAME: 2 });
const Suits = ["♠", "♥", "♦", "♣", "NT"];
const DOUBLES = ["", "x", "xx"];

//Info for the game
const app = {
    //This socket id
    socket: io(),
    //The data from the server
    data: resetData(),
    //List of buttons
    buttons: [],
    //The current state of the client
    state: STATE.MENU,
};

//Constants to handle the display of the client
const display = {
    centreDist: [0, 0], //Distance from centre to hands
    cardScalePlayer: 3.5, //Scale of the player's card

```

```

    cardScaleOpponent: 2, //Scale of opponent's card
    cardScaleBid: 3, //Scale of the bidding card buttons
    cardSize: [32, 48], //Pixel size of the cards
    cardSizeBid: [8, 15], //Pixel size of the bid buttons
    screen: { cx: 0, cy: 0, size: { w: 0, h: 0 } }, //Size and positions of the screen
    myColour: [0, 0, 0], //Colour of this client's pair
    oppColour: [0, 0, 0], // Colour of this client's opponent's pair
    scoreSheetSize: [150, 350], //Width and height of the scoresheet
    scoreSheetSizeModifier: 0.75, //Size modifier of the scoresheet - when in trick
};
//List of clickable cards - so can iterate through them and check for inputs
let clickableCards = [];
//GFX && SFX
//Tile atlas - loaded in preload
let spriteSheet;
//Music & sounds variables
let music; //Background music
let cardSound; //Card swipe sound
let buttonSound; //UI click sound
let looping = false; //Whether the background music is looping

//Sets-up stored data - takes in id for when it needs to reset say 4 pass bids
function resetData(id = null) {
    return {
        currentDeal: null, //The deal
        currentHand: null, //The client's hand
        currentTrick: [], //The cards played this trick
        tricksWon: [0, 0], //The number of tricks won by both pairs
        handLengths: [13, 13, 13, 13], //The length of each player's hand
        bid: [null, null, null, null], //The past bids of this round
        declarer: null, //Idx of the declarer
        dummyHand: null, //The dummy's hand
    };
}

```

```

dummyPlr: null, //Idx of the dummy
iAmDummy: false, //Whether this player is a dummy
me: null, //This client's seat idx - not the same as id
playable: [], //All seats this client can play - theirs and dummy's
id, //This players id
lobbyId: null, //The lobby id the client is in
playerMap: null, //The player map from the server - socket:seat
playerAvatars: new Array(4).fill({ name: '', avatar: -1 }), //The players' names and avatars
members: [], //The members in this lobby (list of socket ids)
phase: -1, //The phase of the game -- auction, trick, score
};

}

//Suit functions
const suit = {
  //It is necessary to convert from server to client and vice versa because I was stupid in -
  // the ordering of the suits in the server and these didn't match well with what would be in the client.
  //This code translates between the two
  server: ['♠', '♥', '♦', '♣', 'NT'], //Server list
  client: ['♠', '♣', 'NT', '♦', '♥'], //Client list
  //Server to client index
  toClientSide: (serverIdx) => {
    const serverSuit = suit.server[serverIdx]; //Gets what the server is showing
    return suit.client.indexOf(serverSuit); //Returns index of this for the client's list
  },
  //Client to server index
  toServerSide: (clientIdx) => {
    const clientSuit = suit.client[clientIdx]; //Gets what the client is showing
    return suit.server.indexOf(clientSuit); //Returns index of this for the server's list
  },
};

```

```

//Audio Playback
//Plays the UI click sound
const playButtonSound = () => {
  buttonSound.play(0, 1, 1, 0.65);
};

//Plays the card swipe sound
const playCardSwipeSound = () => {
  cardSound.play(0, 1, 1, 0);
};

//Input Handling
//Sends input to the server
const handleInput = (input) => input && app.socket.emit("game:input", input);
//Returns whether the client is hovering over the card - handles rotated rectangles
const isHovering = (mouse, rect) => {
  //Translate mouse to card-centered coords
  const delta = { x: mouse.x - rect.x, y: mouse.y - rect.y };
  //Calculates the trig for the mouse's rotations
  angle = -rect.angle || 0;
  const trigA = { cos: cos(angle), sin: sin(angle) };
  //Calculates the new position of the card
  const local = {
    x: delta.x * trigA.cos - delta.y * trigA.sin, //Rotated x-position
    y: delta.x * trigA.sin - delta.y * trigA.cos, //Rotated y-position
  };
  //Returns if this rotate position is within the rectangles bounds
  return local.x > -rect.w / 2 && local.x < rect.w / 2 && local.y > -rect.h / 2 && local.y < rect.h / 2;
};

//Socket Event Router
app.socket.on("connect", () => {
  //Sets player's stored id to their socket id

```

```

    app.data.id = app.socket.id;
    //Logs it to the console
    console.log(`Connected with id: ${app.data.id}`);
  });
  //Occurs when the server sends a message to the client
  app.socket.on("game:update", (message) => {
    //Logs it to the console for debugging
    console.log(`Server message:\n${JSON.stringify(message)}`);
    //Reports a change of game phase
    if (message.phase && message.phase !== app.data.phase) {
      app.data.phase = message.phase;
      //Lays out this game
      layoutUI(width, height);
    }

    //The function to run based on the type of message
    const handler = eventHandlers[message.type];
    //Sets the current player if it is specified from the server
    if (message.currentPlayer !== undefined) app.data.currentPlayer = message.currentPlayer;
    //Runs said handler if it exists otherwise just warns the console (some old messages still exist)
    handler ? handler(message.payload || message.errorMsg) : console.warn(`No handler for type: ${message.type}`);
  });

  //Socket Event Handling
  const eventHandlers = {
    //System - something has happened like player joined or another player has left
    // - just needs to be shown to player using the toast animation
    system: ({ msg }) => showSystemMessage(msg) || console.warn(`System notifications not handled!`),
    //Actions - something like a player playing a bid or trick
    action: (payload) => handleAction(payload),
    //Data - there is some data that the player has to update - like their hand
    data: (payload) => handleData(payload),
  };

```

```

    //Error - if something goes wrong then show the error msg using a toast animation
    error: (msg) => anims.showToast(msg),
};
//Handle system notifications
function showSystemMessage(msg) {
    //If on the lobby creation screen then show the message through a toast
    if (app.state == STATE.LOBBY_CREATE) return anims.showToast(msg, `info`);
    //Otherwise return true - if return false then warns the console eg toast failed
    return true;
}
//Handle data notifications
function handleData(payload) {
    //Lobby State
    if (app.state == STATE.MENU) app.data.lobbyId = ``; //Sets lobby id to nothing if the client is now on the menu
    if (payload.lobbyId && app.state == STATE.MENU) {
        //If on menu then reset the data and join new lobby
        //Gets reset data and set current app.data to it
        Object.assign(app.data, resetData(app.data.id));
        console.log(`Joined lobby ${payload.lobbyId}`);
        //Sets the state to the lobby-creation
        setState(STATE.LOBBY_CREATE);
    } else if (payload.lobbyId === null)
        //Leaving lobby
        //Sets the state to the menu screen
        setState(STATE.MENU);

    //Set hand lengths
    if (payload.cardCount !== undefined) app.data.handLengths[payload.handOwner] = payload.cardCount;

    //Set pair colours
    if (payload.colour === 0 || payload.colour === 1) {
        //If the client is red or black then run

```



```

    const red = [130, 41, 41];
    display.myColour = payload.colour == 0 ? 0 : red; //If player colour is 0 then set player's colour to
    black otherwise set to red
    display.oppColour = payload.colour == 1 ? 0 : red; //If player colour is 1 then set the opp's colour to
    black otherwise set to red
  }

  //Set all data - including stuff set especially before and afterwards
  Object.entries(payload).forEach(([key, val]) => {
    if (val) app.data[key] = val;
  });

  //Player Mapping
  if (payload.playerMap) {
    //Sets the player's seat id
    app.data.me = app.data.playerMap[app.data.id];
    //Sets the player's playable seats - them and dummy?
    app.data.playable.push(app.data.me);
    //Sets state to the game
    setState(STATE.GAME);
  }

  //Dummy Logic
  if (payload.dummyPlr && payload.dummyPlr != null) {
    //Dummy is 2 seats away (opposite player)
    if (app.data.dummyPlr === (app.data.playerMap[app.data.id] + 2) % 4
    && !app.data.playable.includes(app.data.dummyPlr)) app.data.playable.push(app.data.dummyPlr);
    //Client is the dummy
    else if (app.data.dummyPlr === app.data.me) app.data.iAmDummy = true;
  }

  //Current deal

```

```

    if (payload.currentDeal) {
        //Sets current deal
        app.data.currentDeal = {
            num: payload.currentDeal.num,
            trump: suit.toClientSide(payload.currentDeal.trump), //Translates the current trump suit to client-
            side
            doubled: payload.currentDeal.doubled,
        };
    }
}
//Helper function to set state
function setState(newState) {
    //Changes state
    app.state = newState;
    //Setups buttons and layout of the new state
    layoutUI(width, height);
}
//Handle action notifications
function handleAction(payload) {
    //Auction
    if (app.data.phase == 1) {
        //Sets bid
        let plr = (app.data.currentPlayer + 3) % 4; //Finds what player it is
        //If there is a bid that isn't pass or dbl then set current deal to that
        if (typeof payload.bid == `object`) {
            app.data.currentDeal = payload.bid;
        }
        //Set this bid to that message
        app.data.bid[plr] = payload.bid;
    }
}
}

```

```

//UI & Rendering Logic
//UI Functions

//Handles Animations
const anims = {
  //Stores all current toasts
  toasts: [],
  //Colours of the two toast types:
  colours: {
    info: [80, 80, 255],
    error: [255, 100, 100],
  },
  //Creates the toast objects
  showToast: (msg, type = `error`) => {
    //Adds this new toast to the toast array
    anims.toasts.push({
      msg,
      type,
      colour: anims.colours[type] || anims.colours.error,
      time: millis(),
      scale: 0.7,
      yOffset: 40,
      alpha: 0,
    });
    return true;
  },
  //Displays the toast objects
  drawToasts: () => {
    //Calculates current time - animations are based on this
    const now = millis();
    //Sets position to start at this
    const baseY = height - 80;
  }
};

```

```

//Durations
const appearTime = 300; //Zoom in over 0.3s
const stayTime = 1800; //Stay visible for ~2s
const fadeTime = 500; //fade out
//Total duration
const totalLife = appearTime + stayTime + fadeTime;

//Loops through all toasts
for (let idx = anims.toasts.length - 1; idx >= 0; idx--) {
  //Gets this toast
  const toast = anims.toasts[idx];
  //Calculates the age of the toast
  const age = now - toast.time;

  //Deletion
  if (age >= totalLife) {
    //Removes from the toasts array and ignore
    anims.toasts.splice(idx, 1);
    continue;
  }

  //Map age to progress stages 0-1
  let appearT = constrain(age / appearTime, 0, 1);
  let fadeT = constrain((age - appearTime - stayTime) / fadeTime, 0, 1);

  //Smooth transitions of size
  toast.scale = age < appearTime ? lerp(0.7, 1, anims.easeOutBack(appearT)) : age < appearTime +
  stayTime ? 1 : lerp(1, 0.9, anims.easeInOut(fadeT));
  //Transparency
  toast.alpha = age < appearTime ? lerp(0, 255, appearT) : age < appearTime + stayTime ? 255 :
  lerp(255, 0, fadeT);
  //And y position

```

```

toast.yOffset = age < appearTime ? lerp(40, 0, appearT) : age < appearTime + stayTime ? 0 : lerp(0,
20, fadeT);

//Draw
push();
translate(width / 2, baseY - idx * 70 + toast.yOffset);
scale(toast.scale);
noStroke();

//Sets colour to this toast's type's colour
const [r, g, b] = toast.colour;
fill(r, g, b, toast.alpha * 0.9);

textSize(20);

//Gets width + padding
const w = textWidth(toast.msg) + 30;
//Draws background
rect(0, 0, w, 60, 12);

//Draws the message
fill(200, toast.alpha);
text(toast.msg, 0, 0);

pop();
}
},
//Easing Functions - Derived from Robert Penner's Easing Functions
//Easeoutback - smoothly overshoots then returns
easeOutBack: (x) => {
  const c1 = 1.70158;
  const c3 = c1 + 1;

```

```

        return 1 + c3 * Math.pow(x - 1, 3) + c1 * Math.pow(x - 1, 2);
    },
    //easeinout - smoothly eases into and then out
    easeInOut: (x) => {
        return x < 0.5 ? 2 * Math.pow(x, 2) : 1 - Math.pow(-2 * x + 2, 2) / 2;
    },
};

//P5.js Functions
//Preload runs before anything
function preload() {
    //Load image file
    spriteSheet = loadImage("/Art/Sprite_Sheet.png");
    //Load audio files
    music = loadSound("/Audio/Music.wav");
    cardSound = loadSound("/Audio/Card.mp3");
    buttonSound = loadSound("/Audio/Button.mp3");
}
//Setup runs after preload
function setup() {
    //Creates canvas
    createCanvas(windowWidth, windowHeight);

    //Sets the distance to centre - for display
    display.centreDist = [width / 3, height / 2.5];

    //Sets base data that'll reign true for all UI
    //Keep pixels
    pixelDensity(2);
    noSmooth();

    //Align things centered

```

```

    textAlign(CENTER, CENTER);
    textFont(`Roboto`); //I do like roboto
    rectMode(CENTER, CENTER);
    imageMode(CENTER, CENTER);

    //Sets up the button and UI layout by default
    layoutUI(windowWidth, windowHeight);
}
//Runs every animation frame the same as running requestAnimationFrame(draw)
function draw() {
    //Music
    //If playing game and music not looping
    if (app.state == STATE.GAME && !looping) {
        //Then loop it
        music.loop();
        looping = true;
    }

    //Draw Background
    background(36, 64, 52);

    //Draw State Specific
    switch (app.state) {
        //Menu screen
        case STATE.MENU:
            drawMenu();
            break;

        //Lobby creation screen
        case STATE.LOBBY_CREATE:
            drawLobbyCreate();
            break;

        //Game play screen

```

```

        case STATE.GAME:
            drawGame();
            break;
    }

    //Draw Buttons
    app.buttons.forEach((btn) => btn.draw());

    //Draw Toast Animations
    anims.drawToasts();
}

//INPUTs
//Mobile touch
function touchStarted() {
    click(mouseX, mouseY);
}

//Desktop mouse press & touch is sent here
function mousePressed() {
    click(mouseX, mouseY);
}

//Handles all type of clicks - mouse and touch
function click(x, y) {
    //If not game just checks if any button is pressed
    if (app.state !== STATE.GAME) return app.buttons.forEach((btn) => btn.click());

    const isAuction = app.data.phase === 1;
    const isTrick = app.data.phase === 2;
    //Auction 'bid' buttons
    if (isAuction) {
        //Loop through buttons
        for (const btn of bid.buttons) {
            //Continues if not hovering over said button

```



```

        if (!isHovering({ x, y }, btn)) continue;
        //Play click sound
        playButtonSound();
        //Checks if said bid is playable
        const currentNum = app.data.currentDeal?.num || 0;
        //If not then show a toast animation to say as such
        if (btn.value <= currentNum) return anims.showToast(`You must bid higher than the current level.`);
        //But if it is then send message to server
        handleInput(`bid(${btn.suit},${btn.value})`);
        return;
    }
}
//Trick 'cards' buttons
if (isTrick) {
    //Loop through cards
    for (const card of clickableCards.reverse()) {
        //Continues if not hovering over said card
        if (!isHovering({ x, y }, card)) continue;
        //Play card sound
        playCardSwipeSound();
        //Send message to server
        handleInput(`trick(${card.suit},${card.pip})`);
        return;
    }
}
//Didn't click one of those so check the buttons
app.buttons.forEach((btn) => btn.click());
}
//p5 windowResize function doesn't like the emulator tool, which I used a lot in testing
//This I also found to be generally more reliable
window.addEventListener(`resize`, () => {
    const [w, h] = [window.innerWidth, window.innerHeight];

```

```

    //Changes canvas size
    resizeCanvas(w, h);
    //Sets centre distance
    display.centreDist = [w / 3, h / 3];
    //Resets the buttons' and UI's position
    layoutUI(w, h);
  });

  //Layout buttons
  function layoutUI(newWidth, newHeight) {
    //Set current deal to null if in score card - useful for next round's auction
    if (app.state == STATE.GAME && app.data.phase == 3) app.data.currentDeal = null;

    //Setup screen and display variables
    display.screen = {
      size: { w: min(1000, width), h: min(1100, height) },
      cx: width / 2,
      cy: min(1100, height) / 2,
    };
    display.centreDist = [display.screen.size.w / 3, display.screen.size.h / 2.5];

    //Reinitialises the apps button list
    app.buttons = [];

    //Sets if the screen is within certain sizing boundaries
    const narrow = newWidth < 680;
    const short = newHeight < 800 && app.state == STATE.LOBBY_CREATE;

    //Calculates the central position
    const cx = newWidth / 2;
    const cy = short ? 400 : newHeight / 2;
  }
}

```

```

switch (app.state) {
  //Create menu buttons
  case STATE.MENU:
    app.buttons.push(
      //Create lobby - creates a new lobby
      new Button(`CREATE LOBBY`, narrow ? cx : cx - 165, cy - 60, 300, 60, () => {
        handleInput(`create`);
      }),
      //Join lobby - joins the lobby with the code you write
      new Button(`JOIN LOBBY`, narrow ? cx : cx + 165, narrow ? cy + 20 : cy - 60, 300, 60, () => {
        const id = prompt(`Enter Lobby Code: `, ``);
        handleInput(`join(${id})`);
      }),
      //Settings - does nothing
      new Button(
        `SETTINGS`,
        cx,
        narrow ? cy + 100 : cy + 20,
        225,
        60,
        () => {
          handleInput();
        },
        [200, 200, 200],
        [150, 150, 150]
      )
    );
    break;
  //Create lobby setup buttons
  case STATE.LOBBY_CREATE:
    app.buttons.push(
      //Start starts a game for a lobby

```

```

new Button(`START`, cx, cy - 60, 300, 60, () => {
    handleInput(`start`);
}),
//Back exits out of a lobby
new Button(
    `BACK`,
    cx,
    cy - 15,
    225,
    50,
    () => {
        handleInput(`leave`);
    },
    [200, 200, 200],
    [150, 150, 150]
)
);
break;
//Create game play buttons
case STATE.GAME:
    if (app.data.phase == 1) {
        //Auction
        const w = display.cardSizeBid[0] * display.cardScaleBid * 7;
        app.buttons.push(
            //Pass
            new Button(
                `PASS`,
                display.screen.cx,
                display.screen.cy + 100,
                w,
                30,
                () => {

```

```

        handleInput(`bid(pass)`);
    },
    [50, 150, 50],
    [25, 100, 25],
    [50, 50, 50]
),
//Double / redouble
new Button(
    `DOUBLE`,
    display.screen.cx,
    display.screen.cy + 140,
    w,
    30,
    () => {
        handleInput(`bid(double)`);
    },
    [150, 50, 50],
    [100, 25, 25],
    [50, 50, 50]
)
);
}
if (app.data.phase == 3) {
    //Score Sheet
    app.buttons.push(
        //Next deal
        new Button(`NEXT DEAL`, display.screen.cx, display.screen.cy + display.centreDist[1],
        300, 60, () => {
            app.data.playable = [app.data.me];
            handleInput(`next`);
        })
    );
};

```

```

        }
        break;
    }
}

//Draw main-menu Screen
function drawMenu() {
    fill(212, 175, 55);

    //Draw Title
    textSize(90);
    text(`BRIDGE`, width / 2, height / 2 - 140);

    //Draw Water Mark
    fill(212, 175, 55);
    textSize(20);
    textAlign(RIGHT, BOTTOM);
    text(`By Alfie W Pearce`, width - 30, height - 10); //30 from far edge for split view overlays
    textAlign(CENTER, CENTER);
}

//Draw lobby set screen
function drawLobbyCreate() {
    fill(212, 175, 55);

    //Draw all members
    textSize(30);
    textAlign(CENTER, TOP);
    text(`MEMBERS:\n${app.data.members.join(`\n`)}`, width / 2, 20);

    //Draw Lobby Id
    let size = 60;

```

```

const txt = `LOBBY: ${app.data.lobbyId.toUpperCase()}`;
textSize(size);
//Decreases size until it is less than 90% of the width - limitting to 10
while (textWidth(txt) > width * 0.9 && size > 10) {
    size -= 2;
    textSize(size);
}
textAlign(CENTER, CENTER);
//Draw said text
text(txt, width / 2, (height < 800 ? 400 : height / 2) - 140);
}

```

Public/drawGame.js

```

/*
    drawGame.js
    Contains the logic to render the game screen
*/

//Helper Functions
const formatContract = (suit, num, dbl) => `${num}${SUITS[suit]}${DOUBLES[dbl] ?? ""}`;

// Draws the Canvas - whilst in the game state
function drawGame() {
    const isMobile = width < 600; //rough breakpoint for portrait screens

    //Sets some display variables based on the size of the screen and allowing for cross-platform displays
    updateDisplay(isMobile);

    //Draws Baize background
    drawBackground();
}

```

```

//Draws the hands
hands.draw(display.screen.cx, display.screen.cy);

//Draws the centre
centre.draw(isMobile, display.screen.cx, display.screen.cy);

//Draws the edges of the table if too large of a monitor
if (width > 1000) table.sides((width - 1000) / 2);
if (height > 1100) table.bottom(height - 1100);

//Draws the header
drawHeader();

//Simplifies if statements
//Game's phase
const isAuction = app.data.phase == 1;
const isTrick = app.data.phase == 2;
const isResults = app.data.phase == 3;
//Viewport size
const narrow = width < 580;

//Draws auction table
if (isAuction) bid.draw(display.screen.cx, display.screen.cy - 50);

//Draws the scoresheet
const scoreSheetSize = display.scoreSheetSize;
const scoreSheetSizeModifier = display.scoreSheetSizeModifier;
//Draws scoresheet during a trick
if (isTrick && !narrow) score.draw(display.screen.cx + display.screen.size.w / 2 - scoreSheetSize[0] *
scoreSheetSizeModifier * 0.5, display.screen.size.h - scoreSheetSize[1] * scoreSheetSizeModifier * 0.5,
scoreSheetSize[0] * scoreSheetSizeModifier, scoreSheetSize[1] * scoreSheetSizeModifier);

```



```

    //Draws scoresheet after a trick
    if (isResults) score.draw(display.screen.cx, display.screen.cy, scoreSheetSize[0], scoreSheetSize[1]);
}
//Updates some display variables to allow for resizing screen and cross-platform
function updateDisplay(isMobile) {
    //Updates size of cards
    display.cardScalePlayer = isMobile ? 3 : 3.5;
    display.cardScaleOpponent = isMobile ? 1.5 : 2;
    display.cardScaleBid = 3;
    //Update angle of fan of card
    display.fanAngle = PI / (isMobile ? 4 : 3);
    //Sets values about the screen eg the centre of the screen
    display.screen = {
        size: { w: min(1000, width), h: min(1100, height) },
        cx: width / 2,
        cy: min(1100, height) / 2,
    };
    //Calculates the distance from the centre to each hand
    const cardH = display.cardSize[1] * display.cardScaleOpponent;
    display.centreDist = isMobile ? [(display.screen.size.w - cardH) / 2, (display.screen.size.h - cardH) / 2 - 22] : [min(250, display.screen.size.w / 3), min(346, display.screen.size.h / 2.5)];
}
//Draws the Baize background pattern
function drawBackground(w = 64, h = 64) {
    //Background is split up into square cells which are looped through
    for (let x = 0; x < width + w / 2; x += w) {
        for (let y = 0; y < height + h / 2; y += h) {
            //Draw the baize pattern in each cell
            image(spriteSheet, x, y, w, h, 416, 160, 32, 32);
        }
    }
}
}

```

```

//Draws the header - which displays data and buttons - eventually
function drawHeader() {
  rectMode(CENTER);
  noStroke();

  //Bottom Border
  fill(70, 40, 0);
  rect(width / 2, 20, width + 20, 105);

  //Main fill
  fill(100, 60, 0);
  rect(width / 2, 20, width + 20, 75);

  //Current Deal
  if (!app.data.currentDeal) return; //Guard clause if no deal currently
  ellipseMode(CENTER);
  textAlign(CENTER);
  textSize(50);

  //Simplifies output code
  const trumpSuit = app.data.currentDeal.trump;
  const contract = formatContract(trumpSuit, app.data.currentDeal.num, app.data.currentDeal.doubled);

  const isAuction = app.data.phase == 0;

  const [x, y] = [width / 2, 10];
  const w = app.data.currentDeal ? max(120, textWidth(contract)) : 120;

  //Cuurent Trick counter background
  fill(200, !isAuction ? 150 : 0);
  rect(x, y + 10, w * 2.5, 75, 20);

```

```

//Current Deal
//Background
fill(120, 80, 0);
ellipse(x, y, w + 30, w + 25);
//Text
fill(0);
text(contract, x, 32);

textAlign(CENTER);

//Currrent Trick Counter
if (app.data.phase < 2) return;
const dist = 50;
const tricksNeeded = app.data.currentDeal.num + 6;

//North & South pair
const nsColour = app.data.me % 2 == 0 ? display.myColour : display.oppColour;
//Heading text
textSize(25);
fill(nsColour);
text(`N-S`, x - w / 2 - dist, 15);
//Scores test
textSize(30);
text(`${app.data.tricksWon[0]}/${app.data.declarer % 2 == 0 ? tricksNeeded : 14 - tricksNeeded}`, x - w / 2 -
(dist / 5) * 4, 40);

//East & West Pair
const ewColour = app.data.me % 2 == 1 ? display.myColour : display.oppColour;
//Heading text
textSize(25);
fill(ewColour);

```

```

text(`E-W`, x + w / 2 + dist, 15);
//Scores test
textSize(30);
text(`${app.data.tricksWon[1]}/${app.data.declarer % 2 == 1 ? tricksNeeded : 14 - tricksNeeded}`, x + w / 2 +
(dist / 5) * 4, 40);

textAlign(CENTER);
}
//Draws table edges
const table = {
  //Side edges - if width is large
  sides: (edgeWidth) => {
    rectMode(CENTER);

    noStroke();
    fill(100, 60, 0);
    rect(0, height / 2, edgeWidth * 2, height); //Left
    rect(width, height / 2, edgeWidth * 2, height); //Right
  },
  //Bottom edge - if height is large
  bottom: (edgeHeight) => {
    rectMode(CENTER);

    noStroke();
    fill(100, 60, 0);
    rect(width / 2, height, width, edgeHeight);
  },
};
//Draws players' hands
const hands = {
  //Main - draws the hands
  draw: (cx, cy) => {

```

```

clickableCards = []; //Array of cards that can be clicked

//List of positions (meta data about each hand's fanning)
const positions = [
  { x: cx, y: cy + display.centreDist[1] - 20, type: `fan`, angle: 0 }, //S
  { x: cx - display.centreDist[0], y: cy, type: `stack`, angle: -HALF_PI }, //W
  { x: cx, y: cy - display.centreDist[1] + 50, type: `stack`, angle: 0 }, //N
  { x: cx + display.centreDist[0], y: cy, type: `stack`, angle: HALF_PI }, //E
];

//Rotates the positions array
const rotated = hands.rotate(positions, app.data.me);

//Loops through the seats
for (let seat = 0; seat < 4; seat++) {
  const pos = rotated[seat];
  if (!pos) continue; //No pos (error probably)
  const hand = hands.find(seat);
  if (app.data.handLengths[seat] == 0) continue; //Empty
  if (!hand || hand == null) continue; //No hand (error probably)

  //Draws main player's hand
  if (pos.type == `fan`) hands.fan(hand, pos);
  //Draws other players' hands
  else hands.stack(seat, hand, pos);
}
},
//Rotates array so every player's is to their south
rotate: (array, shift) => {
  const n = array.length;
  const result = [];
  //Loops through arrays

```

```

    for (let idx = 0; idx < 4; idx++) {
        let shiftedIdx = (idx - shift + 4) % n;
        if (shiftedIdx == -1) shiftedIdx = n - 1;
        result.push(array[shiftedIdx]);
    }
    return result;
},
//Returns with the hand - having had the seat index
find: (seat) => {
    //Player
    if (seat === app.data.me) return app.data.currentHand;
    //Dummy
    const isAuction = app.data.phase == 1;
    if (seat === app.data.dummyPlr && !isAuction) return app.data.dummyHand;
    //All else
    return Array.from({ length: app.data.handLengths[seat] }, () => ({ pip: 0, suit: 0 }));
},
//Draws player hand (south)
fan: (hand, position) => {
    //(Has to be a playable hand - only the players)

    const total = hand.length;

    //Only one card
    if (total == 1) {
        const { x, y, angle } = position;
        const { pip, suit } = hand[0];

        hands.oneCard(0, {
            x,
            y,
            angle,

```

```

        scale: display.cardScalePlayer,
        pip,
        suit,
    });
    return;
}

//Sets values for how the fan will look
const spread = map(total, 1, 13, 0, display.fanAngle); //fan angle
const xRadius = 350; //Curvature Radii
const yRadius = 300;
//Loops through cards
for (let idx = 0; idx < total; idx++) {
    //Gets angle and positions
    const theta = map(idx, 0, total - 1, -spread / 2, spread / 2);
    const x = position.x + sin(theta) * xRadius;
    const y = position.y - cos(theta) * yRadius + yRadius;
    //Creates card
    const c = new Card(hand[idx].pip, hand[idx].suit, true, x, y, position.angle + theta,
        display.cardScalePlayer);
    //Adds card to clickable list
    clickableCards.push(c);
    //Draws card
    c.draw();
}
},
//Draws all other hands
stack: (seat, hand, position) => {
    const total = hand.length;

    //Only one card
    if (total == 1) {

```

```

    const { x, y, angle } = position;
    const { pip, suit } = hand[0];

    hands.oneCard(seat, {
        x,
        y,
        angle,
        scale: display.cardScaleOpponent,
        pip,
        suit,
    });
    return;
}

//Sets values for how the stack will look
const spread = map(total, 1, 13, 0, 300);
//Loops through cards
for (let idx = 0; idx < total; idx++) {
    //Gets position of card
    const delta = map(idx, 0, total - 1, -spread / 2, spread / 2);
    const x = position.x + cos(position.angle) * delta;
    const y = position.y + sin(position.angle) * delta;
    //Creates card
    const c = new Card(hand[idx].pip, hand[idx].suit, hand[idx].pip !== 0, x, y, position.angle,
        display.cardScaleOpponent);
    //If dummy then adds card to clickable list
    if (seat === app.data.dummyPlr && seat === 2) clickableCards.push(c);
    //Draws card
    c.draw();
}
},
//Draws hand - when only one card in said hand

```



```

oneCard: (seat, card) => {
  //Creates new card
  const c = new Card(card.pip, card.suit, card.pip !== 0, card.x, card.y, card.angle, card.scale);
  //If card is for seat 0 (player) or dummy (2 - playable by player) then add to clickable
  if ((seat === app.data.dummyPlr && seat === 2) || seat === 0) clickableCards.push(c);
  //Draw card
  c.draw();
},
};
//Draws the central area of the table
const centre = {
  //Main - draws the centre
  draw: (isMobile, cx, cy) => {
    //Avatar & Names
    centre.avatar(isMobile, cx, cy);

    //Current Trick
    //Stops if there is no trick
    if (!app.data.currentTrick) return;
    //Loops through all of the cards currently played
    for (let idx = 0; idx < app.data.currentTrick.length; idx++) {
      //Displays said card
      centre.trick(idx, cx, cy);
    }
  },
  //Displays the avatar images and name texts and current player
  avatar: (isMobile, cx, cy) => {
    noFill();
    strokeWeight(6);
    //Distance from hands to positions
    const yPos = 120;
    const xPos = 75;

```

```

//Mobile main-direction offset
const offset = isMobile ? 10 : 0;
//Mobile sub-direction offset
const offset2 = offset + isMobile ? 15 : 0;
//Calculates the positions and angles of the 4 players' avatars and names
const avatarPos = [
  { x: cx - display.centreDist[0] / 2 - offset2, y: cy + display.centreDist[1] - yPos + offset, left:
    false, angle: 0 }, //S
  { x: cx - display.centreDist[0] + xPos - offset, y: cy - display.centreDist[1] / 2 - offset2, left:
    true, angle: -PI / 2 }, //W
  { x: cx + display.centreDist[0] / 2 + offset2, y: cy - display.centreDist[1] + yPos - offset, left:
    true, angle: 0 }, //N
  { x: cx + display.centreDist[0] - xPos + offset, y: cy + display.centreDist[1] / 2 + offset2, left:
    true, angle: PI / 2 }, //E
];
//Calculates who the current player is based on this player being south
const relativeCurrentPlayer = (app.data.currentPlayer - app.data.me + 4) % 4;
//Calculates the colours of the 4 avatars and names
const nsColour = app.data.me % 2 == 0 ? display.myColour : display.oppColour;
const ewColour = app.data.me % 2 == 1 ? display.myColour : display.oppColour;

//Loops through each seat
avatarPos.forEach((pos, seat) => {
  //Calculates who they are
  const relativeSeat = (seat + app.data.me + 4) % 4;
  const player = app.data.playerAvatars[relativeSeat];

  //Positions avatar and name
  push();
  translate(pos.x, pos.y);
  rotate(pos.angle); //Rotates for both avatar and name

```

```

//Avatar
//Highlight active player
stroke(seat == relativeCurrentPlayer ? [212, 175, 55] : [0, 0]);
ellipse(0, 0, 30);

//Figures out this seat's colour
const colour =
relativeSeat % 2 == 0 //this pair is 0 or 2 then n/s else e/w
  ? nsColour
  : ewColour;
//Calculates the y position of the avatar (for the tile atlas)
let sy = (player.avatar == 2 ? 3 : player.avatar == 3 ? 2 : player.avatar) * 16;
//Changes positions if the seat is black
if (colour === 0) sy += 4 * 16;
//Draws the avatar
image(spriteSheet, 0, 0, 30, 30, 464, sy, 16, 16);

//Name
noStroke();
fill(colour);
textSize(20);
//Gets name to display
const name = player.name;
//Calculates the central position of the name and thus the x location from the avatar's position
const halfW = textWidth(name) / 2;
const padding = 20;
let x = (pos.left ? -1 : 1) * (halfW + padding);
//Draws name
text(name, x, 0);

pop();
});

```

```

},
//Displays a card in the centre
trick: (idx, cx, cy) => {
  //Figures which card to display
  const card = app.data.currentTrick[idx].card;
  if (!card) return; //Skip empty slots if they exist (shouldn't but just in case)

  //Global index of the player who played this card
  const globalIdx = app.data.currentTrick[idx].player;
  //Relative to the viewer's PoV
  const relativeIdx = (globalIdx - app.data.me + 4) % 4;

  //Calculates the sizing of the cards
  const w = display.cardSize[0] * display.cardScalePlayer;
  const h = display.cardSize[1] * display.cardScalePlayer;
  //Offset by seat
  const ySpacing = h / 2;
  const xSpacing = 0;

  //Base positions - centre of the screen
  let x = cx;
  let y = cy;

  //Calculates final positions based on which seat played it
  switch (relativeIdx) {
    case 0:
      y += ySpacing;
      x += (xSpacing / 3) * 2;
      break; //S
    case 1:
      x -= ySpacing;
      y += (xSpacing / 3) * 2;

```

```

        break; //W
    case 2:
        y -= ySpacing;
        x -= (xSpacing / 3) * 2;
        break; //N
    case 3:
        x += ySpacing;
        y -= (xSpacing / 3) * 2;
        break; //E
    }

    push();
    translate(x, y);
    rotate(relativeIdx * HALF_PI); //90* per seat

    //Calculates where in the tile atlas the card is
    const sx = display.cardSize[0] * (card.pip - 1);
    const sy = display.cardSize[1] * card.suit;
    //Displays said card
    image(spriteSheet, 0, 0, w, h, sx, sy, display.cardSize[0], display.cardSize[1]);

    pop();
},
};
//Draws the auction (bid) table
const bid = {
    //List of buttons for the bid table
    buttons: [],
    //Main - draws the table
    draw: (cx, cy) => {
        stroke(0);
        strokeWeight(1);

```

```

//Background
fill(50);
rect(cx, cy, display.cardSizeBid[0] * display.cardScaleBid * 8, 440, 20);

bid.drawButtons(cx, cy);
bid.drawButtonCover(cx);

bid.drawPastBids(cx, cy + 50); //50 is the offset for the whole table
},
//Display and create the buttons for the different bidding actions
drawButtons: (cx, cy) => {
  //Calculates the top of the buttons (due to padding)
  const top = cy - 180;
  //Calculates the distance between buttons
  const seperation = display.cardSizeBid[0] * display.cardScaleBid * 1.4;
  //Resets button list
  bid.buttons = [];
  //Loops through all suits (0.4)
  for (let idx = 0; idx < 5; idx++) {
    //Calculates the x position of said column
    const x = cx - 2 * seperation + idx * seperation;
    //Loops through all values (1..6)
    for (let yIdx = 0; yIdx < 7; yIdx++) {
      //Calculates size of button
      const w = display.cardSizeBid[0] * display.cardScaleBid;
      const h = display.cardSizeBid[1] * display.cardScaleBid;
      //Calculates y position of said row
      const y = top + yIdx * h;
      //Calcualtes the position of the button in the tile atlas
      const sx = 416 + idx * display.cardSizeBid[0];
      const sy = 48 + yIdx * display.cardSizeBid[1];

```

```

        //Store data for clicking
        bid.buttons.push({ x, y, w, h, suit: suit.toServerSide(idx), value: yIdx + 1 });
        //Draws the button
        image(spriteSheet, x, y, w, h, sx, sy, display.cardSizeBid[0], display.cardSizeBid[1]);
        //Hover Animation
        if (isHovering({ x: mouseX, y: mouseY }, { x, y, w, h })) {
            //if the mouse is hovering the button
            //If can't bid said value then skip it
            if (app.data.currentDeal && app.data.currentDeal.num >= yIdx + 1) continue;
            //Display highlight rectangle
            noStroke();
            fill(50, 150);
            rect(x - 1, y - 1, w + 2, h + 2);
        }
    }
},
//Covering up impossible buttons
drawButtonCover: (cx) => {
    //Checks if there is a current deal
    if (!app.data.currentDeal || app.data.currentDeal.trump === null) return;

    fill(50, 150);
    noStroke();
    //Get position of a rectangle thatt will cover all unplayable buttons
    const w = display.cardSizeBid[0] * display.cardScaleBid * 7.5;
    const h = display.cardSizeBid[1] * display.cardScaleBid * app.data.currentDeal.num;
    //And position
    const y = top + h / 2 - (display.cardSizeBid[1] * display.cardScaleBid) / 2;
    //Display said rectangle
    rect(cx, y, w, h);
}

```

```

},
//Display us to 4 past bids
drawPastBids: (cx, cy) => {
  //Loop through all (up to one each) current bids played this round
  app.data.bid.forEach((bid, idx) => {
    //If there is no bid then end (shouldn't happen but just in case)
    if (!bid) return;
    //Calculates the relative seat
    const relSeat = (idx - app.data.me + 4) % 4;
    //Sets starting positions
    let [x, y] = [cx, cy];
    //Set new position based on what seat it is
    switch (relSeat) {
      case 0:
        y += displaycentreDist[1];
        break; //You
      case 1:
        x -= displaycentreDist[0];
        break; //Left
      case 2:
        y -= displaycentreDist[1] - 50;
        break; //Opposite
      case 3:
        x += displaycentreDist[0];
        break; //Right
    }
    //The text to display
    const txt = typeof bid === `object` ? `${bid.num}${SUITS[bid.trump]}${DOUBLES[bid.doubled]}` :
    `${bid}`;

    textSize(20);
    //Calculates the width of the text to display + padding size

```



```

    const w = textWidth(txt) + 20;

    //Draws background
    strokeWeight(1);
    stroke(0);
    fill(50, 150);
    rect(x, y, w, 30, 10);

    //Draws the text
    noStroke();
    fill(255);
    text(txt, x, y);
  });
},
};
//Draws the scorecard
const score = {
  //Main - draws the scorecard
  draw: (cx, cy, w, h) => {
    //Calculates and sets the size of the text
    const txtSize = h * 0.05;
    textSize(txtSize);

    //Calculates the centre of both columns
    const leftCentre = cx - w / 4;
    const rightCentre = cx + w / 4;

    //displays the background
    score.drawBackground(cx, cy, w, h, { we: leftCentre, they: rightCentre }, txtSize);

    //Sets variables to simplify later code
    const { metadata, rounds } = app.data.score;

```

```

const { gameLines, rubberOver } = metadata;
//Calculates which pair this user is a part of
const pair = app.data.me % 2;
//Calculates what x side they are and their opp is on
const x = pair == 0 ? { we: leftCentre, they: rightCentre } : { they: leftCentre, we: rightCentre };

const totals = score.drawRounds(cx, cy, w, h, x, rounds, gameLines);

//Draw Totals
text(totals.we, x.we, cy + h / 2 - 10);
text(totals.they, x.they, cy + h / 2 - 10);

//Displays end of the Rubber
//Checks if the rubber is over
if (rubberOver) {
  //Draws a RUBBER OVER text
  textSize(txtSize * 2);
  push();
  translate(cx, cy);
  rotate(PI / 4);
  text(`Rubber Over`, 0, 0);
  pop();
}

},
//Displays the background
drawBackground: (cx, cy, w, h, x) => {
  //Draws the background
  noStroke();
  fill(255, 255, 238);
  rect(cx, cy, w, h);

  //Draws the lines

```

```

stroke(56, 59, 62);
strokeWeight(1);
//Draws the we / them seperator
line(cx, cy - h / 2, cx, cy + h / 2);
//Draws the horizontal lines - for above and below the lines
line(cx - w / 2, cy, cx + w / 2, cy);

//Draws the headers 'we' and 'they'
textAlign(CENTER);
noStroke();
fill(56, 59, 62);
text(`we`, x.we, cy - h / 2 + 10);
text(`they`, x.they, cy - h / 2 + 10);
},
//Displays the numbers for each round - points above and below the line as well as penalties and totals
drawRounds(cx, cy, w, h, x, rounds, gameLines, totals = [0, 0]) {
  //Sets default positions
  let yUnder = cy;
  let yOver = { we: cy, they: cy };
  let yPenalty = { we: cy - h / 2 + 20, they: cy - h / 2 + 20 };

  fill(56, 59, 62);
  //Loops through each round
  rounds.forEach((round, idx) => {
    //Sets which pair the declarer is on and which they are against
    const declarer = round.declarer;
    const side = declarer % 2;
    const opp = 1 - side;

    //Contract made - add points to declarer's side
    if (round.contractMade) {
      //Points under

```

```

//Iterates position
yUnder += 16;
noStroke();
//Draws the number of points under
text(round.pointsUnder, x[declarer == 0 ? `we` : `they`], yUnder);
//Adds to totals
totals[side] += round.pointsUnder;

//Draws game lines - if one exists here
if (gameLines.includes(idx + 1)) {
  stroke(56, 59, 62);
  strokeWeight(2);
  line(cx - w / 2, yUnder + 10, cx + w / 2, yUnder + 10);
  //Iterates position further
  yUnder += 20;
}

//Draws points above the line
if (round.pointsAbove > 0) {
  //Iterates position
  yOver[declarer == 0 ? `we` : `they`] -= 16;
  noStroke();
  text(round.pointsAbove, x[declarer == 0 ? `we` : `they`], yOver[declarer == 0 ? `we` :
`they`]);
  //Adds to totals
  totals[side] += round.pointsAbove;
}

//Contract not made - add penalty points to other side
} else {
  //Penalty points
  yPenalty[opp == 0 ? `we` : `they`] += 16;

```

```

        noStroke();
        text(round.pointsPenalty, x[opp == 0 ? `we` : `they`], yPenalty[declarer == 0 ? `we` :
`they`]);
        //Adds to totals
        totals[opp] += round.pointsPenalty;
    }
});
//Return totals to be displayed
return totals;
},
};

```

Public/buttons.js

```

/*
    buttons.js
    Contains the class to create and use buttons
*/

//All buttons will be a button instance
class Button {
    constructor(label, x, y, w, h, onClick, colour = [212, 175, 55], hover = [181, 146, 31], textColour = [36, 64,
52]) {
        //Sets basic values about location, colour, and functions
        Object.assign(this, { label, x, y, w, h, onClick, colour, hover, textColour });
        //Stores the state of animations
        this.anim = 0;
        this.scale = 1;
    }

    //Changes the current state of animations

```

```

update() {
  const hovered = this.isHovered();
  this.anim = lerp(this.anim, hovered ? 1 : 0, 0.1);
  this.scale = 1 + this.anim * 0.08;
}

//Draws the button
draw() {
  this.update();

  //Moves button to the class's position and scale
  rectMode(CENTER);
  textAlign(CENTER);
  push();
  translate(this.x, this.y);
  scale(this.scale);

  noStroke();

  //Background fade
  fill(lerpColor(color(...this.colour), color(...this.hover), this.anim));
  //Draws the background
  rect(0, 0, this.w, this.h, 10);
  //Sets to text colour
  fill(this.textColour);
  //Sets size of text based on size of button
  textSize(min(this.w, this.h) * 0.55);
  //Draws the text
  text(this.label, 0, 0);

  pop();
}

```

```

//Returns if the mouse is within the button's bounds
isHovered() {
    //Calculates half the size so to calculate the buttons bounding box
    const halfW = (this.w * this.scale) / 2;
    const halfH = (this.h * this.scale) / 2;
    //Returns if the mouse is within said bounding box
    return mouseX > this.x - halfW && mouseX < this.x + halfW && mouseY > this.y - halfH && mouseY < this.y +
    halfH;
}
//Handles a mouse click
click() {
    //Checks if the mouse is hovering over this button and there is a click function
    if (this.isHovered() && this.onClick) {
        //Makes small
        this.scale = 0.95;
        //Makes normal size after 100ms
        setTimeout(() => (this.scale = 1.08), 100);
        //Plays a noise
        playButtonSound();
        //Runs said function
        this.onClick();
    }
}
}

Public/cards.js
/*
    cards.js
    Contains the class which displays a card
*/

```

```

//Pixel size of card art
const cardSize = [32, 48];

//Every card is an instance of this class
class Card {
  constructor(pip, suit, faceUp, x, y, angle, scale) {
    //Sets basic values about the card and its visibility
    Object.assign(this, { pip, suit, faceUp, x, y, angle, scale });

    //Calculates the initial size of the card
    this.w = cardSize[0] * this.scale;
    this.h = cardSize[1] * this.scale;
  }
  //Draws the card
  draw() {
    push();
    translate(this.x, this.y);
    rotate(this.angle);

    //Calculates where the card is in the tile atlas
    const sx = this.faceUp ? (this.pip - 1) * cardSize[0] : 13 * cardSize[0];
    const sy = this.faceUp ? this.suit * cardSize[1] : 0;

    //Draws the image (the card)
    image(spriteSheet, 0, 0, this.w, this.h, sx, sy, cardSize[0], cardSize[1]);

    pop();
  }
}

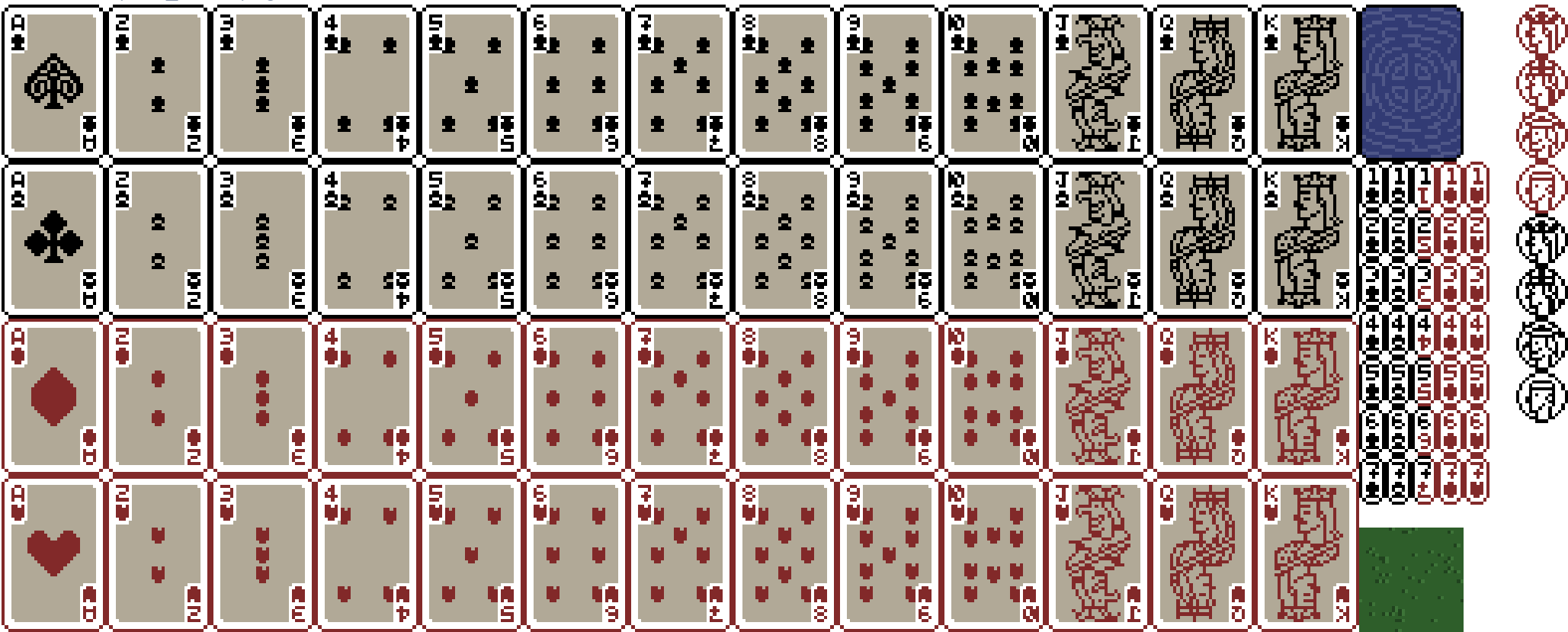
```


Public/Art/ - GFX for client-side

Public/Art/favicon.png



Public/Art/Sprite_Sheet.png



Public/Audio/ - SFX for client-side {All of the following will be heard in the video}

Public/Audio/Music.wav

Public/Audio/Card.mp3

Public/Audio/Button.mp3